# 데이터 수집/처리

SDS

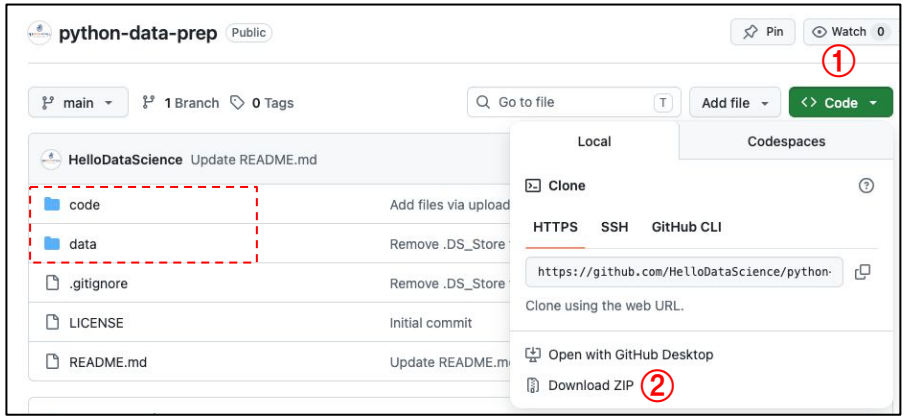
© 2026 HelloDataScience. All rights reserved.

# 강의 내용

- 웹 스크레이핑의 이해
- HTTP 통신 기초
- HTML 요소 기초
- 크롬 개발자 도구 활용법
- 웹 스크레이핑 실습 1~4
- Open API 활용법
- Selenium 활용법
- 데이터 입출력
- 정형 데이터 전처리
- 비정형 데이터 전처리
- 문서 간 유사도

# 실습 데이터셋 준비

- 이번 강의에서 사용할 실습 파일을 내려받을 수 있는 GitHub 저장소 링크를 안내합니다.
  - 링크: https://github.com/HelloDataScience/python-data-prep




*① 링크에 접속하고 초록색 Code 버튼을 클릭합니다.*

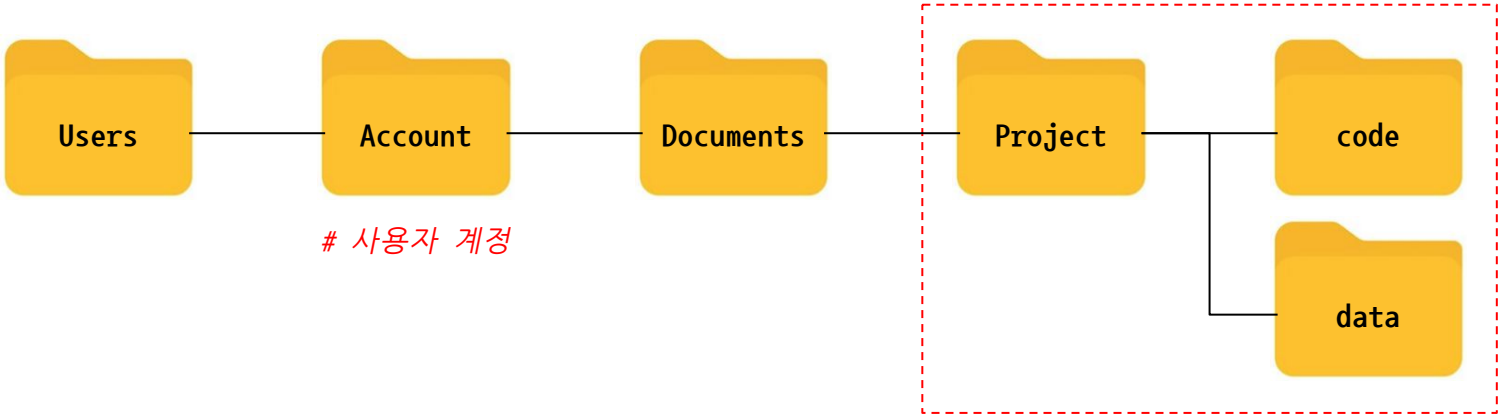
*② Download ZIP을 클릭하여 압축 파일을 Downloads 폴더로 내려받습니다.*

*③ 탐색기를 열고 Downloads 폴더에서 압축 파일을 확인하고, 압축 파일을 해제합니다.*

*# [참고] code 폴더에는 완성된 Jupyter Notebook 파일, data 폴더에는 전처리 실습 파일이 포함되어 있습니다.*

# 프로젝트 폴더 생성

- 이번 강의용 프로젝트 폴더를 다음 순서에 따라 생성합니다.
  - 탐색기를 열고 C: → Users → Account → Documents 폴더로 이동합니다.
  - Documents 폴더 안에 Project 폴더를 생성합니다.
  - Project 폴더 안에 code 파일과 data 파일을 저장할 폴더를 각각 생성합니다.

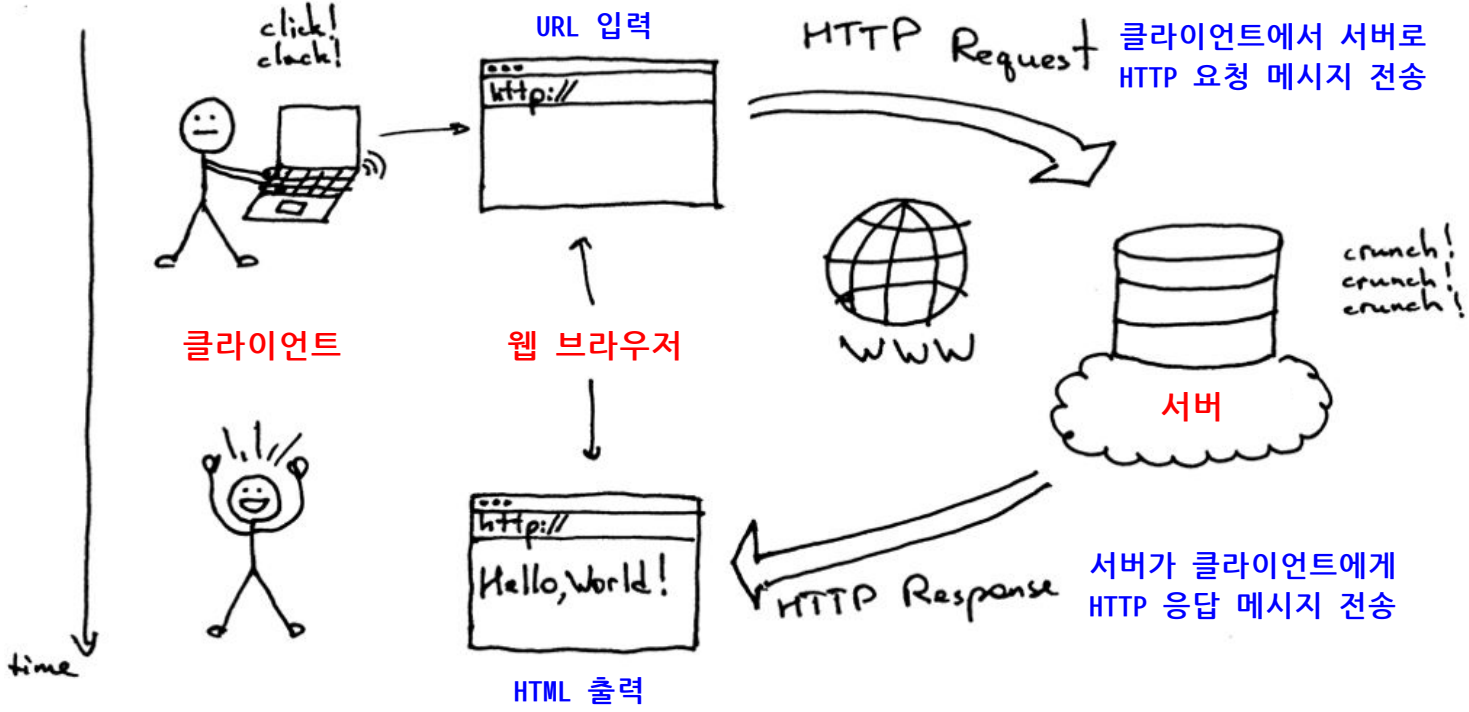


# 웹 스크레이핑의 이해

# 웹 스크레이핑 vs 웹 크롤링

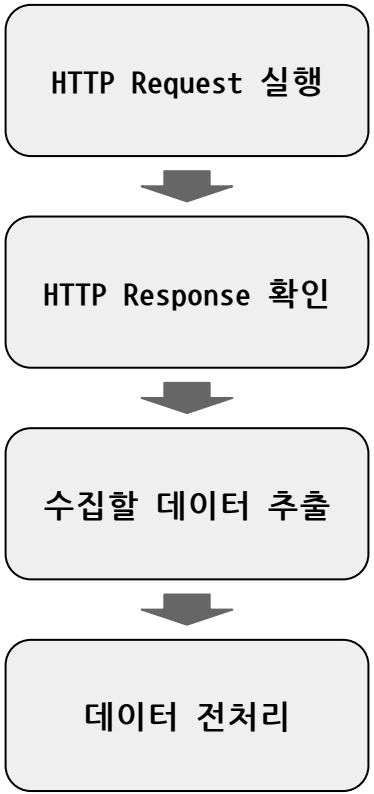
- 웹 스크레이핑Web Scraping은 웹 페이지에서 수집하려는 정보를 추출하여 저장하는 작업입니다.
  - 수집할 데이터가 여러 페이지에 걸쳐 있을 경우, 먼저 각 페이지의 링크를 수집한 뒤에 이를 순차적으로 방문하며 스크레이핑 작업을 수행합니다.
- 웹 크롤링Web Crawling은 검색 엔진이 웹 페이지를 자동으로 탐색하며 새로운 링크를 수집하거나, 기존에 수집했던 링크를 업데이트하는 작업을 의미합니다.
  - 웹 크롤링을 수행하는 프로그램을 웹 크롤러Web Crawler라고 부릅니다.
- 웹 페이지마다 HTTP 통신 방식이 다를 수 있으므로 웹 크롤러를 개발할 때, 해당 웹 페이지에 적합한 접근 방식을 적용해야 합니다.
  - 크롬 개발자 도구에서 웹 크롤러 개발에 필요한 리소스를 확인할 수 있습니다.

# [참고] 인터넷에서 정보를 검색하는 방법




*# 출처: https://blogs.sap.com/2016/02/08/odata-everything-that-you-need-to-know-part-1/*

# 웹 스크레이핑 프로세스




- 정적인 웹 페이지의 요청 방식을 찾아 실행합니다.
- JavaScript를 사용하는 웹 페이지의 요청 방식을 찾아 실행합니다.
- Selenium을 사용하여 웹 페이지의 바디 문자열을 획득합니다.

- 응답 상태 코드를 확인합니다.
- 응답 헤더에서 콘텐트 유형과 문자 인코딩 방식을 확인합니다.

- 바디 문자열이 HTML이면 관련 객체로 변환하고 CSS 선택자 또는 XPATH로 수집할 텍스트를 포함하는 HTML 요소를 선택합니다.
- 바디 문자열이 XML 또는 JSON이면 딕셔너리로 변환합니다.

- 수집한 텍스트 데이터를 다양한 방식으로 전처리합니다.
- 최종 객체를 다양한 형태의 파일로 저장합니다.

**requests**
**selenium**

**bs4**
**xmltodict**
**json**

**pandas**
**openpyxl**

# 추가로 알아야 할 사항

**크롬 개발자 도구 활용법**

- 수집하려는 텍스트를 포함하는 리소스를 찾습니다.
- HTTP 요청에 필요한 요청 방식, Base URL 및 쿼리 문자열 등을 확인합니다.

**다양한 에러 해결법**

- HTTP 요청에 헤더를 추가하지 않으면 에러가 발생할 수 있습니다.
- HTTP 요청에 필요한 헤더를 확인합니다.(user-agent, referer 등)

**문자 인코딩과 로케일**

- 응답 바디의 문자 인코딩 방식에 따라 사용할 함수와 인수가 달라집니다.
- 로케일은 사용자 인터페이스에서 출력하는 형식을 지정한 문자열입니다.

**정규 표현식**

- 정규 표현식은 특정 규칙을 갖는 문자열의 집합을 표현할 때 사용합니다.
- 정규 표현식을 활용하면 텍스트를 쉽고 빠르게 전처리할 수 있습니다.

# 웹 스크레이핑 관련 주의사항

- 웹 페이지의 콘텐츠를 무단으로 수집하거나 사용하면 저작권의 침해가 될 수 있고, 크롤러의 과도한 접속으로 인해 웹 사이트 서비스가 방해를 받으면 업무방해죄가 성립할 수 있습니다.
  - 사람인HR: https://zdnet.co.kr/view/?no=20170927180839
  - 여기어때: https://www.etnews.com/20200211000365
- 웹 스크레이핑을 실행하기 전에 반드시 다음 사항을 확인하시기 바랍니다.
  - robots.txt에서 허용 범위를 확인하고, 사이트 관리자에게 사전 승인을 받습니다.
  - 오픈 API 및 공개적으로 접근 가능한 데이터를 제한적으로 수집합니다.(타임 아웃 설정)
  - 변호사나 법무팀을 통해 법률 검토를 진행하는 것이 좋습니다.

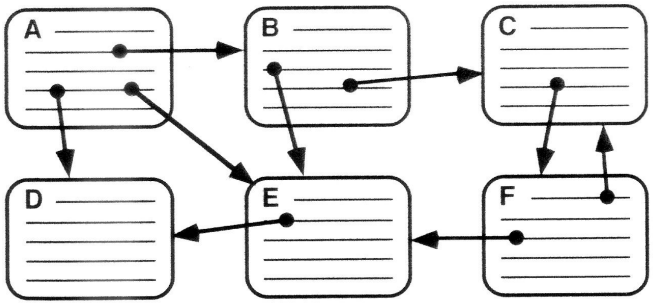
# HTTP 통신 기초

# HTTP HyperText Transfer Protocol

- HTTP는 초본문 전송 규약입니다.
  - 초본문은 인터넷에서 주고받는 문서의 형태를 의미합니다.
- 인터넷에서 데이터를 주고받는 주체는 클라이언트Client와 서버Server입니다.
  - 클라이언트는 서버에 HTTP 요청을 전송하며, 서버는 HTTP 요청에 대해 응답합니다.
- 웹 스크레이핑 과정에서는 주로 GET(조회)과 POST(저장) 요청 메서드를 사용합니다.
  - API에서는 PUT(전체 수정), PATCH(부분 수정), DELETE(삭제) 메서드도 사용됩니다.
  - 웹 크롤러를 개발할 때는 반드시 웹 페이지의 요청 방식을 확인해야 합니다.
  - 크롬 개발자 도구의 네트워크 탭에서 요청 방식을 확인할 수 있습니다.

# [참고] 초본문Hypertext이란?

- 초본문은 참조hyperlink를 통해 특정 지점에서 다른 지점으로 즉시 접근할 수 있는 텍스트를 의미합니다.[출처: 위키백과]
  - 일반 문서는 처음부터 순서대로 내용을 열람해야 합니다.
  - 초본문은 순서에 상관없이 링크를 클릭하면 관련 내용으로 즉시 이동할 수 있습니다.
  - 웹 페이지를 읽는 중에 마우스로 링크를 클릭하면 해당 링크가 연결된 페이지로 바로 이동할 수 있습니다.




*# 출처: http://www.isko.org/cyclo/hypertext*

# [참고] HTTP 요청 방식의 차이: GET vs POST

| 구분 | GET 방식 | POST 방식 |
|---|---|---|
| 데이터 보안성 | - 요청 데이터(쿼리 문자열)가 URL에 포함되어 브라우저 기록, 로그 등에 노출될 수 있습니다.<br>- 민감한 정보를 포함하는 요청에 부적합합니다. | - 요청 데이터가 HTTP 바디에 포함되어 URL에는 직접 노출되지 않습니다.<br>- HTTP 환경에서는 바디가 암호화되지 않으므로 민감 정보는 HTTPS 환경에서 전송해야 합니다. |
| 데이터 크기 | - URL 길이에 제한이 있습니다.<br>- 대량 데이터 전송에는 부적합합니다. | - 요청 데이터의 크기에 제한이 없습니다.<br>- 대량의 데이터 전송에 적합합니다. |
| 요청 목적 | - 데이터 조회에 주로 사용됩니다.<br>- 원칙적으로 서버 상태를 변경하지 않습니다.<br>- 브라우저 및 프록시 서버에서 캐싱될 수 있어 동일 요청을 반복할 수 있습니다. | - 새 게시물 작성, 파일 업로드 또는 로그인 등 데이터 생성 및 수정 등에 사용됩니다.<br>- 서버 상태가 변경될 수 있으므로 동일 요청을 반복하면 중복 처리될 수 있습니다. |
| 사용 사례 | - 검색 요청<br>- 페이지 번호 전달<br>- URL을 즐겨찾기로 저장 | - 로그인 정보 전송(ID, PW)<br>- 사용자 데이터 입력(회원가입, 설문조사 등)<br>- 파일 업로드 |

# HTTP 요청

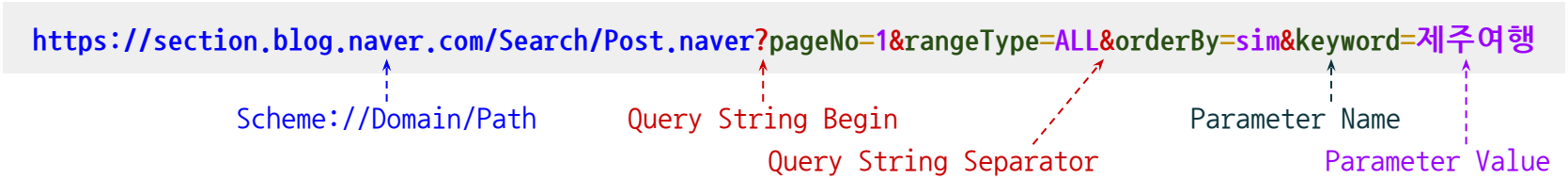
- 클라이언트는 HTTP 요청 방식에 적합한 요청 메시지를 서버에 전송해야 합니다.
- GET 방식의 요청 메시지는 요청 라인과 요청 헤더로 구성됩니다.
    - 요청 라인은 필수이고, 요청 헤더를 생략하면 기본값을 전송합니다.
- POST 방식은 메시지 바디를 추가합니다.
    - 메시지 바디는 클라이언트가 서버로 전송하려는 요청 데이터입니다.

| 구분 | 세부 항목 |
|---|---|
| **요청 라인** | - 요청 방식(GET, POST)<br>- 요청 경로(Base URL과 쿼리 문자열)<br>- HTTP 버전 등 |
| **요청 헤더** | - 인증 스펙(authorization)<br>- 원하는 데이터 형식(accept)<br>- 가능한 인코딩 방식(accept-encoding)<br>- 가능한 콘텐트 유형(content-type)<br>- 브라우저 쿠키(cookies)<br>- HTTP 리퍼러(referer)<br>- 사용자 에이전트(user-agent) 등 |
| **메시지 바디** | - 파라미터(길이 제한 없음) |

# URL의 구조

- URL[Uniform Resource Locator]은 웹 상의 특정 리소스를 가리키는 주소로, 일반적으로 Base URL과 쿼리 문자열로 구성됩니다. 이때 Base URL은 Scheme, Domain 및 Path를 결합한 부분입니다.
  - Scheme: 리소스에 접근하는 방식을 정의합니다.(예: http, https, ftp, mailto 등)
  - Domain: 리소스가 위치한 서버의 주소를 나타냅니다.
  - Path: 서버 내에서 리소스의 정확한 위치(경로)를 지정합니다.
  - Query String: 리소스에 대한 추가적인 정보를 전달하는 파라미터 문자열입니다.

[네이버 블로그에서 '제주여행'으로 검색한 웹 페이지 주소]



# HTTP 응답

- 서버는 클라이언트에게 헤더와 바디를 포함한 응답 메시지를 전송합니다.
- 응답 헤더는 클라이언트가 받은 응답에 대한 정보를 담고 있습니다.
    - HTTP 버전, 상태 코드, 요청 일시, 콘텐트 유형, 문자 인코딩 방식, 본문 크기 등을 포함합니다.
- 응답 바디(본문)는 HTML, XML 또는 JSON 형태의 문자열입니다.

| 구분 | 상세 내용 |
|---|---|
| 1XX | 정보 교환<br>(요청 처리가 계속 진행 중임을 표시) |
| 2XX | 데이터 전송 성공 또는 수락됨<br>(클라이언트의 요청이 성공적으로 처리됨) |
| 3XX | 방향 바꿈 / 서버 측 리다이렉트<br>(요청한 리소스가 다른 위치로 이동했음) |
| 4XX | 클라이언트 측 문제로 요청이 실패했음<br>(잘못된 요청, 권한 없음, 접근 금지 등) |
| 5XX | 서버가 올바른 요청을 처리하지 못했음<br>(내부 서버 오류, 서비스 불가 등) |

# HTTP 요청 관련 코드

- 필요한 모듈을 임포트합니다.

```
>>> import requests  # [참고] HTTP 요청과 응답 관련 작업을 실행할 때 requests 라이브러리를 사용합니다.
```

- GET 방식으로 HTTP 요청을 실행하고 res에 할당합니다.

```
>>> res = requests.get(url=url, params=query, headers=headers)

# [참고] get 메서드의 url 매개변수에 Base URL을 문자열로, params 매개변수와 headers 매개변수에 쿼리 문자열과
  요청 헤더를 각각 딕셔너리를 지정합니다.
```

- POST 방식으로 HTTP 요청을 실행하고 res에 할당합니다.

```
>>> res = requests.post(url=url, json=data, headers=headers)

# [참고] post 메서드의 json 매개변수에 메시지 바디를 딕셔너리로 지정합니다.
```

# HTTP 응답 관련 코드

- HTTP 응답 상태 코드를 확인합니다.

```
>>> res.status_code
```

- HTTP 응답 헤더에서 콘텐트 유형을 확인합니다.

```
>>> res.headers['content-type'] # [참고] 응답 헤더의 자료형은 딕셔너리입니다.
```

- HTTP 응답 바디 문자열을 출력합니다.

```
>>> print(res.text) # [참고] 응답 바디 문자열의 자료형은 str입니다.
                    # res.content 속성은 응답 바디의 원본 바이트 데이터(bytes)를 반환합니다.
```

- HTTP 요청 URL을 출력합니다.

```
>>> print(res.url) # [참고] 수집하려는 텍스트가 응답 바디 문자열에 없으면 콘솔에 출력된 요청 URL을 클릭하여
                   # 웹 브라우저에서 제대로 열리는지 확인합니다. 만약 제대로 열리면 요청 코드를 수정합니다.
```

# HTML 요소 기초

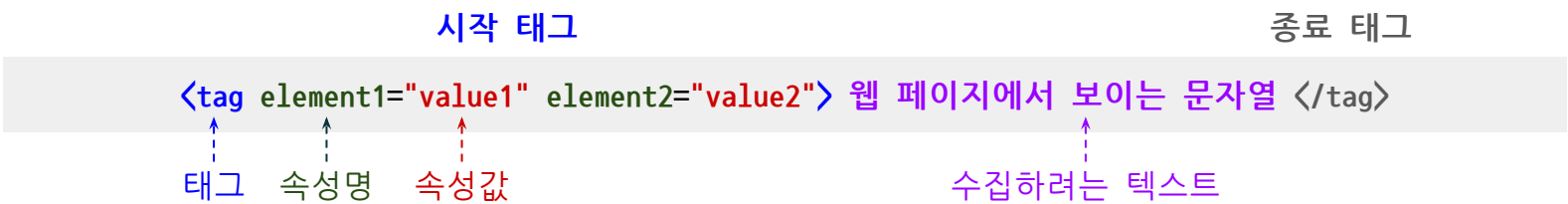
# HTML HyperText Markup Language

- HTML은 웹 페이지의 구조와 내용을 정의하는 마크업 언어입니다.
  - 제목, 단락, 목록 등을 표현합니다.
  - 홑화살괄호 안에 태그와 속성을 포함한 요소element들로 웹 페이지를 구성합니다.
- CSS 선택자는 HTML 요소를 선택하여 스타일을 적용하는 규칙입니다.
- JavaScript는 웹 페이지에서 동적 및 대화형 동작을 가능하게 하는 프로그래밍 언어입니다.



# HTML 요소

- HTML 요소는 HTML 문서 및 웹 페이지를 구성하는 개별 항목입니다.
  - HTML 요소는 시작 태그와 종료 태그 사이에 텍스트 또는 기타 콘텐츠를 포함합니다.
  - 시작 태그와 종료 태그는 홑화살괄호로 감싸줍니다.
  - 시작 태그에 속성명과 속성값을 포함할 수 있습니다.
  - 속성은 태그의 동작이나 스타일을 정의하는 데 사용됩니다.
  - 종료 태그는 시작 태그와 동일한 구조이지만, 앞에 슬래시(/)를 추가하여 구분합니다.



# HTML 주요 태그 소개 #1

| 태그 | 기능 |
|---|---|
| <body> | 웹 페이지의 실제 내용을 담는 영역 |
| <h1> ~ <h6> | 문서 또는 섹션의 제목을 표현 |
| <hr>, <br> | 수평선 삽입, 강제 줄바꿈 |
|  | 여러 HTML 요소를 그룹으로 묶음 |
|  | 한 줄에서 공간 분할(줄바꿈 아님) |
| <ol>, <ul> | 순서형 목록, 순서 없는 목록 표현 |
| <li> | <ol>, <ul>에서 개별 항목을 정의 |
| <a> | 하이퍼링크 생성(href 속성 사용) |
| <img> | 이미지 삽입(src 속성 사용) |

| 태그 | 기능 |
|---|---|
| <p> | 텍스트 단락 표현(위 아래 여백 추가) |
| <table> | 데이터를 행과 열로 구성된 표로 표현 |
| <thead> | 표의 열 이름 영역 정의 |
| <tbody> | 표의 본문 영역 정의 |
| <tr> | 표의 행row을 정의 |
| <th> | <tr>에서 열 이름header 표현 |
| <td> | <tr>에서 데이터data 표현 |
| <input> | 폼form에서 데이터 입력 받음 |
| <iframe> | 현재 문서에 다른 HTML 문서 삽입 |

# HTML 주요 태그 소개 #2

| 태그 | 기능 |
|---|---|
| <b> | 글자 굵게 |
|  | 글자 굵게(의미론적 강조) |
| <i> | 글자 기울이기 |
|  | 글자 기울이기(의미론적 강조) |
| <mark> | 글자에 배경색 추가 |
| <small> | 글자 작게 |
| <big> | 글자 크게 |

| 태그 | 기능 |
|---|---|
| <del> | 취소선 추가(삭제된 글자) |
| <ins> | 밑줄 추가(삽입된 글자) |
| <sub> | 아랫첨자로 표시 |
| <sup> | 윗첨자로 표시 |
| <q> | 따옴표 추가(인용된quoted 글자) |
| <blockquote> | 들여쓰기로 표시(인용된 글자 덩어리) |
| <address> | 글자 기울이기(연락처 정보) |

*# HTML 요소 참고: https://developer.mozilla.org/ko/docs/Web/HTML/Element*

# [참고] HTML 요소 예시

```
<html lang="ko" data-useragent="Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/118.
0.0.0 Safari/537.36">
▶<head>…</head>
▼<body onload="getGNB();">
   <script type="text/javascript"> var nclk_evt = 3; nclk_do(); </script>
  ▶<script type="text/javascript">…</script>
  ▶<div id="u_skip">…</div>
  ▼<div id="header">
    ▼<div class="header_area">
      ▼<div class="header_inner">
        ▼<div class="logo_area">
          ▼<h1 class="service">
            ▶<a href="https://www.naver.com/" class="logo_link" onclick="clickcr(this, 'STA.naver', '', '', event);">…</a>
            ▶<a href="https://new-m.pay.naver.com/" class="logo_link" onclick="clickcr(this, 'STA.pay', '', '', event);">…</a>
            ▶<a href="/" class="logo_link" onclick="clickcr(this, 'STA.finance', '', '', event);">…</a>
           </h1>
         </div>
        ▼<div class="right_area">
          ▶<div class="search_area">…</div>
          ▶<div class="gnb_area">…</div>
         </div>
       </div>
```

*# [참고] 네이버 증권 국내증시 페이지를 구성하는 HTML의 일부입니다.*

# CSS 선택자 vs XPath

- CSS는 계단식 스타일 시트로 HTML 페이지의 스타일(디자인)을 담당합니다.
  - HTML 요소를 스타일 기반으로 식별할 때 CSS 선택자selector를 사용합니다.
  - CSS 선택자는 간단하고 가독성이 좋아 사용하기 편리합니다.
  - Python에서 bs4.BeautifulSoup 객체의 HTML 요소를 선택할 때 CSS 선택자만 사용합니다.
- XPath는 XML 경로 언어로 XML 문서에서 요소를 식별할 때 사용합니다.
  - 계층 구조를 갖는 XML 문서에서 노드와 속성을 탐색하는 데 유용합니다.
  - Selenium에서는 XPath가 CSS 선택자보다 속도가 빠르다는 의견도 있습니다.
  - R에서는 HTML 요소를 선택할 때 CSS 선택자와 XPath로 모두 사용할 수 있습니다.

# CSS 선택자 표기법

| 표현식 | 의미 | 예시 코드 |
|---|---|---|
| '태그' | 태그로 선택 | >>> soup.select(selector='a') |
| '태그1 > 태그2' | 태그의 부모 자식 관계로 선택 | >>> soup.select(selector='ul > div > p > a') |
| '[속성명]' | 속성명으로 선택 | >>> soup.select(selector='[title]') |
| '[속성명="속성값"]' | 속성명과 속성값 관계로 선택 | >>> soup.select(selector='[title="asian"]') |
| '#속성값' | id의 속성값으로 선택 | >>> soup.select(selector='#lang') |
| '.속성값' | class의 속성값으로 선택 | >>> soup.select(selector='.site') |
| ':nth-child(n)' | n번째 자식 태그 선택 | >>> soup.select(selector='a:nth-child(n)') |

*# [참고] 속성명 지정 코드의 대괄호에서 등호 앞에 속성값의 정규 표현식의 위치 지정자(^, $)를 사용할 수 있습니다.*
*예를 들어 '[title^="h"]'은 속성값이 'h'로 시작하는 조건, '[title$="r"]'은 속성값이 'r'로 끝나는 조건을 의미합니다.*

*[주의] 'a:nth-child(2)'는 부모 요소의 두 번째 자식이면서 a 태그를 선택합니다.*
*'a:nth-of-type(2)'는 부모 요소에서 두 번째 a 태그를 선택합니다. 이때 a 태그는 전체에서 두 번째가 아니어도 됩니다.*

# CSS 선택자 표기법(계속)

| 표현식 | 의미 | 예시 코드 |
|---|---|---|
| '태그1 태그2' | 조상 자손 관계로 선택(공백 필수!) | >>> soup.select(selector='ul a') |
| '태그1, 태그2' | 태그1과 태그2를 or 조건으로 선택 | >>> soup.select(selector='p, a') |
| '태그1 + 태그2' | 태그1의 바로 다음 형제인 태그2 선택 | >>> soup.select(selector='p + a') |
| '태그1 ~ 태그2' | 태그1과 형제 관계인 모든 태그2 선택 | >>> soup.select(selector='p ~ a') |
| ':first-child' | 부모의 첫 번째 자식 태그 선택 | >>> soup.select(selector='p:first-child') |
| ':last-child' | 부모의 마지막 자식 태그 선택 | >>> soup.select(selector='p:last-child') |
| ':not(태그)' | 특정 태그 제외한 모든 요소 선택 | >>> soup.select(selector=':not(p)') |

*# [참고] https://hds.ai.kr/css-selector-demo*

# HTML 데이터 처리 예제 코드

- bs4 라이브러리의 BeautifulSoup 클래스를 임포트합니다.

```
>>> from bs4 import BeautifulSoup as BTS
```
*# [참고] 문자열을 HTML로 변환하고 HTML 요소를 선택할 때 bs4 라이브러리를 사용합니다.*

- HTTP 응답 바디 문자열을 BeautifulSoup 객체로 파싱하고 soup에 할당합니다.

```
>>> soup = BTS(markup=res.text, features='html.parser')
```
*# [참고] features 매개변수에 parser를 문자열로 지정합니다.(기본값: None)*

- soup을 확인합니다.

```
>>> soup
```
*# [참고] soup은 bs4.BeautifulSoup 클래스로 생성된 인스턴스입니다. soup 자체로는 HTML의 트리 구조를 확인하기 어렵습니다.*

- soup을 HTML의 계층 구조(들여쓰기)를 반영한 문자열로 변환하여 출력합니다.

```
>>> print(soup.prettify())
```
*# [참고] prettify 메서드는 BeautifulSoup 객체를 HTML의 계층 구조에 맞게 포매팅하여 가독성 좋은 문자열로 반환합니다.*

# HTML 데이터 처리 예제 코드(계속)

- 수집할 텍스트를 포함하는 HTML 요소를 모두 선택하고 items에 할당합니다.

```
>>> items = soup.select(selector='CSS 선택자') # [참고] items는 bs4.element.ResultSet 클래스로 생성된 인스턴스이며, 리스트처럼 동작합니다.
```

- items의 첫 번째 원소를 문자열로 변환하여 태그의 구조를 확인합니다.

```
>>> items[0].prettify() # [참고] items[0]의 자료형은 bs4.element.Tag입니다.
```

- items의 첫 번째 원소에서 문자열을 추출합니다.

```
>>> items[0].text
```

- items의 첫 번째 원소에서 'href'(속성명)에 연결된 값(문자열)을 반환합니다.

```
>>> items[0]['href']
```

# [참고] HTML 요소 선택 메서드 비교

| 구분 | 상세 내용 | 예시 코드 |
|---|---|---|
| select(),<br>select_one() | - CSS 선택자를 사용하여 요소 탐색<br>- CSS 선택자 문법 지원(직계 관계, #/. 기호 등)<br>- 문자열만 선택할 수 없음<br>- select 메서드는 모든 요소 선택<br>- select_one 메서드는 첫 번째 요소만 선택 | >>> soup.select(selector='CSS 선택자') |
| find_all(),<br>find() | - 태그와 속성명으로 요소 탐색<br>- CSS 선택자 문법을 지원하지 않음<br>- string 매개변수에 True를 지정하면 문자열만 선택할 수 있음<br>- find_all 메서드는 모든 요소 선택<br>- find 메서드는 첫 번째 요소만 선택 | >>> soup.find_all(name='tag_name',<br>id='id_name',<br>class_='class_name',<br>string=None) |

# 크롬 개발자 도구 활용법

네트워크 및 요소 탭

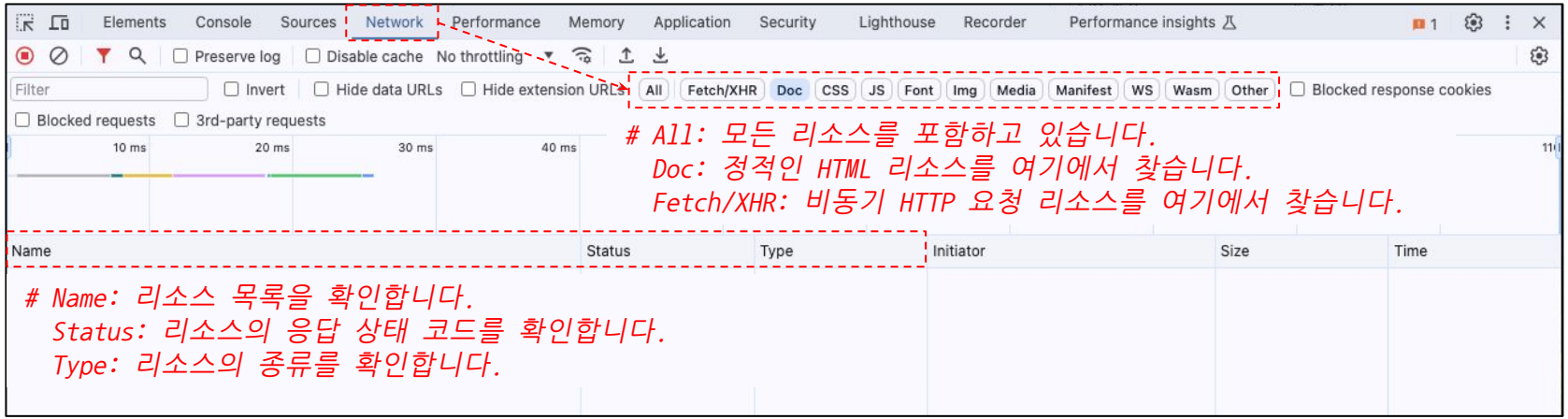
# 크롬 개발자 도구를 사용하는 이유

- 크롬 개발자 도구는 다음 두 가지 문제를 해결하는 데 유용합니다.
  - 네트워크Network 탭: HTTP 요청 및 응답 리소스를 찾을 수 있습니다.
  - 요소Elements 탭: 수집하려는 텍스트를 포함한 HTML 요소를 찾을 수 있습니다.
- 크롬 개발자 도구를 여는 방법은 다음과 같습니다.
  - 방법 1: 크롬 브라우저 오른쪽 상단에 있는 삼점 메뉴( ⋮ ) 클릭 → 도구 더보기More Tools → 개발자 도구Developer Tools를 차례대로 선택합니다.
  - 방법 2: 크롬 브라우저에서 마우스 오른쪽 버튼을 클릭하면 팝업 메뉴가 열리고, 팝업 메뉴에서 검사Inspect를 선택합니다.
  - 방법 3: 단축키를 사용합니다. Windows: ctrl + shift + I, MacOS: cmd + opt + I

*# [참고] Windows는 F12 키를 지원합니다.*

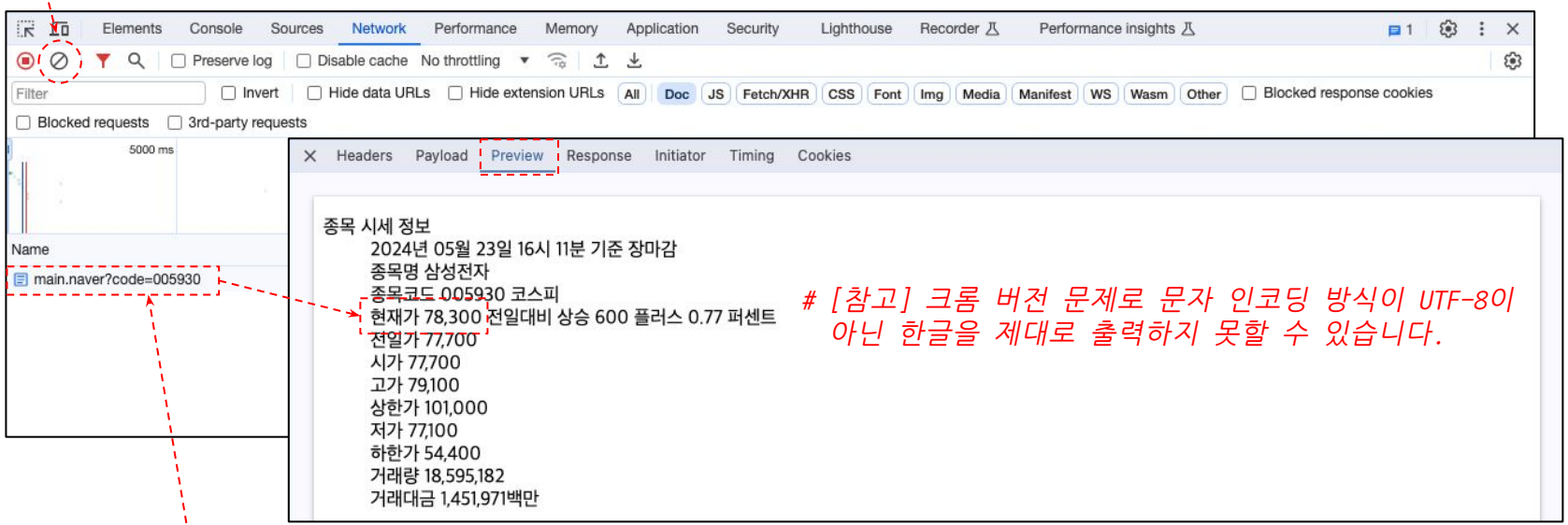
# HTTP 통신 리소스 찾기

- 타겟 페이지로 이동하여 수집할 데이터를 확인하고 크롬 개발자 도구를 엽니다.
  - 타겟 페이지: https://finance.naver.com/item/main.naver?code=005930



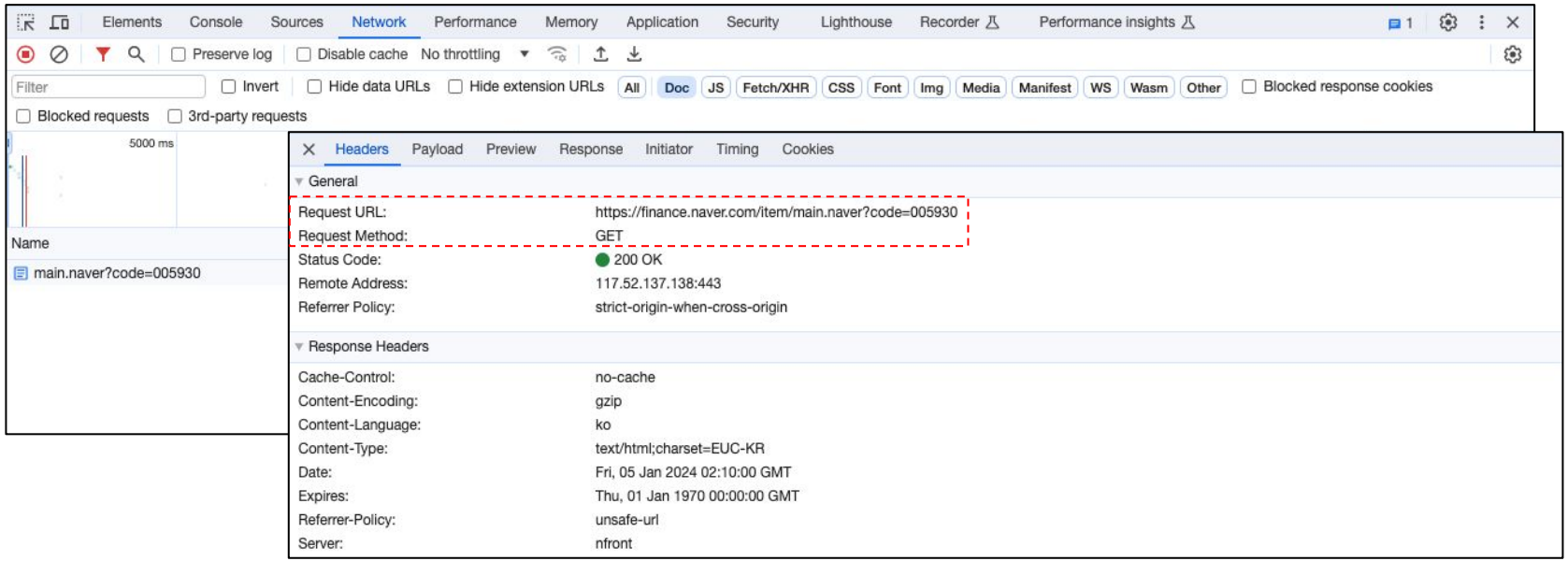
# HTTP 통신 리소스 찾기(계속)

- 삭제 버튼을 클릭하고 새로고침하면 HTTP 요청을 실행하고 리소스를 표시합니다.



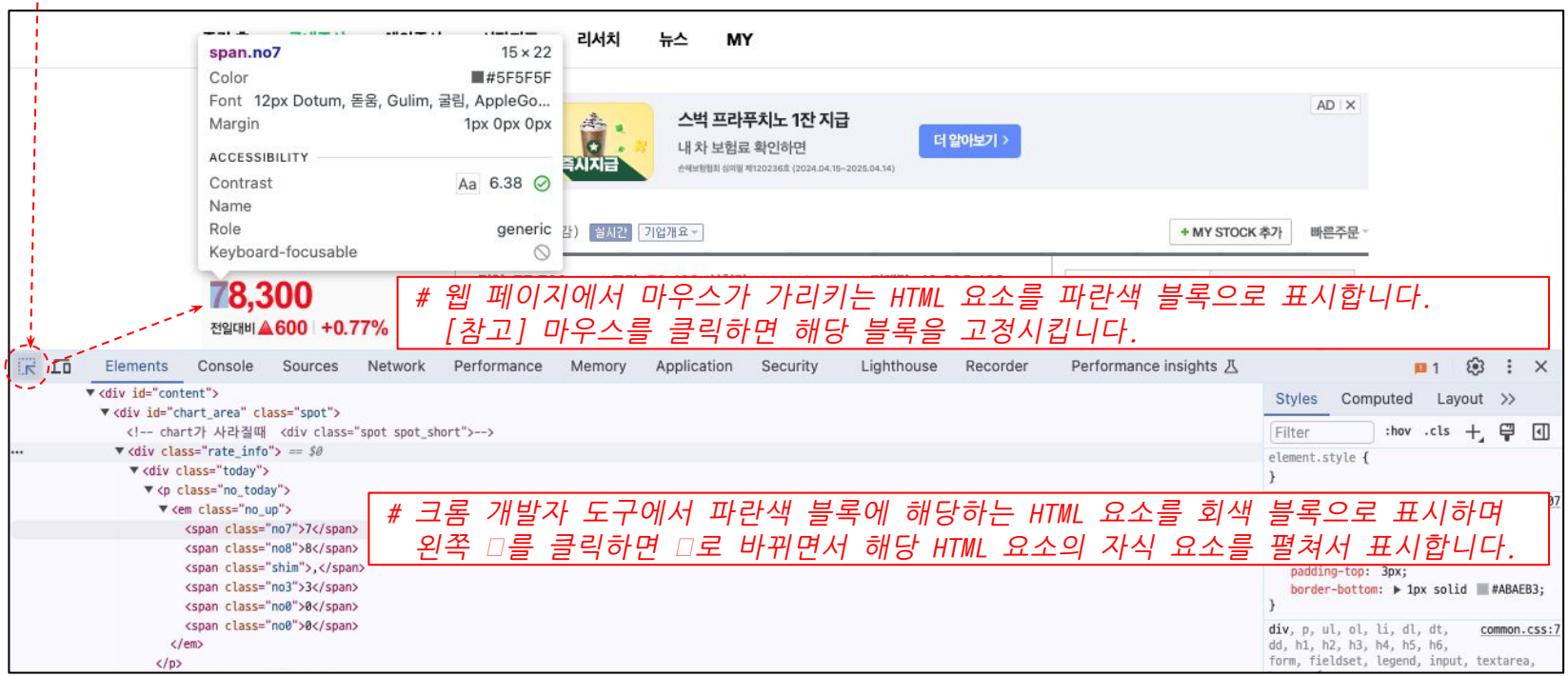
# HTTP 통신 리소스 찾기(계속)

- 리소스의 Headers로 이동하여 요청 방식과 요청 URL을 확인합니다.



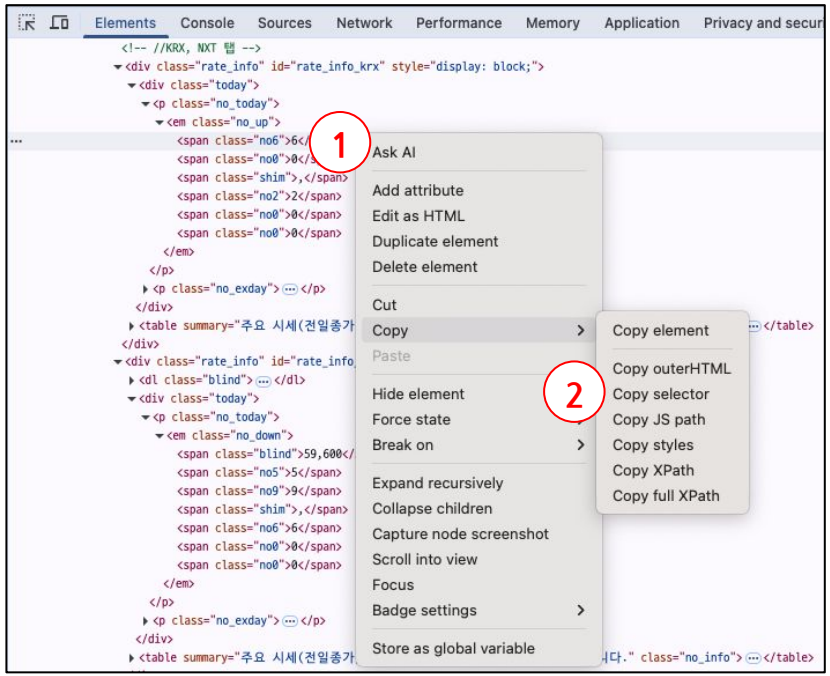
# HTML 요소 찾기

- 선택 버튼을 한 번 클릭하고(파란색) 웹 페이지에서 수집할 텍스트를 선택합니다.



# HTML 요소의 CSS 선택자 복사

- 다음 과정을 거쳐 HTML 요소의 CSS 선택자를 복사합니다.




① *HTML 요소 위에 마우스 오른쪽 버튼을 클릭합니다.*
② *팝업 메뉴에서 Copy → Copy selector를 선택합니다.*
③ *Jupyter Notebook에 복사한 값을 붙여넣습니다.*

3

```
[ ]: items = soup.select(selector = '#rate_info_krx > div > p.no_today > em > span.no6')
     len(items)
```

# 웹 스크레이핑 실습 1

현재 주식 가격 수집

# 현재 주식 가격 페이지 탐색

- 네이버 증권(https://finance.naver.com/) 페이지로 이동합니다.
- 검색창에 종목명을 입력하고 관련 페이지로 이동합니다.(예: 삼성전자)
- 종목명 오른쪽에 있는 6자리 종목코드를 확인합니다.(예: 005930)
- 크롬 개발자 도구를 열고 네트워크 탭에서 Doc로 이동합니다.
- 삭제 버튼을 클릭하고 웹 페이지를 새로고침합니다.
- 'main.naver'로 시작하는 리소스를 클릭합니다.
- Preview에서 수집하려는 텍스트를 찾습니다.
- Headers로 이동하여 HTTP 요청 방식을 확인하고 요청 URL을 복사합니다.

# [참고] HTTP 프록시 서버 설정

- HTTP 프록시 서버는 클라이언트와 서버 간의 통신을 중계하는 역할을 합니다.
    - 프록시 서버는 클라이언트의 IP 주소를 숨기고 캐싱을 통해 네트워크 성능을 높입니다.
    - 프록시 서버를 사용하면 로깅 및 접근 제어가 가능해져 보안을 강화할 수 있습니다.
- 필요한 모듈을 임포트합니다.

```
>>> import os
```

- 프록시 서버를 환경 변수에 추가합니다.

```
>>> os.environ['HTTP_PROXY'] = '프록시 서버 주소 붙여넣기'
>>> os.environ['HTTPS_PROXY'] = '프록시 서버 주소 붙여넣기'
```

# HTTP 요청 실행

- 필요한 모듈을 임포트합니다.

```
>>> import requests
```

- 상장회사의 종목코드를 설정합니다.

```
>>> ticker = '005930' # [참고] '005930'은 삼성전자 보통주의 종목코드입니다.
```

- 요청 URL을 설정합니다.

```
# [참고] 크롬 개발자 도구 리소스의 Headers에서 요청 URL을 복사하여 붙입니다.
  마지막 여섯 자리 숫자는 종목코드이므로 f-문자열로 대체하는 것이 좋습니다.
>>> url = f'https://finance.naver.com/item/main.naver?code={ticker}'
```

- HTTP 요청을 실행하고 res에 할당합니다.

```
>>> res = requests.get(url=url) # [참고] verify 매개변수에 False를 지정하면 SSL 인증서 검증을 비활성
                                  하므로(기본값: True) 원인 확인 없이 사용하는 것은 권장하지 않습니다.
```

# HTTP 응답 확인

- HTTP 응답 상태 코드를 확인합니다.

```
>>> res.status_code # [참고] 상태 코드가 200이면 다음 코드를 실행할 수 있습니다.
```

- HTTP 응답 헤더에서 콘텐트 유형을 확인합니다.

```
>>> res.headers['content-type'] # [참고] 응답 바디의 콘텐트 유형은 'text/html;charset=EUC-KR'입니다.
```

- HTTP 응답 바디 문자열을 출력합니다.

```
>>> print(res.text) # [참고] 응답 바디 문자열을 print 함수로 출력하는 것이 보기에 좋습니다.
```

- HTTP 요청 URL을 출력합니다.

```
>>> print(res.url) # [참고] 만약 응답 바디 문자열에 수집하려는 텍스트가 없으면 요청 URL을 웹 브라우저에
                  # 붙여넣고 실행했을 때 웹 페이지가 제대로 열리면 HTTP 요청 코드를 수정해야 합니다.
```

# HTML 데이터 처리

- 필요한 모듈을 임포트합니다.

```
>>> from bs4 import BeautifulSoup as BTS
```

- HTTP 응답 바디 문자열을 BeautifulSoup 객체로 파싱하고 soup에 할당합니다.

```
>>> soup = BTS(markup=res.text, features='html.parser')  # [참고] features 매개변수를 생략하면 'lxml', 'html5lib', 'html.parser'(내장) 순으로 자동 선택합니다.
```

- soup을 확인합니다.

```
>>> soup
```

- soup을 HTML의 계층 구조(들여쓰기)를 반영한 문자열로 변환하여 출력합니다.

```
>>> print(soup.prettify())  # [참고] prettify 메서드는 BeautifulSoup 객체를 HTML의 계층 구조에 맞게 포매팅하여 가독성 좋은 문자열로 반환합니다.
```

# 현재 주식 가격 수집

- 현재 주식 가격 정보를 포함하는 HTML 요소를 모두 선택하고 items에 할당합니다.

```
>>> items = soup.select(selector='#rate_info_krx > div > p.no_today > em > span')
```

- items의 원소 개수를 확인합니다.

```
>>> len(items)
```

- items를 확인합니다.

```
>>> items
```

*# [참고] items는 리스트 형태의 자료형이며 첫 번째 원소에 수집하려는 텍스트가 포함되어 있습니다.*

- items의 첫 번째 원소에서 문자열을 추출합니다.

```
>>> items[0].text
```

*# [참고] items의 첫 번째 원소에서 시작 태그와 종료 태그 사이에 있는 문자열을 선택합니다.*



# 현재 주식 가격 수집 함수 생성

- 종목코드를 지정하면 현재 주식 가격을 반환하는 함수를 생성합니다.

```
>>> def stock_price(ticker):
        url = f'https://finance.naver.com/item/main.naver?code={ticker}'
        res = requests.get(url=url)
        soup = BTS(markup=res.text, features='html.parser')
        items = soup.select(selector='#rate_info_krx > div > p.no_today > em > span')
        return items[0].text
```

*# [참고] 사용자 정의 함수 내부에 없는 모듈(requests)과 클래스(BTS)는 전역 변수 목록에서 참고합니다.*
*[주의] 만약 전역 변수 목록에 해당 모듈과 클래스가 없으면 사용자 정의 함수를 실행했을 때 에러를 일으킵니다.*

# 여러 종목의 현재 주식 가격 수집

- 관심 있는 종목코드를 리스트로 생성합니다.

```
>>> tickers = ['005930', '000660', '402340', '005935', '009150']
```

- 관심 있는 종목코드의 종목명을 리스트로 생성합니다.

```
>>> cpnames = ['삼성전자', '하이닉스', 'SK스퀘어', '삼전우선', '삼성전기']
```

- 반복문으로 종목별 현재 주식 가격을 수집하고 출력합니다.

```
>>> for ticker, cpname in zip(tickers, cpnames):
        price = stock_price(ticker=ticker)
        print(f'{cpname}의 현재 주식 가격은 {price}원입니다.') # [참고] 마지막 행에 time.sleep
                                                              함수를 추가하는 것이 좋습니다.
```

# 웹 스크레이핑 실습 2

시가총액 상위 50위 현재 주식 가격 수집

# 시가총액 페이지 탐색

- 네이버 증권(https://finance.naver.com/) 페이지로 이동합니다.
- 상단 메뉴에서 국내증시를 클릭, 왼쪽 메뉴에서 시가총액을 클릭합니다.
- 크롬 개발자 도구를 열고 네트워크 탭에서 Doc로 이동합니다.
- 삭제 버튼을 클릭하고 웹 페이지를 새로고침합니다.
- 'sise_market_sum'으로 시작하는 리소스를 클릭합니다.
- Preview에서 수집하려는 텍스트를 찾습니다.
- Headers로 이동하여 HTTP 요청 방식을 확인하고 요청 URL을 복사합니다.

# HTTP 요청 실행

- 필요한 모듈을 임포트합니다.

```
>>> import requests
```

- 요청 URL을 설정합니다.

```
>>> url = 'https://finance.naver.com/sise/sise_market_sum.naver?&page=1'
```

*# [참고] 웹 페이지 하단에 있는 페이지 네비게이션을 클릭하면 요청 URL에 쿼리 문자열이 추가됩니다.*

- HTTP 요청을 실행하고 res에 할당합니다.

```
>>> res = requests.get(url=url)
```

# HTTP 응답 확인

- HTTP 응답 상태 코드를 확인합니다.

```
>>> res.status_code
```

- HTTP 응답 헤더에서 콘텐트 유형을 확인합니다.

```
>>> res.headers['content-type'] # [참고] 응답 바디의 콘텐트 유형은 'text/html;charset=EUC-KR'입니다.
```

- HTTP 응답 바디 문자열을 출력합니다.

```
>>> print(res.text)
```

- HTTP 요청 URL을 출력합니다.

```
>>> print(res.url)
```

# HTML 데이터 처리

- 필요한 모듈을 임포트합니다.

```
>>> from bs4 import BeautifulSoup as BTS
```

- HTTP 응답 바디 문자열을 BeautifulSoup 객체로 파싱하고 soup에 할당합니다.

```
>>> soup = BTS(markup=res.text, features='html.parser')
```

- soup을 확인합니다.

```
>>> soup
```

- soup을 HTML의 계층 구조(들여쓰기)를 반영한 문자열로 변환하여 출력합니다.

```
>>> print(soup.prettify())
```

# 시가총액 상위 50위 현재 주식 가격 수집

- 현재 주식 가격을 포함하는 HTML 요소를 모두 선택하고 items에 할당합니다.

```
>>> items = soup.select(selector='td.number:nth-child(3)')
```

- items의 원소 개수를 확인합니다.

```
>>> len(items)
```

- items를 확인합니다.

```
>>> items
```

- items의 첫 번째 원소에서 문자열을 추출합니다.

```
>>> items[0].text
```

# 시가총액 상위 50위 현재 주식 가격 수집(계속)

- 결과를 저장할 빈 리스트를 생성합니다.

```
>>> item_texts = []
```

- 반복문으로 items의 각 원소에 있는 텍스트를 item_texts에 추가합니다.

```
>>> for item in items:
        item_texts.append(item.text) # [주의] 반복문에서 append 메서드를 사용할 때는 먼저 빈 리스트를 생성해야 합니다.
```

- 위 코드를 리스트 컴프리헨션으로 바꾸면 코드가 간결해집니다.

```
>>> [item.text for item in items] # [참고] 리스트 컴프리헨션을 사용할 때는 빈 리스트를 미리 생성하지 않아도 됩니다.
```

# 시가총액 상위 50위 종목명 수집

- 종목명을 포함하는 HTML 요소를 모두 선택하고 items에 할당합니다.

  ```
  >>> items = soup.select(selector='td:nth-child(2) > a.tltle')
  ```

- items의 원소 개수를 확인합니다.

  ```
  >>> len(items)
  ```

- items를 확인합니다.

  ```
  >>> items
  ```

- items의 각 원소에 있는 텍스트를 원소로 갖는 리스트를 생성합니다.

  ```
  >>> [item.text for item in items]
  ```

# 웹 스크레이핑 실습 3

주요 시장지표 수집

# 주요 시장지표 페이지 탐색

- 네이버 증권(https://finance.naver.com/) 페이지로 이동합니다.
- 상단 메뉴에서 시장지표를 클릭합니다.
- 크롬 개발자 도구를 열고 네트워크 탭에서 Doc로 이동합니다.
- 삭제 버튼을 클릭하고 웹 페이지를 새로고침합니다.
- 'marketindex'로 시작하는 리소스를 클릭합니다.
- Preview에서 수집하려는 텍스트를 찾습니다.
- Headers로 이동하여 HTTP 요청 방식을 확인하고 요청 URL을 복사합니다.

# HTTP 요청 실행

- 필요한 모듈을 임포트합니다.

```
>>> import requests
```

- 요청 URL을 설정합니다.

```
>>> url = 'https://finance.naver.com/marketindex/'
```

- HTTP 요청을 실행하고 res에 할당합니다.

```
>>> res = requests.get(url=url)
```

# HTTP 응답 확인

- HTTP 응답 상태 코드를 확인합니다.

```
>>> res.status_code
```

- HTTP 응답 헤더에서 콘텐트 유형을 확인합니다.

```
>>> res.headers['content-type'] # [참고] 응답 바디의 콘텐트 유형은 'text/html;charset=EUC-KR'입니다.
```

- HTTP 응답 바디 문자열을 출력합니다.

```
>>> print(res.text)
```

- HTTP 요청 URL을 출력합니다.

```
>>> print(res.url)
```

# HTML 데이터 처리

- 필요한 모듈을 임포트합니다.

  ```
  >>> from bs4 import BeautifulSoup as BTS
  ```

- HTTP 응답 바디 문자열을 BeautifulSoup 객체로 파싱하고 soup에 할당합니다.

  ```
  >>> soup = BTS(markup=res.text, features='html.parser')
  ```

- soup을 확인합니다.

  ```
  >>> soup
  ```

- soup을 HTML의 계층 구조(들여쓰기)를 반영한 문자열로 변환하여 출력합니다.

  ```
  >>> print(soup.prettify())
  ```

# 화폐 단위 수집

- 화폐 단위를 포함하는 HTML 요소를 모두 선택하고 items에 할당합니다.

```
>>> items = soup.select(selector='#exchangeList > li > a > h3 > span.blind')
```

*# [참고] '#exchangeList span.blind'와 같이 태그 사이에 홑화살괄호를 생략하면 여러 세대를 건너뜁니다.*
*[주의] 여러 세대를 건너뛰면 결과가 달라질 수 있습니다.*

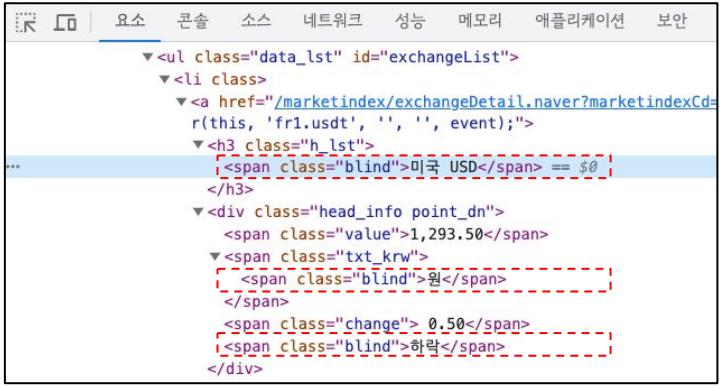
- items의 원소 개수를 확인합니다.

```
>>> len(items)
```

- items를 확인합니다.

```
>>> items
```

*# [참고] items는 리스트 형태의 자료형이며*
*모든 원소에 수집할 텍스트가 있습니다.*



# 화폐 단위 수집(계속)

- 결과를 저장할 빈 리스트를 생성합니다.

```
>>> item_texts = []
```

- 반복문으로 items의 각 원소에 있는 텍스트를 item_texts에 추가합니다.

```
>>> for item in items:
        item_texts.append(item.text)
```

- 위 코드를 리스트 컴프리헨션으로 바꾸면 코드가 간결해집니다.

```
>>> [item.text for item in items]
```

# 기존 클래스를 상속하고 메서드 추가

- bs4.BeautifulSoup 클래스를 상속하고, 텍스트를 리스트로 반환하는 메서드를 추가합니다.

```
>>> class MyBTS(BTS): # BTS 클래스를 상속한 MyBTS 클래스를 생성합니다.

        def html_text(self, css_selector): # MyBTS 클래스에 html_text라는 메서드를 추가합니다.
                                           # [참고] self는 MyBTS로 생성한 인스턴스를 가리킵니다.
            items = self.select(selector=css_selector)

            return [item.text for item in items]
```

- MyBTS 클래스로 soup2를 생성합니다.

```
>>> soup2 = MyBTS(markup=res.text, features='html.parser')

# [참고] BTS 클래스로 생성한 인스턴스(이번 예제에서의 soup)에는 html_text라는 메서드가 없습니다.
  MyBTS 클래스로 생성한 인스턴스(이번 예에서의 soup2)에는 html_text라는 메서드가 있습니다.
```

# 시장지표 수집

- 환전 고시 환율을 리스트로 반환합니다.

```
>>> soup2.html_text(css_selector='#exchangeList > li > a > div > span.value')
```

- 국제 시장 환율을 리스트로 반환합니다.

```
>>> soup2.html_text(css_selector='#worldExchangeList > li > a > div > span.value')
```

- 유가 · 금시세를 리스트로 반환합니다.

```
>>> soup2.html_text(css_selector='#oilGoldList > li > a > div > span.value')
```

# 웹 스크레이핑 실습 4

증권 주요뉴스 수집

# 증권 주요뉴스 페이지 탐색

- 네이버 증권(https://finance.naver.com/) 페이지로 이동합니다.
- 상단 메뉴에서 뉴스를 클릭하고, 왼쪽 메뉴에서 주요뉴스를 클릭합니다.
- 크롬 개발자 도구를 열고 네트워크 탭에서 Doc로 이동합니다.
- 웹 페이지 하단으로 이동하여 특정 날짜와 페이지를 클릭합니다. *# [참고] 요청 URL에 쿼리 문자열이 추가됩니다.*
- 'mainnews'로 시작하는 리소스를 클릭합니다.
- Preview에서 수집하려는 텍스트를 찾습니다.
- Headers로 이동하여 HTTP 요청 방식을 확인하고 요청 URL의 물음표 앞까지 복사합니다.
- Payload로 이동하여 쿼리 문자열을 확인합니다. *# [참고] 만약 요청 URL에 쿼리 문자열이 없으면 Payload가 보이지 않습니다.*

# HTTP 요청 실행

- 필요한 모듈을 임포트합니다.

```
>>> import requests
```

- 요청 URL을 설정합니다.

```
>>> url = 'https://finance.naver.com/news/mainnews.naver' # [참고] Headers에서 복사한 요청 URL의 물음표 앞까지 문자열로 붙여넣습니다.
```

- 쿼리 문자열을 딕셔너리로 설정합니다.

```
>>> query = dict(date='2026-01-01', page=1) # [참고] Payload에서 복사한 값을 딕셔너리로 생성합니다. dict 클래스를 사용하는 것이 편리합니다.
```

- HTTP 요청을 실행하고 res에 할당합니다.

```
>>> res = requests.get(url=url, params=query) # [참고] params 매개변수에 쿼리 문자열을 지정합니다.
```

# HTTP 응답 확인

- HTTP 응답 상태 코드를 확인합니다.

```
>>> res.status_code
```

- HTTP 응답 헤더에서 콘텐트 유형을 확인합니다.

```
>>> res.headers['content-type'] # [참고] 응답 바디의 콘텐트 유형은 'text/html;charset=EUC-KR'입니다.
```

- HTTP 응답 바디 문자열을 출력합니다.

```
>>> print(res.text)
```

- HTTP 요청 URL을 출력합니다.

```
>>> print(res.url)
```

# HTML 데이터 처리

- 필요한 모듈을 임포트합니다.

  ```
  >>> from bs4 import BeautifulSoup as BTS
  ```

- HTTP 응답 바디 문자열을 BeautifulSoup 객체로 파싱하고 soup에 할당합니다.

  ```
  >>> soup = BTS(markup=res.text, features='html.parser')
  ```

- soup을 확인합니다.

  ```
  >>> soup
  ```

- soup을 HTML의 계층 구조(들여쓰기)를 반영한 문자열로 변환하여 출력합니다.

  ```
  >>> print(soup.prettify())
  ```

# 뉴스 정보 수집

- 뉴스 제목을 포함하는 HTML 요소를 모두 선택하고 items에 할당합니다.

```
>>> items = soup.select(selector='.articleSubject > a')
```

*# [참고] 썸네일 유무에 따라 <dt> 또는 <dd> 태그로 달라집니다.*

- items의 원소 개수를 확인합니다.

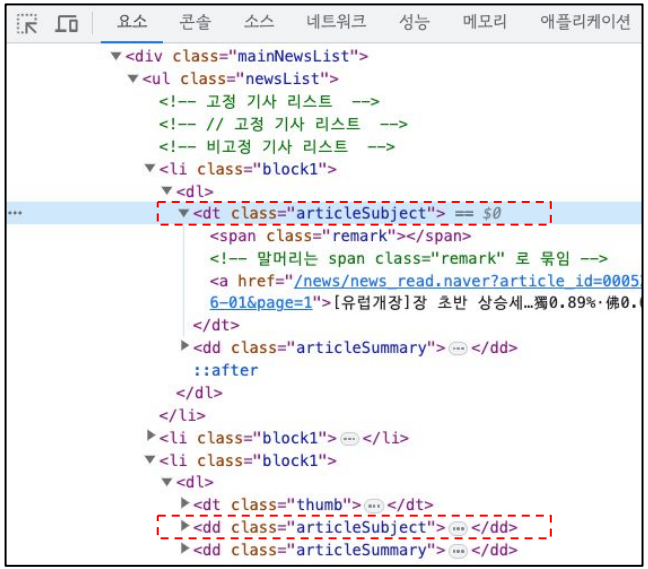
```
>>> len(items)
```

- items의 첫 번째 원소에서 문자열을 추출합니다.

```
>>> items[0].text
```

- items의 첫 번째 원소에서 링크를 추출합니다.

```
>>> items[0]['href']
```



# 기존 클래스를 상속하고 메서드 추가

- bs4.BeautifulSoup 클래스를 상속하고, 링크를 리스트로 반환하는 메서드를 추가합니다.

```
>>> class MyBTS(BTS):
        def html_text(self, css_selector):
            items = self.select(selector=css_selector)
            return [item.text for item in items]
        def html_attr(self, css_selector, key='href'):   # [참고] key 매개변수에 전달인수 기본값을 설정하면, 메서드를 호출할 때 전달인수를 생략할 수 있습니다.
            items = self.select(selector=css_selector)
            return [item[key] for item in items]
```

# 뉴스 정보 수집

- MyBTS 클래스로 soup2를 생성합니다.

```
>>> soup2 = MyBTS(markup=res.text, features='html.parser')
```

- 뉴스 제목을 리스트로 반환합니다.

```
>>> soup2.html_text(css_selector='.articleSubject > a')
```

- 뉴스 링크를 리스트로 반환합니다.

```
>>> soup2.html_attr(css_selector='.articleSubject > a') # [참고] key 매개변수를 생략하면 인수의 기본값을 적용합니다.(기본값: 'href')
```

- 뉴스 등록일을 리스트로 반환합니다.

```
>>> soup2.html_text(css_selector='.articleSummary > span.wdate')
```

# 데이터프레임 생성

- 필요한 모듈을 임포트합니다.

```
>>> import pandas as pd
```

- 제목, 링크 및 등록일 열을 갖는 데이터프레임을 생성합니다.

```
>>> pd.DataFrame(data={
        '제목': soup2.html_text(css_selector='.articleSubject > a'),
        '링크': soup2.html_attr(css_selector='.articleSubject > a'),
        '등록일': soup2.html_text(css_selector='.articleSummary > span.wdate')
    }) # [참고] 데이터프레임에 '언론사'도 추가해보세요.
```

# Open API 활용법

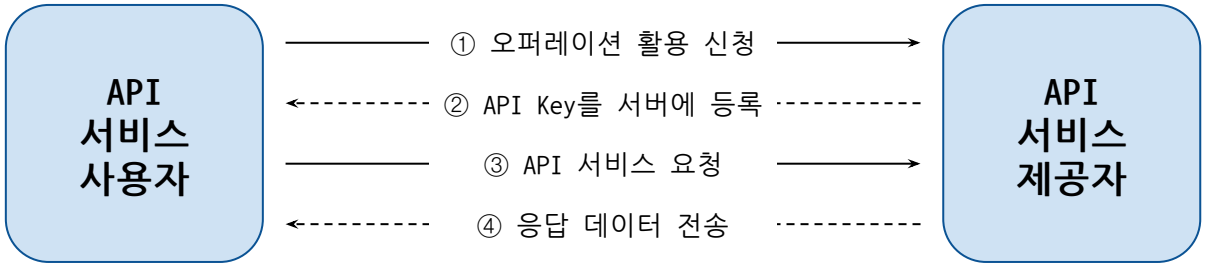
공공데이터 수집 및 처리

# Open API 개요

- API[Application Programming Interface]는 응용프로그램 간 통신을 위한 인터페이스입니다.
  - 클라이언트와 서버가 데이터를 주고받는 용도로 API를 사용합니다.
  - API를 활용하면 데이터를 효율적으로 수집하고 다양한 기능을 쉽게 통합할 수 있습니다.
- Open API는 활용 신청을 완료한 사용자에게 제공되는 API 서비스입니다.
  - 오퍼레이션은 데이터베이스에서 하나의 데이터 테이블에 해당하는 단위입니다.
  - API 서비스 사용자가 오퍼레이션 활용 신청을 완료하면 API Key가 서버에 등록됩니다.
    - 사용자별로 하나의 API Key가 발급되며, 이를 통해 사용자 식별 및 API 사용량을 관리합니다.
  - 각 오퍼레이션은 일별 정해진 한도 내에서 데이터를 자유롭게 호출할 수 있습니다.

# Open API 이용 프로세스

- API 서비스 이용 프로세스로 도식화하면 다음과 같습니다.




- API 서비스 사용자가 오퍼레이션 활용을 완료하면 API 서비스 제공자는 적법한 사용자에게 API Key를 발급합니다.
    - 공공데이터포털에서 오퍼레이션 활용 신청이 완료될 때 API Key가 서버에 등록되지만 서버 처리에는 약 1~2시간 정도 소요될 수 있으니 참고하시기 바랍니다.

# Open API 이용 프로세스(계속)

- API 서비스를 이용하려면 API 요청 함수에 API Key를 추가해야 합니다.
  - 사용자별로 고유한 API Key가 발급됩니다.
  - 신청한 오퍼레이션에 대해서만 API Key를 사용할 수 있습니다.
  - 공공데이터포털은 REST 방식으로 통신하며, 주로 GET 방식의 HTTP 요청을 사용합니다.
  - 요청 시 쿼리 문자열에 발급받은 API Key를 포함해야 합니다.
- API 서비스 제공자는 일반적으로 XML 또는 JSON 형태로 데이터를 응답하며, 기본 응답 형식은 XML인 경우가 많습니다.
  - 쿼리 문자열에 type 매개변수를 추가하여 응답받을 데이터의 형식을 지정할 수 있습니다.

# XML

- XML은 인터넷으로 연결된 시스템 간 데이터를 주고받기 위해 만들어진 마크업 언어입니다.
  - XML은 Extensible Markup Language의 약자로 데이터를 구조화하는 데 사용됩니다.
  - HTML과 비슷한 형태지만 XML은 특정 기능보다 데이터를 표현하는 데 중점을 둡니다.
  - 많은 API에서 XML 형태의 텍스트 데이터를 주고받습니다.
- XML은 시작 태그와 종료 태그 사이에 데이터를 포함하는 구조이므로 HTML과 유사합니다.
  - XML 노드는 별도의 기능 없이 데이터를 표현하는 역할을 합니다.
  - XML 노드는 대소문자를 구분합니다.
  - XML은 사용자 정의 태그를 지원하며, 데이터 구조를 유연하게 설계할 수 있습니다.

# JSON

- JSON은 JavaScript로 데이터를 주고받을 때 사용하는 교환 형식입니다.
  - JSON은 JavaScript Object Notation의 약자입니다.
  - 사람과 컴퓨터 모두 쉽게 이해할 수 있으며 데이터 용량도 작아 효율적입니다.
  - 많은 API에서 XML을 대체하는 데이터 교환 형식으로 JSON을 사용합니다.
- JSON은 Python의 딕셔너리와 유사한 구조를 가진 문자열입니다.
  - 중괄호 안에 key: value 형태의 원소를 포함하며, 원소 간에는 콤마로 구분합니다.
  - key는 XML의 노드와 유사하며, 데이터를 설명하는 역할을 합니다.
  - value는 하나의 값(스칼라) 또는 여러 값을 포함하는 리스트로 지정할 수 있습니다.

# XML과 JSON 데이터 처리

- XML 형태의 문자열을 처리할 때 xmltodict 라이브러리를 사용합니다.
  - xmltodict.parse 함수는 XML 형태의 문자열을 딕셔너리로 변환합니다.
  - 딕셔너리로 변환된 데이터를 데이터프레임으로 쉽게 변환할 수 있습니다.
  - XML 노드는 대소문자를 구분하므로 태그 이름을 정확히 입력해야 합니다.
- JSON 형태의 문자열을 처리할 때 json 라이브러리를 사용합니다.
  - json.loads 함수는 JSON 형태의 문자열을 딕셔너리로 변환합니다.
  - 딕셔너리로 변환된 데이터를 데이터프레임으로 쉽게 변환할 수 있습니다.
  - JSON 문자열은 반드시 중괄호로 시작하고 중괄호로 끝나야 합니다.

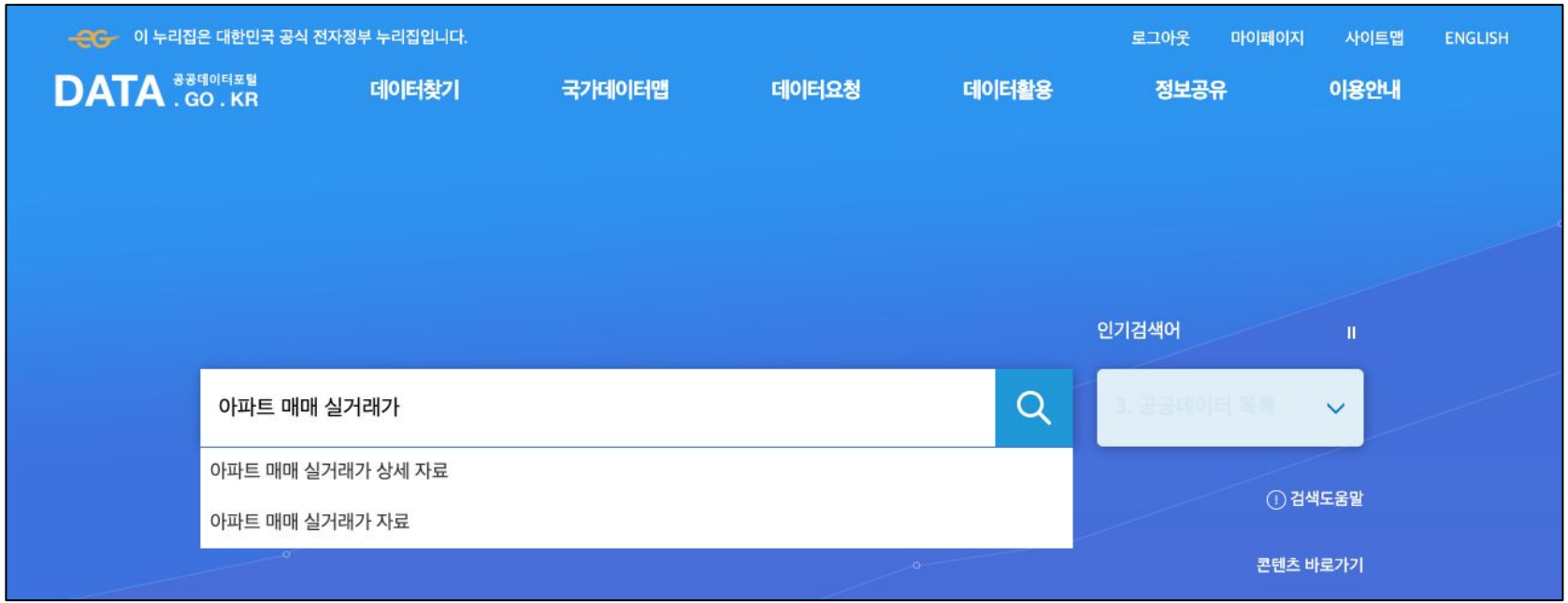
# 우리나라 공공기관의 주요 Open API 서비스

- 우리나라는 2013년에 공공데이터 제공 및 이용 활성화에 관한 법률을 제정하였고 공공데이터의 민간 활용을 증대시키려는 노력을 경주해오고 있습니다.

| Open API | 제공자 | 웹 사이트 |
|---|---|---|
| 공공데이터포털 | 행정안전부 | https://www.data.go.kr/ |
| 서울열린데이터광장 | 서울특별시 | https://data.seoul.go.kr/ |
| 경기데이터드림 | 경기도 | https://data.gg.go.kr/ |
| ECOS 경제통계 OpenAPI | 한국은행 | https://ecos.bok.or.kr/api/# |
| OPEN DART | 금융감독원 | https://opendart.fss.or.kr/ |
| KOSIS 국가통계포털 | 통계청 | https://kosis.kr/serviceInfo/openAPIGuide.do |

# 공공데이터포털 오픈 API 서비스 신청

- 공공데이터포털에 접속하여 로그인하고 '아파트 매매 실거래가'로 검색합니다.



# 오픈API 탭에서 검색

- 오픈 API 탭에서 관심 있는 오퍼레이션을 찾습니다.



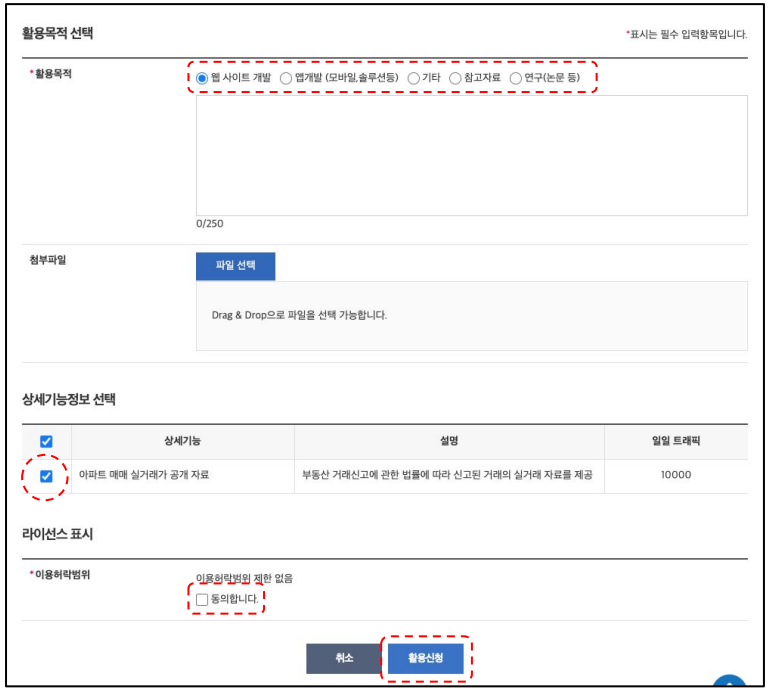
# 오퍼레이션 상세정보 확인

- 관심 있는 오퍼레이션을 클릭하고 상세 정보를 확인합니다.



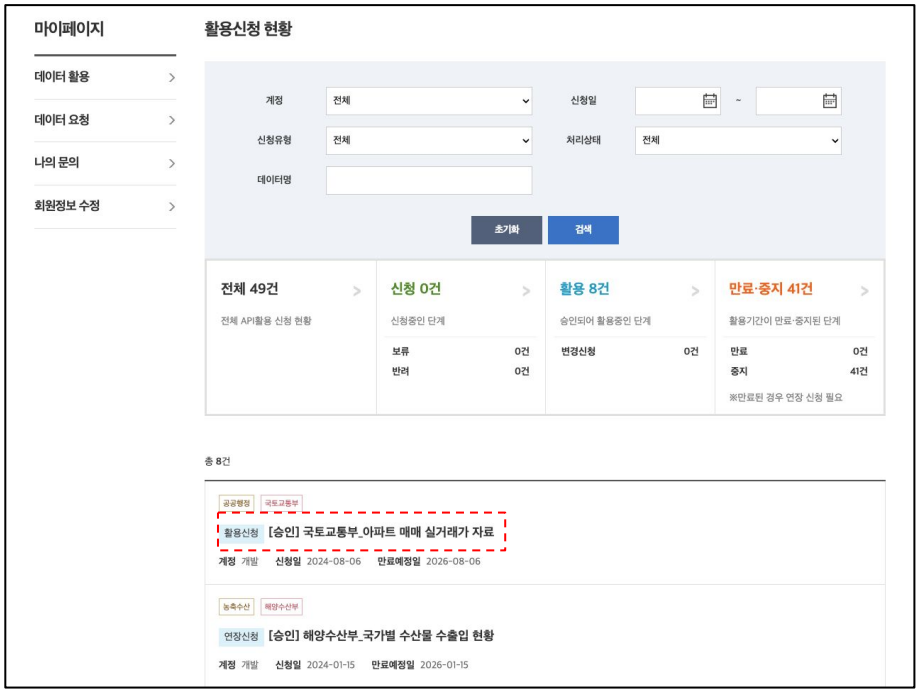
# 오퍼레이션 활용신청

- 활용목적: 다섯 가지 항목 중에서 택1하고, 상세 내용을 입력합니다.
- 상세기능정보 선택: 체크박스에 체크합니다.
    - 만약 항목이 여러 개 나열되어 있으면 신청하려는 항목을 모두 체크합니다.
- 라이센스 표시: '동의합니다'에 체크합니다.
- 마지막으로 하단의 활용신청 버튼 클릭하면, 오퍼레이션 활용 신청을 완료합니다.



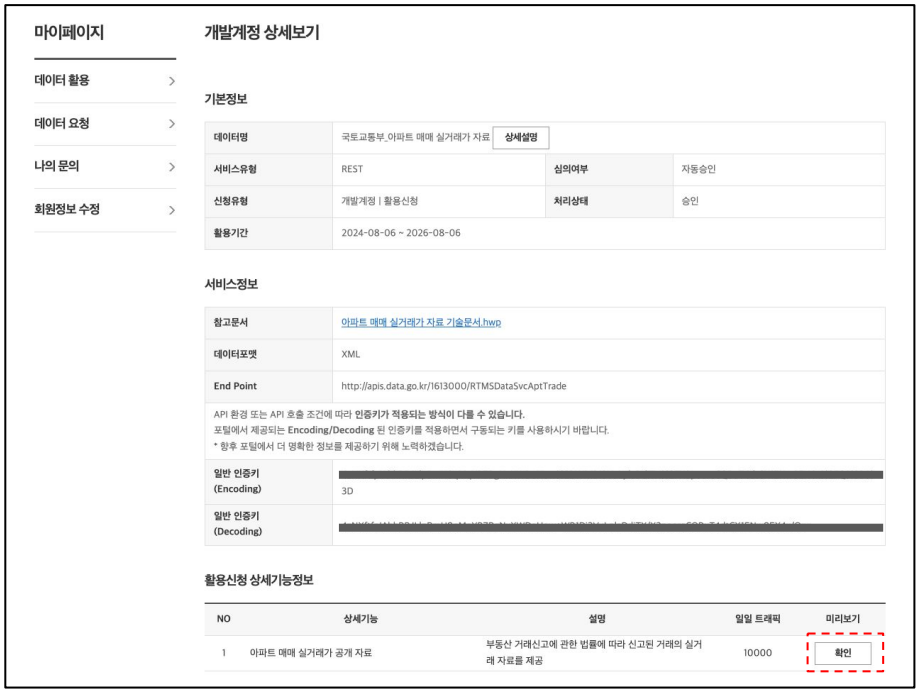
# 신청 완료 후 마이페이지에서 확인

- 오퍼레이션 활용 신청을 완료한 후에, 마이페이지로 이동합니다.
- 마이페이지에서 왼쪽 메뉴에서 데이터 활용 → Open API → 활용신청 현황을 차례대로 클릭하면 지금까지 활용신청한 오퍼레이션 목록을 확인할 수 있습니다.
- 오퍼레이션을 클릭하면 화면 중간에서 발급받은 API 인증키 2종(Encoding 및 Decoding)을 확인할 수 있습니다.



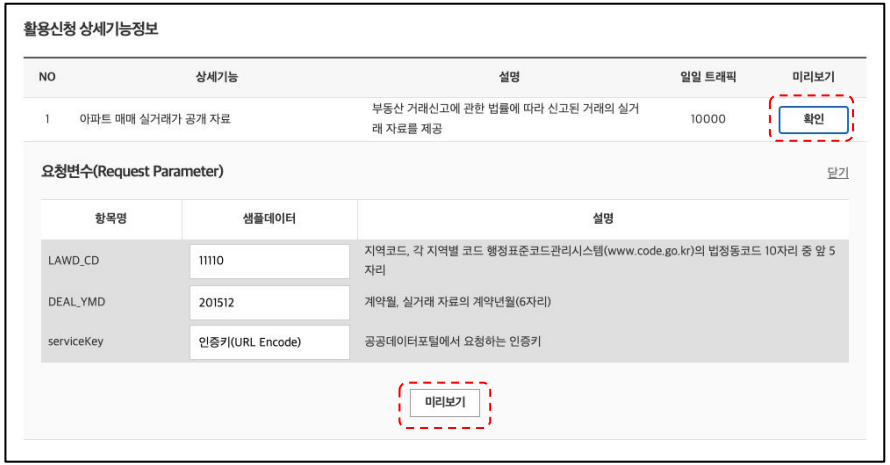
# 일반 인증키 발급 확인

- 오른쪽 이미지 하단에 일반 인증키 2종을 확인할 수 있습니다.
  - 일반 인증키(Decoding) 문자열을 복사하여 Jupyter Notebook 코드 셀에 붙여넣습니다.
  - 일반 인증키(Encoding) 문자열은 사용하지 않습니다.
- 활용신청 상세기능정보에서 확인 버튼을 클릭하여 응답 메시지를 미리보기합니다.



# 상세기능정보 미리보기

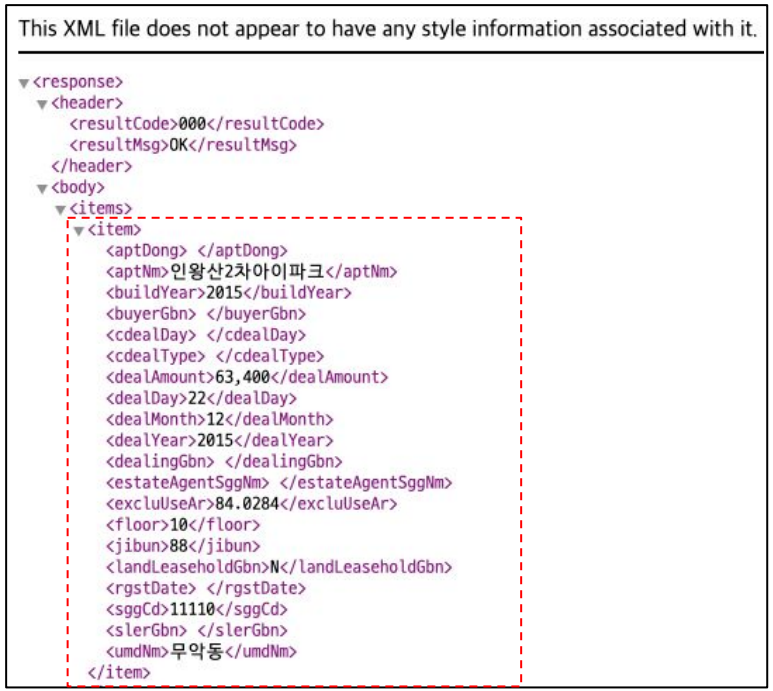
- 활용신청 상세기능정보에서 미리보기 확인 버튼을 클릭하면, 요청변수가 웹 페이지에 추가됩니다.
  - LAWD_CD: 11110(서울특별시 종로구)
  - DEAL_YMD: 201512(2015년 12월)
  - serviceKey: 인증키(자동 입력됨)
- 요청변수에 적당한 값을 입력하고 하단의 미리보기 버튼을 클릭하면 새 창에서 XML 형태의 응답 메시지를 확인할 수 있습니다.




*# [참고] 오퍼레이션에 따라 인증키 승인에 소요되는 시간이 다릅니다. 일반적으로 1~2시간 가량 소요됩니다.*

# 상세기능정보 미리보기(계속)

- 일정 시간이 경과한 후에 미리보기를 실행하면 XML 데이터를 확인할 수 있습니다.
  - item 노드는 한 건의 거래를 의미합니다.
  - XML 데이터를 데이터프레임으로 변환하면 item 노드는 행이 되고, 하위 노드들은 열 이름으로 자동 적용됩니다.
- 만약 'SERVICE ACCESS DENIED ERROR'라는 응답 메시지를 출력했다면, 아직 API Key가 서버에 정상 등록되지 않았다는 것을 의미합니다.



# API 인증키 저장 파일 생성

- API 인증키를 .env 파일로 저장합니다. *# [참고] 이 작업은 API Key별로 한 번씩만 실행하면 됩니다.*

```
>>> with open(file='.env', mode='a') as f: # [참고] 'w'는 쓰기, 'a'는 추가, 'r'은 읽기 모드로 파일을 엽니다.
        f.write(f'DATA_GO_KR_API_KEY=일반 인증키(Decoding) 붙여넣기')
```

*# [주의] 일반 인증키(Encoding)을 붙여넣으면 에러를 일으킵니다.*
*requests.get 함수의 params 매개변수에 지정한 문자열을 자동으로 퍼센트 인코딩을 자동으로 수행하기 때문입니다.*

- 필요한 모듈을 임포트합니다.

```
>>> import os
```

- 현재 작업 경로에 있는 폴더명과 파일명을 확인합니다.

```
>>> os.listdir() # [참고] 목록에서 .env 파일이 포함되어 있는지 확인합니다.
```

# API 인증키 불러오기

- 필요한 모듈을 임포트합니다.

```
>>> from dotenv import load_dotenv, dotenv_values
```

- .env 파일을 환경변수에 로드합니다.

```
>>> load_dotenv()
```

*# [참고] .env 파일을 정상적으로 불러왔으면 콘솔에 True를 반환합니다.*

- .env 파일에 저장된 변수명 목록을 확인합니다.

```
>>> dotenv_values(dotenv_path='.env').keys()
```

*# [참고] .env 파일을 저장할 때 사용한 변수명을 기억하지 못할 수 있으므로 위 코드를 통해 변수명을 확인합니다.*

# HTTP 요청 실행

- 필요한 모듈을 임포트합니다.

```
>>> import requests
```

- 요청 URL을 설정합니다.

```
>>> url = 'https://apis.data.go.kr/1613000/RTMSDataSvcAptTrade/getRTMSDataSvcAptTrade'
```

*# [주의] SSL/TLS 에러가 발생하면 API 문서에서 HTTP 요청을 지원하는지 확인한 뒤 'https'를 'http'로 변경합니다.*
*requests의 기본 TLS 설정이 일부 서버와 맞지 않으면, TLS 1.2를 강제로 적용하고 낮은 보안 수준을 허용합니다.*

- 공공데이터포털 오픈 API 인증키를 설정합니다.

```
>>> api_key = os.getenv(key='DATA_GO_KR_API_KEY')
```

*# [주의] API Key를 코드에 직접 작성하면 Jupyter Notebook 파일을 공유할 때 다른 사람에게 노출될 수 있습니다.*

# HTTP 요청 실행(계속)

- 요청 변수를 딕셔너리로 설정합니다.

```
>>> query = dict(LAWD_CD='11110', # 지역코드를 다섯 자리 문자열로 지정합니다.(11110: 종로구)
                                  # [참고] https://www.code.go.kr에서 법정동 코드를 확인합니다.
                 DEAL_YMD='201512', # 거래년월을 여섯 자리 문자열로 지정합니다.
                 serviceKey=api_key, # 공공데이터포털 일반 인증키(Decoding)를 지정합니다.
                 pageNo=1, # 페이지수를 4자리 정수로 설정합니다.(기본값: 1)
                 numOfRows=10) # 한 번에 제공할 건수를 4자리 정수로 설정합니다.(기본값: 10)
```

- HTTP 요청을 실행하고 res에 할당합니다.

```
>>> res = requests.get(url=url, params=query)
```

# HTTP 응답 확인

- HTTP 응답 상태 코드를 확인합니다.

```
>>> res.status_code
```

- HTTP 응답 헤더에서 콘텐트 유형을 확인합니다.

```
>>> res.headers['content-type']  # [참고] 응답 바디의 콘텐트 유형은 'application/xml;charset=utf-8' 입니다.
```

- HTTP 응답 바디 문자열을 출력합니다.

```
>>> print(res.text)
```

- HTTP 요청 URL을 출력합니다.

```
>>> print(res.url)
```

# XML 데이터 처리

- 필요한 모듈을 임포트합니다.

```
>>> import xmltodict  # [참고] xmltodict 라이브러리는 XML 형태의 문자열을 딕셔너리로 변환할 때 사용합니다.
                      # JSON 형태의 문자열을 딕셔너리로 변환하려면 json 라이브러리를 임포트합니다.

>>> from pprint import pprint  # [참고] pprint는 중첩된 딕셔너리의 키와 값을 정렬하여 보기 좋게 출력하는
                               # 표준 라이브러리입니다.
```

- HTTP 응답 바디 문자열을 딕셔너리로 변환하고 data_dict에 할당합니다.

```
>>> data_dict = xmltodict.parse(xml_input=res.text)

# [참고] JSON 형태의 문자열을 딕셔너리로 변환하려면 다음 코드를 실행합니다.
#   >>> data_dict = json.loads(s=res.text)
```

- data_dict(중첩된 딕셔너리)의 키와 값을 정렬하여 출력합니다.

```
>>> pprint(data_dict)
```

# 데이터프레임 생성

- 필요한 모듈을 임포트합니다.

```
>>> import pandas as pd
```

- data_dict로 데이터프레임을 생성하고 apt_df에 할당합니다.

```
>>> apt_df = pd.DataFrame(data=data_dict['response']['body']['items']['item'])
```

- apt_df의 처음 5행을 확인합니다.

```
>>> apt_df.head() # [참고] n 매개변수에 행 개수를 정수로 지정합니다.(기본값: 5)
```

- apt_df의 정보를 확인합니다.

```
>>> apt_df.info() # [참고] 행 개수, 열 개수, 열 이름, 열별 결측값 아닌 개수 및 열별 자료형을 출력합니다.
```

# Selenium 활용법

# Selenium의 이해

- Selenium은 다양한 웹 브라우저에서 웹 기반 애플리케이션의 동작을 자동화하기 위해 설계된 도구입니다.
    - 주로 테스트 자동화와 웹 브라우저 제어에 사용됩니다.
- Selenium의 구성 요소는 다음과 같습니다.
    - Selenium Client: Selenium 스크립트를 작성하고 실행할 수 있는 라이브러리입니다.
    - WebDriver: 브라우저와 통신을 담당하는 프로토콜입니다. Selenium Client와 Browser Driver 사이의 명령을 전달합니다.
    - Browser Driver: 특정 브라우저를 제어하기 위한 드라이버입니다. 브라우저마다 별도의 드라이버가 필요합니다.

# Selenium으로 수행할 수 있는 동작

- 크롬, 엣지 또는 파이어폭스 등 다양한 웹 브라우저를 제어할 수 있습니다.
    - 특정 URL로 이동하거나 브라우저 창 크기를 조정할 수 있습니다.
- 검색창에 키워드를 입력하고 버튼을 클릭하는 등의 동작을 자동화할 수 있습니다.
    - 아이디와 비밀번호를 입력한 뒤 로그인 버튼을 클릭하는 시뮬레이션이 가능합니다.
- 스크롤을 위/아래로 이동하거나 페이지를 이전/다음으로 넘길 수 있습니다.
    - 무한 스크롤을 사용한 웹 페이지에서 여러 번 스크롤을 내리는 것도 가능합니다.
- 웹 페이지의 HTML을 문자열 형태로 가져올 수 있습니다.
    - 단순 HTTP 요청으로 접근할 수 없는 (예를 들어 로그인이 필요한) 페이지에 유용합니다.

# WebDriver 객체 생성

- 필요한 모듈을 임포트합니다.

```
>>> from selenium import webdriver
```

- 크롬 브라우저를 제어하기 위한 WebDriver 객체를 생성하고 driver에 할당합니다.

```
>>> driver = webdriver.Chrome()
```

*# [참고] Selenium 4.6부터 Selenium Manager가 내장되어서 왼쪽 코드를 실행할 때 자동으로 크롬 드라이버를 찾아서 내려받습니다. 만약 에러가 발생하면 '부록1'을 참고하세요.*

- HTML 요소를 찾을 때, 최대 3초까지 대기하도록 설정합니다.

```
>>> driver.implicitly_wait(time_to_wait=3)
```

*# [참고] driver.find_element 또는 driver.find_elements로 HTML 요소를 찾을 때 최대 대기해야 할 시간을 설정합니다.*

- 타겟 페이지로 이동합니다.

```
>>> driver.get(url='https://www.naver.com')
```

# 하이퍼링크 클릭

- 필요한 모듈을 임포트합니다.

```
>>> from selenium.webdriver.common.by import By

# [참고] Selenium에서 HTML 요소를 찾는 기준을 제공하는 By 클래스를 임포트합니다.
```

- 크롬 브라우저에서 링크 텍스트가 '블로그'와 일치하는 첫 번째 HTML 요소를 클릭합니다.

```
>>> driver.find_element(by=By.LINK_TEXT, value='블로그').click()

# [참고] a 태그에 포함된 전체 문자열이 value와 동일한 HTML 요소를 찾습니다.
  개발자 도구를 열지 않고 브라우저 화면에 보이는 링크 텍스트를 기준으로 요소를 찾을 수 있으므로 편리합니다.
```

# [참고] 주요 By 소개

| 구분 | 상세 내용 | 동일 기능 문자열 |
|---|---|---|
| By.XPATH | XPath 문자열로 HTML 요소를 찾습니다. | 'xpath' |
| By.CSS_SELECTOR | CSS 선택자 문자열로 HTML 요소를 찾습니다. | 'css selector' |
| By.ID | id 속성에 연결된 문자열로 찾습니다. | 'id' |
| By.CLASS_NAME | class 속성에 연결된 문자열로 찾습니다. | 'class name' |
| By.NAME | name 속성에 연결된 문자열로 찾습니다. | 'name' |
| By.TAG_NAME | 태그 이름으로 찾습니다. | 'tag name' |
| By.LINK_TEXT | a 태그 사이에 있는 전체 문자열로 찾습니다. | 'link text' |
| By.PARTIAL_LINK_TEXT | a 태그 사이에 있는 부분 문자열로 찾습니다. | 'partial link text' |

*# [참고] By 모듈은 자동 완성 기능을 제공하므로 'xpath'와 같이 문자열을 지정하는 것보다 편리합니다.*

# 윈도우 이동

- 현재 열려 있는 크롬 브라우저 윈도우의 핸들 리스트를 확인합니다.

```
>>> driver.window_handles
```

*# [참고] Selenium에서 윈도우는 브라우저 창 또는 탭을 의미합니다.*
*핸들은 각 윈도우나 탭을 구분하기 위해 사용하는 고유 식별값입니다.*

- 현재 작업 대상을 마지막 윈도우로 전환합니다.

```
>>> driver.switch_to.window(window_name=driver.window_handles[-1])
```

*# [참고] Selenium에서 새 탭을 열었으면 작업 대상을 해당 탭으로 이동해야 후속작업을 진행할 수 있습니다.*
*WebDriver 객체에서 크롬 개발자 도구도 하나의 윈도우로 간주되므로 크롬 개발자 도구를 닫는 것이 좋습니다.*

*하이퍼링크를 클릭하여 새 탭을 열고, 작업 대상을 새 탭으로 전환하는 코드를 스크립트(py 파일)로 한 번에*
*실행하면 새 탭이 열릴 때까지 명시적으로 대기하는 코드를 추가할 필요가 있습니다.*

# [참고] 명시적 대기

- 필요한 모듈을 임포트합니다.

```
>>> from selenium.webdriver.support.ui import WebDriverWait

>>> from selenium.webdriver.support import expected_conditions as EC
```

- 최대 3초까지 특정 조건을 기다리는 WebDriverWait 객체를 생성합니다.

```
>>> wait = WebDriverWait(driver=driver, timeout=3)
```

- 새 탭이 열려 윈도우 개수가 3이 될 때까지 최대 3초까지 기다립니다.

```
>>> wait.until(method=EC.number_of_windows_to_be(num_windows=3))
```

*# [주의] 위 코드를 실행하면 최대 3초까지 기다렸다가 에러를 일으키는데, 이것은 의도된 에러입니다.*
*스크립트를 한 번에 실행할 때 명시적 대기 코드를 추가하지 않으면 의도한 동작을 수행하지 못할 수 있습니다.*

# 검색어 입력

- 필요한 모듈을 임포트합니다.

```
>>> from selenium.webdriver.common.keys import Keys
```

*# [참고] Selenium에서 enter, back space 등 특수 키를 입력할 때 사용하는 Keys 클래스를 임포트합니다.*

- 크롬 브라우저에서 검색창 관련 첫 번째 HTML 요소를 찾아 search_box에 할당합니다.

```
>>> search_box = driver.find_element(by=By.NAME, value='sectionBlogQuery')
```

*# [참고] value 매개변수에 검색창 관련 HTML 요소의 name 속성값을 문자열로 지정합니다.*

- search_box에 검색어를 입력하고 엔터 키를 누릅니다.

```
>>> search_box.send_keys('제주여행', Keys.ENTER)
```

# [참고] 주요 Keys 소개

| 구분 | 상세 내용 | 동일 기능 문자열 |
|---|---|---|
| Keys.ENTER | 엔터 키를 한 번 누릅니다. | '\ue007' |
| Keys.BACKSPACE | 백스페이스 키를 한 번 누릅니다. | '\ue003' |
| Keys.SHIFT | 시프트 키를 한 번 누릅니다. | '\ue008' |
| Keys.CONTROL | 컨트롤 키를 한 번 누릅니다. | '\ue009' |
| Keys.ALT | 알트 키를 한 번 누릅니다. | '\ue00a' |
| Keys.COMMAND | 커맨드 키를 한 번 누릅니다. | '\ue03d' |
| Keys.PAGE_DOWN | 페이지를 아래로 한 번 내립니다. | '\ue00e' |
| Keys.PAGE_UP | 페이지를 위로 한 번 올립니다. | '\ue00f' |

*# [참고] 크롬 브라우저에서 현재 열려 있는 웹 페이지를 위/아래로 이동시키려면 CSS 선택자로 'body' 태그를 선택하고 send_keys 메서드 괄호 안에 해당 키를 지정합니다.*

# 페이지 이동 & 화면 새로고침

- 이전 페이지로 이동합니다.(뒤로 가기)

  ```
  >>> driver.back()
  ```

- 다음 페이지로 이동합니다.(앞으로 가기)

  ```
  >>> driver.forward()
  ```

- 현재 페이지를 새로고침합니다.

  ```
  >>> driver.refresh()
  ```

# 화면 스크린샷

- 현재 브라우저 창에 보이는 화면을 png 파일로 저장합니다.

```
>>> driver.save_screenshot(filename='test.png')
```

*# [참고] 크롬 개발자 도구나 주소창을 제외하고, 현재 웹 페이지에 표시된 영역을 캡처합니다.*

- 필요한 모듈을 임포트합니다.

```
>>> from IPython.display import Image
```

- png 파일을 주피터 노트북에 렌더링합니다.

```
>>> Image(filename='test.png')
```

# 스크롤 이동: 상대위치

- 현재 스크롤 위치에서 아래로 500 픽셀 이동합니다.

```
>>> driver.execute_script(script='window.scrollBy(0, 500);')
```

*# [참고] script 매개변수에 JavaScript 코드를 문자열로 지정합니다.*
*  window.scrollBy 함수에 x축과 y축의 픽셀 값을 설정하고, 코드 마지막에 세미콜론을 추가하는 것을 권장합니다.*

- 현재 스크롤 위치에서 위로 300 픽셀 이동합니다.

```
>>> driver.execute_script(script='window.scrollBy(0, -300);')
```

*# [참고] 픽셀 값을 양수로 지정하면 x축은 오른쪽으로, y축은 아래로 이동시킵니다.*
*  픽셀 값을 음수로 지정하면 x축은 왼쪽으로, y축은 위로 이동시킵니다.*

# 스크롤 이동: 절대위치

- 현재 웹 페이지의 전체 문서 높이를 반환하는 JavaScript 코드를 작성합니다.

```
>>> scroll_height_js = 'return document.documentElement.scrollHeight;'
```

- 현재 웹 페이지의 전체 문서 높이를 document_height에 할당합니다.

```
>>> document_height = driver.execute_script(script=scroll_height_js)
```

- 스크롤을 웹 페이지의 맨 아래로 내립니다.

```
>>> driver.execute_script(script=f'window.scrollTo(0, {document_height});')
```

- 스크롤을 웹 페이지의 맨 위로 올립니다.

```
>>> driver.execute_script(script='window.scrollTo(0, 0);')
```

# 첫 번째 블로그로 이동

- 크롬 브라우저에서 첫 번째 블로그 링크를 클릭합니다.

```
>>> driver.find_element(by=By.PARTIAL_LINK_TEXT, value='제주여행').click()
```

```
# [참고] a 태그의 링크 텍스트에 value 문자열이 포함된 HTML 요소를 찾습니다.
```

- 현재 작업 대상을 마지막 윈도우로 전환합니다.

```
>>> driver.switch_to.window(window_name=driver.window_handles[-1])
```

# iframe 이동

- HTML 요소에서 iframe 태그를 모두 찾아 items에 할당합니다.

```
>>> items = driver.find_elements(by=By.TAG_NAME, value='iframe')
```

*# [참고] find_element 메서드는 첫 번째 요소만, find_elements 메서드는 모든 요소를 선택하고 리스트로 반환합니다.*
*items 원소의 클래스는 selenium.webdriver.remote.webelement.WebElement입니다.*

- items의 원소 개수를 확인합니다.

```
>>> len(items)
```

- 현재 작업 대상을 지정한 iframe으로 전환합니다.

```
>>> driver.switch_to.frame(frame_reference='mainFrame')
```

*# [참고] iframe의 id 또는 name 속성값을 지정할 수 있습니다.*

*# [참고] 기본 페이지와 iframe 내부는 분리되어 있으므로, iframe 내부는 별도의 HTML 문서처럼 동작합니다.*
*따라서 위 코드를 실행해야 iframe 내부 요소를 찾을 수 있습니다.*

# 텍스트 수집

- 블로그 본문 관련 CSS Selector(선택자)를 css_selector에 할당합니다.

```
>>> css_selector = 'div.se-module > p > span'
```

- 블로그 본문을 포함하는 모든 HTML 요소를 찾아 items에 할당합니다.

```
>>> items = driver.find_elements(by=By.CSS_SELECTOR, value=css_selector)
```

- items의 원소 개수를 확인합니다.

```
>>> len(items)
```

- 리스트 컴프리헨션으로 items의 원소별 텍스트를 리스트로 생성합니다.

```
>>> [item.text for item in items]
```

```
# [주의] 페이지가 새로고침되거나 HTML 구조가 변경되면 기존에 찾은
  WebElement가 더 이상 유효하지 않을 수 있습니다.
  이때 StaleElementReferenceException이 발생할 수 있습니다.
```

# HTML 텍스트 처리

- 필요한 모듈을 임포트합니다.

```
>>> from bs4 import BeautifulSoup as BTS
```

- 현재 페이지의 HTML 문자열을 BeautifulSoup 객체로 파싱하고 soup에 할당합니다.

```
>>> soup = BTS(markup=driver.page_source, features='html.parser')
```

- 블로그 본문을 포함하는 HTML 요소를 모두 선택하고 items에 할당합니다.

```
>>> items = soup.select(selector=css_selector)
```

- items의 원소별 텍스트를 리스트로 생성하고 하나의 문자열로 결합합니다.

```
>>> ' '.join([item.text for item in items])
```

*# [참고] join은 문자열의 메서드이며, join 앞의 문자열은 리스트 원소를 결합할 때 구분자로 사용됩니다.*

# 윈도우 닫고 WebDriver 종료

- 새 탭을 열고 해당 윈도우로 전환합니다.

```
>>> driver.switch_to.new_window(type_hint='tab')
```

- 현재 작업 대상인 윈도우를 하나만 닫습니다.

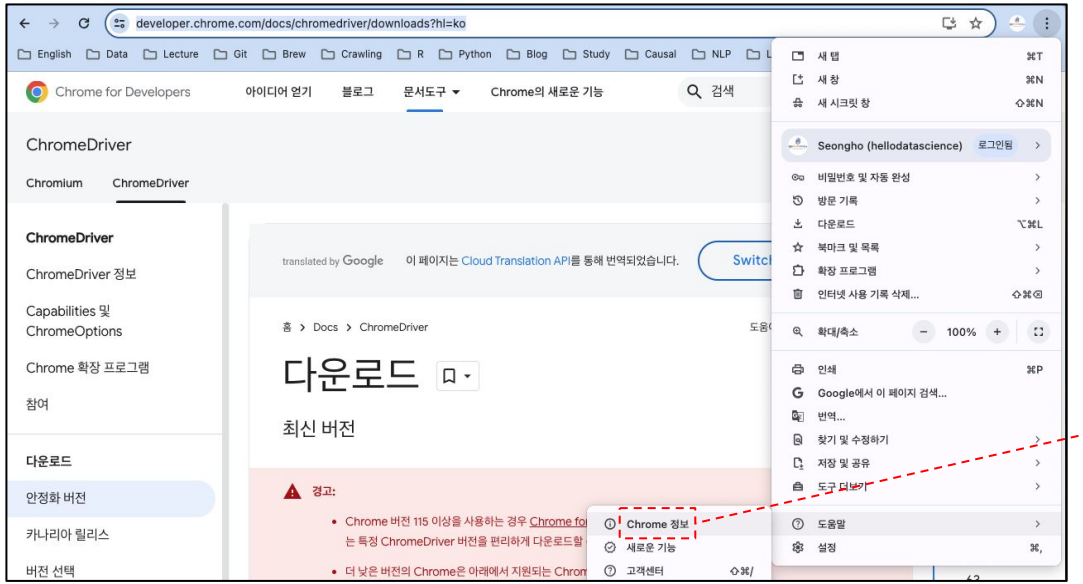
```
>>> driver.close()
```

- 모든 윈도우를 닫고 WebDriver 세션을 종료합니다.

```
>>> driver.quit()
```

# [부록 1] 크롬 드라이버 다운로드

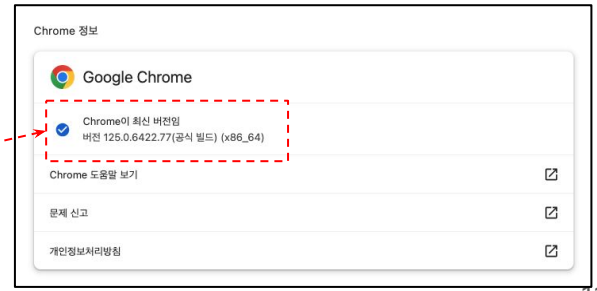
- 크롬 드라이버 다운로드 페이지로 이동하고 현재 크롬 버전을 확인합니다.
  - 경로: https://developer.chrome.com/docs/chromedriver/downloads?hl=ko




*# 삼점 메뉴( : )를 클릭합니다.*

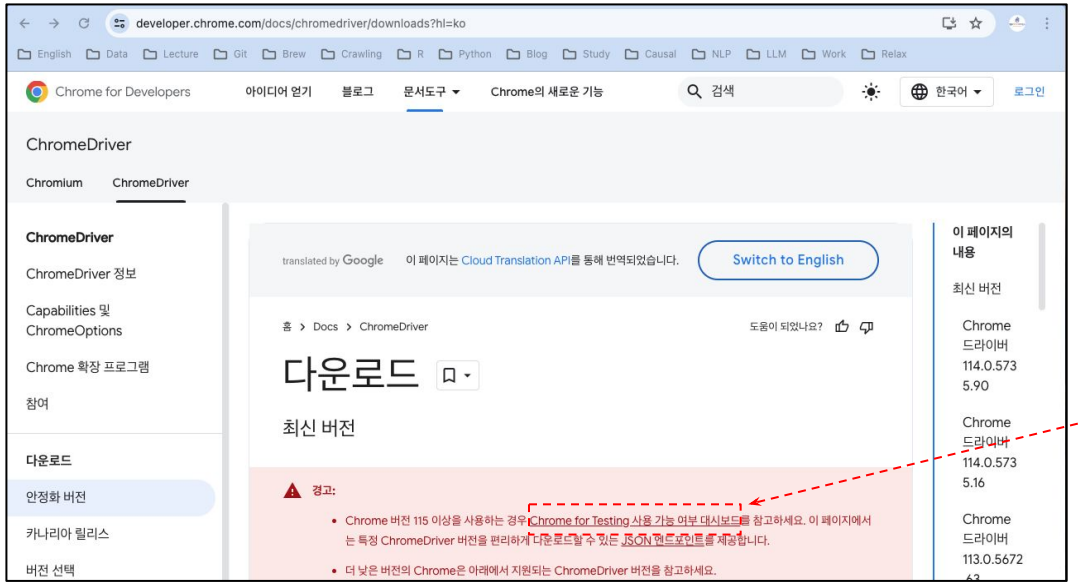
*# 메뉴에서 도움말 → Chrome 정보를 차례대로 클릭합니다.*

*# 현재 크롬 버전을 확인합니다.*



# [부록 1] 크롬 드라이버 다운로드(계속)

- 현재 사용 중인 크롬 브라우저 버전에 해당하는 크롬 드라이버를 선택합니다.
  - 경로: https://developer.chrome.com/docs/chromedriver/downloads?hl=ko




*# [참고] 현재 크롬 버전이 115 이상이면 여기를 클릭하세요.*

*# [참고] 현재 크롬 버전이 114 이하이면 아래 목록에서 선택합니다.*

# [부록 1] 크롬 드라이버 다운로드(계속)

- 현재 사용 중인 컴퓨터의 운영체제에 맞는 링크에 접속하여 크롬 드라이버 압축 파일을 다운로드하고 압축을 해제합니다. *# [참고] 압축 파일은 Downloads 폴더에 있습니다.*

**Stable**

Version: 125.0.6422.76 (r1287751)

| Binary | Platform | URL | HTTP status |
|---|---|---|---|
| chrome | linux64 | https://storage.googleapis.com/chrome-for-testing-public/125.0.6422.76/linux64/chrome-linux64.zip | 200 |
| chrome | mac-arm64 | https://storage.googleapis.com/chrome-for-testing-public/125.0.6422.76/mac-arm64/chrome-mac-arm64.zip | 200 |
| chrome | mac-x64 | https://storage.googleapis.com/chrome-for-testing-public/125.0.6422.76/mac-x64/chrome-mac-x64.zip | 200 |
| chrome | win32 | https://storage.googleapis.com/chrome-for-testing-public/125.0.6422.76/win32/chrome-win32.zip | 200 |
| chrome | win64 | https://storage.googleapis.com/chrome-for-testing-public/125.0.6422.76/win64/chrome-win64.zip | 200 |
| chromedriver | linux64 | https://storage.googleapis.com/chrome-for-testing-public/125.0.6422.76/linux64/chromedriver-linux64.zip | 200 |
| chromedriver | mac-arm64 | https://storage.googleapis.com/chrome-for-testing-public/125.0.6422.76/mac-arm64/chromedriver-mac-arm64.zip | 200 |
| chromedriver | mac-x64 | https://storage.googleapis.com/chrome-for-testing-public/125.0.6422.76/mac-x64/chromedriver-mac-x64.zip | 200 |
| chromedriver | win32 | https://storage.googleapis.com/chrome-for-testing-public/125.0.6422.76/win32/chromedriver-win32.zip | 200 |
| chromedriver | win64 | https://storage.googleapis.com/chrome-for-testing-public/125.0.6422.76/win64/chromedriver-win64.zip | 200 |

# [부록 1] 크롬 드라이버 경로 지정

- 필요한 모듈을 임포트합니다.

```
>>> from selenium.webdriver.chrome.service import Service
```

- 크롬 드라이버 실행 파일 경로를 설정합니다.(Windows 및 MacOS)

```
>>> path = r'C:\Users\Account\Downloads\chromedriver-win64\chromedriver.exe'
```

```
>>> path = '/Users/Account/Downloads/chromedriver-mac-x64/chromedriver'
```

- 크롬 브라우저를 제어하기 위한 WebDriver 객체를 생성하고 driver에 할당합니다.

```
>>> driver = webdriver.Chrome(service=Service(executable_path=path))
```

*# [참고] service 매개변수에 크롬 드라이버 파일을 저장한 경로를 지정합니다.*
*service 매개변수를 생략하면 Selenium Manager가 필요한 드라이버를 자동으로 찾거나 내려받아 실행합니다.*

# [부록 2] 크롬 브라우저의 옵션 설정

- 필요한 모듈을 임포트합니다.

```
>>> from selenium.webdriver.chrome.options import Options
```

- 크롬 브라우저의 옵션을 설정하는 객체를 생성합니다.

```
>>> options = Options()
```

- GUI 없이 백그라운드에서 실행하는 headless 옵션을 추가합니다.

```
>>> options.add_argument('--headless=new')
```

- 크롬 브라우저를 제어하기 위한 WebDriver 객체를 생성하고 driver에 할당합니다.

```
>>> driver = webdriver.Chrome(options=options)
```

# [부록 2] 크롬 브라우저 옵션 종류

| 옵션 | 상세 내용 |
|---|---|
| '--headless=new' | 크롬 브라우저를 GUI 없이 백그라운드에서 실행합니다.(headless 모드) |
| '--disable-gpu' | 크롬 브라우저의 GPU 렌더링 문제를 줄이기 위해 사용합니다. |
| '--window-size=1920,1080' | 크롬 브라우저의 창 크기를 지정합니다.(가로 픽셀, 세로 픽셀 순) |
| '--start-maximized' | 크롬 브라우저를 시작할 때 창을 최대로 설정합니다. |
| '--disable-popup-blocking' | 크롬 브라우저의 팝업 차단을 비활성화합니다. |
| '--no-sandbox' | 크롬 브라우저를 Linux/Container 환경의 권한 문제를 우회할 때 사용합니다. |
| '--user-agent=Mozilla/5.0' | 크롬 브라우저의 User-Agent를 지정합니다. |
| '--incognito' | 크롬 브라우저를 시크릿 모드로 실행합니다.(기존 쿠키와 세션 영향 줄이기 위함) |
| '--lang=ko-KR' | 크롬 브라우저의 언어를 설정합니다. |

# [부록 3] 화면 해상도 확인

- 필요한 모듈을 임포트합니다.

  ```
  >>> import pyautogui
  ```

- 스크린의 가로, 세로 크기를 width, height에 각각 할당합니다.

  ```
  >>> width, height = pyautogui.size()
  ```

- 화면 해상도(가로×세로)를 출력합니다.

  ```
  >>> print(f'화면 해상도: {width} × {height}')
  ```

# [부록 4] 크롬 브라우저 창 제어

- 현재 실행 중인 크롬 브라우저 창의 가로, 세로 픽셀 크기를 확인합니다.

```
>>> driver.get_window_size()  # [참고] 사용 중인 모니터의 화면비율과 해상도에 따라 크롬 브라우저의 가로, 세로 픽셀 크기가 제한될 수 있습니다.
```

- 현재 실행 중인 크롬 브라우저 창의 가로, 세로 픽셀 크기를 변경합니다.

```
>>> driver.set_window_size(width=width // 2, height=height)
```

- 현재 실행 중인 크롬 브라우저 창의 위치를 조정합니다.

```
>>> driver.set_window_position(width // 2, 0)
```

# 데이터 입출력

# 실습 데이터셋 소개

- 데이터 입출력 실습 데이터셋은 iris 데이터를 5가지 형태로 저장한 파일입니다.
  - iris.xlsx, iris.csv, iris.tsv, iris.json, iris.parquet
- 정형 데이터 전처리 실습 데이터셋은 공공데이터포털에서 제공하는 국토교통부 아파트 매매 실거래가 중 서울특별시에서 2021~2025년에 거래된 데이터입니다.
  - APT_Price_Seoul_Merged_2021~2025.xlsx, APT_Price_Seoul_Merged_2021~2025.csv
- 비정형 데이터 전처리 실습 데이터셋은 네이버 쇼핑에 등록된 갤럭시북 상품 리뷰를 수집한 데이터입니다.
  - Galaxy_Book_Review_20250131.xlsx, Galaxy_Book_Review_20250131_Corrected.xlsx

# 작업 경로

- Python에서 외부 파일을 입출력할 때, 입출력 함수의 괄호 안에 '경로/파일명.확장자' 형태의 문자열을 인수로 지정해야 합니다.
    - 입출력하려는 파일이 현재 작업 경로working directory에 있으면 경로를 생략할 수 있습니다.
- 현재 작업 경로는 작성 중인 Jupyter Notebook 파일이 저장된 폴더입니다.

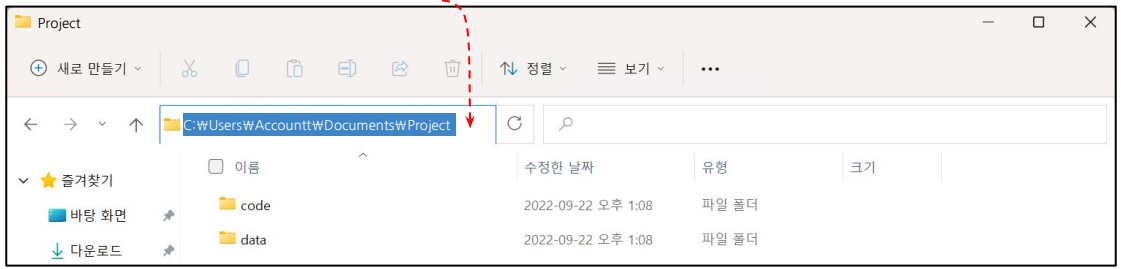
| | |
|---|---|
| Windows | 'C:\Users\Account\Documents\Project\code' |
| MacOS | '/Users/Account/Documents/Project/code' |

```
# [주의] Windows는 역슬래시(\), MacOS는 슬래시(/)를 경로 구분자로 사용합니다. 문자열에 역슬래시를 한 번 사용하면
  에러를 일으킬 수 있으므로 역슬래시 앞에 역슬래시를 추가(이스케이프 처리)하거나 슬래시로 변경해주어야 합니다.

  [참고] 경로 구분자가 역슬래시일 때 따옴표 앞에 r을 추가하면 raw string이 됩니다.
  raw string은 문자열 안에 있는 역슬래시를 메타문자가 아닌 문자 그대로 인식하므로 에러가 발생하지 않습니다.
```

# [참고] Windows 탐색기에서 경로 복사

- Windows 탐색기 주소창에서 오른쪽 공백을 마우스로 클릭하면 경로를 복사할 수 있습니다.




- 경로 구분자가 역슬래시인 경로를 문자열로 생성하면 에러를 일으킵니다.

```
>>> 'C:\Users\Account\Documents\Project'
```

*# [참고] 역슬래시 앞에 역슬래시를 추가(이스케이프 처리) 또는 역슬래시를 슬래시로 변경하면 에러를 일으키지 않습니다.*

- 따옴표 앞에 r을 추가한 raw string은 역슬래시를 메타문자로 인식하지 않습니다.

```
>>> r'C:\Users\Account\Documents\Project'
```

# 절대 경로 vs 상대 경로

| 구분 | 상세 내용 | 우편물 주소 |
|---|---|---|
| 절대<br>경로 | - 절대 경로는 파일/폴더의 시작부터 끝까지 전체 경로를 표시한 것입니다.<br>- 절대 경로는 현재 사용 중인 컴퓨터에서 유일한 경로입니다.<br>- 절대 경로의 길이가 너무 길면 코딩할 때 다소 불편할 수 있습니다.<br>- 현재 작업 경로와 상관 없이 항상 같은 결과를 얻을 수 있습니다. | [일반 우편]<br>서울특별시 강남구<br>역삼동 1번지 ○○기업<br>△△팀 홍길동 과장님 |
| 상대<br>경로 | - 상대 경로는 현재 작업 중인 위치를 기준으로 한 상대적인 경로입니다.<br>- 상대 경로에서 '.'은 현재 폴더, '..'은 상위(부모) 폴더를 의미합니다.<br>- 짧은 경로만으로도 파일/폴더를 지정할 수 있으므로 코딩할 때 편리합니다.<br>- 현재 작업 경로가 달라지면 코드를 실행할 때 에러가 발생할 수 있습니다. | [사내 행낭]<br>△△팀 홍길동 과장님 |

# 필요한 모듈 임포트

- 필요한 모듈을 임포트합니다.

```
>>> import os # [참고] os는 폴더와 파일을 처리하는 함수를 포함하는 표준 라이브러리입니다.

>>> import numpy as np

>>> import pandas as pd
```

# [참고] 폴더와 파일 관련 주요 함수

| 구분 | 상세 내용 |
|---|---|
| os.getcwd() | 현재 작업 경로를 문자열로 반환합니다. |
| os.chdir(path) | 작업 경로를 변경합니다. |
| os.mkdir(path) | 새 폴더를 생성합니다. *# [참고] 생성하려는 폴더가 있으면 에러를 일으킵니다. os.makedirs 함수는 에러를 제어하는 옵션을 제공합니다.* |
| os.rmdir(path) | 빈 폴더를 삭제합니다. *# [참고] 빈 폴더가 아니면 에러를 일으킵니다.* |
| os.listdir(path) | 지정한 경로에 있는 폴더명과 파일명을 문자열 리스트로 반환합니다. |
| os.remove(path) | 지정한 파일을 삭제합니다. *# [참고] 폴더명을 지정하면 에러를 일으킵니다.* |
| os.rename(src, dst) | 폴더명 또는 파일명을 변경합니다. |
| os.path.join(*paths) | 여러 경로명을 하나로 결합합니다. *# [참고] 경로명 사이에 운영체제에 맞는 구분자를 추가합니다.* |

# [참고] 폴더와 파일 관련 주요 함수(계속)

| 구분 | 상세 내용 |
|---|---|
| os.path.exists(path) | 지정한 경로의 존재 여부를 True, False로 반환합니다. |
| os.path.isfile(path) | 지정한 경로가 파일이면 True, 파일이 아니면 False를 반환합니다. |
| os.path.isdir(path) | 지정한 경로가 폴더이면 True, 파일이 아니면 False를 반환합니다. |
| os.path.abspath(path) | 지정한 경로에 대한 절대 경로를 반환합니다. |
| shutil.copy(src, dst) | 파일을 복사합니다. |
| shutil.copytree(src, dst) | 폴더에 있는 모든 파일과 하위 폴더를 통째로 복사합니다. |
| shutil.move(src, dst) | 파일을 옮깁니다. |
| shutil.rmtree(path) | 폴더에 있는 모든 파일과 하위 폴더를 통째로 삭제합니다. |

# 작업 경로 확인 및 변경

- 현재 작업 경로를 확인합니다.

```
>>> os.getcwd() # [참고] 현재 작업 경로는 code 폴더입니다.
                # 현재 작업 경로는 현재 작성 중인 Jupyter Notebook 파일이 있는 폴더입니다.
```

- 데이터 파일이 있는 폴더로 작업 경로를 변경합니다.(절대 경로)

```
>>> os.chdir(path='data 폴더의 절대 경로명을 지정하세요.') # [참고] 현재 작업 경로에서 가까운
                                                          # 폴더를 지정할 때 사용합니다.
>>> os.getcwd() # [참고] 현재 작업 경로는 data 폴더입니다.
```

- 탐색기 또는 파인더에서 복사한 절대 경로로 작업 경로를 변경합니다.

```
>>> os.chdir(path=r'탐색기 또는 파인더에서 복사한 절대 경로명을 붙여넣으세요.')
>>> os.getcwd() # [참고] 현재 작업 경로에서 멀리 있는 폴더를 지정할 때 사용하면 편리합니다.
```

# 작업 경로 확인 및 변경(계속)

- 현재 작업 경로의 부모 폴더로 작업 경로를 변경합니다.(상대 경로)

```
>>> os.chdir(path='..') # [참고] '.'은 현재 폴더, '..'은 부모 폴더입니다.
>>> os.getcwd() # [참고] 현재 작업 경로는 Project 폴더입니다.
```

- 현재 작업 경로의 자식 폴더로 작업 경로를 변경합니다.(상대 경로)

```
>>> os.chdir(path='./code') # [참고] './'를 생략할 수 있습니다.
>>> os.getcwd() # [참고] 현재 작업 경로는 code 폴더입니다.
```

- 현재 작업 경로의 형제 폴더로 작업 경로를 변경합니다.(상대 경로)

```
>>> os.chdir(path='../data') # [주의] '../'는 생략하면 안됩니다.
```

# 작업 경로에 있는 폴더명과 파일명 확인

- 현재 작업 경로에 새 폴더를 생성합니다.

```
>>> os.mkdir(path='temp') # [참고] './temp'에서 './'를 생략한 것입니다.
```

- 현재 작업 경로에 있는 폴더명과 파일명을 확인합니다.

```
>>> os.listdir() # [참고] Windows에서는 폴더명과 파일명을 오름차순 정렬된 리스트를 반환하지만, MacOS에서는
                 # 오름차순 정렬되어 있지 않으므로 오름차순 정렬이 필요하면 sorted 함수를 중첩 사용합니다.
```

- 현재 작업 경로에 있는 빈 폴더를 삭제합니다.

```
>>> os.rmdir(path='temp') # [참고] 빈 폴더가 아니면 에러를 일으킵니다.
                          # [주의] 삭제한 폴더는 휴지통을 거치지 않으므로 되돌릴 수 없습니다.
```

- 현재 작업 경로에 있는 폴더명과 파일명을 확인합니다.

```
>>> os.listdir() # [참고] 현재 작업 경로에 'temp' 폴더가 없습니다.
```

# xlsx 파일 입출력

- xlsx 파일을 읽고 데이터프레임을 생성합니다.

```
>>> df1 = pd.read_excel(io='iris.xlsx')
```

*# [참고] pandas 라이브러리로 xls 파일을 입출력할 때 xlrd, xlsx 파일을 입출력할 때 openpyxl 엔진을 사용합니다.*

- df1의 처음 5행을 확인합니다.

```
>>> df1.head()
```

- df1을 xlsx 파일로 저장합니다.

```
>>> df1.to_excel(excel_writer='test.xlsx', index=False)
```

- 현재 작업 경로에 있는 폴더명과 파일명을 확인합니다.

```
>>> os.listdir()
```

# csv 파일 입출력

- csv 파일을 읽고 데이터프레임을 생성합니다.

```
>>> df2 = pd.read_csv(filepath_or_buffer='iris.csv')
```

*# [참고] csv 파일은 구분자로 콤마(,)를 사용하는 텍스트 기반 저장 포맷입니다.*

- df2의 처음 5행을 확인합니다.

```
>>> df2.head()
```

- df2를 csv 파일로 저장합니다.

```
>>> df2.to_csv(path_or_buf='test.csv', index=False)
```

- 현재 작업 경로에 있는 폴더명과 파일명을 확인합니다.

```
>>> os.listdir()
```

# tsv 파일 입출력

- tsv 파일을 읽고 데이터프레임을 생성합니다.

```
>>> df3 = pd.read_csv(filepath_or_buffer='iris.tsv', sep='\t') # [참고] tsv 파일은 탭(\t)을 구분자로 사용합니다.
```

- df3의 처음 5행을 확인합니다.

```
>>> df3.head()
```

- df3를 tsv 파일로 저장합니다.

```
>>> df3.to_csv(path_or_buf='test.tsv', index=False, sep='\t')
```

- 현재 작업 경로에 있는 폴더명과 파일명을 확인합니다.

```
>>> os.listdir()
```

# json 파일 입출력

- json 파일을 읽고 데이터프레임을 생성합니다.

```
>>> df4 = pd.read_json(path_or_buf='iris.json') # [참고] json 파일은 키: 값을 원소로 갖는 텍스트 파일입니다.
```

- df4의 처음 5행을 확인합니다.

```
>>> df4.head()
```

- df4를 json 파일로 저장합니다.

```
>>> df4.to_json(path_or_buf='test.json')
```

- 현재 작업 경로에 있는 폴더명과 파일명을 확인합니다.

```
>>> os.listdir()
```

# parquet 파일 입출력

- parquet 파일을 읽고 데이터프레임을 생성합니다.

```
>>> df5 = pd.read_parquet(path='iris.parquet')  # [참고] pandas 라이브러리로 parquet 파일을 입출력할 때 pyarrow 엔진을 사용합니다.
```

- df5의 처음 5행을 확인합니다.

```
>>> df5.head()
```

- df5를 parquet 파일로 저장합니다.

```
>>> df5.to_parquet(path='test.parquet')
```

- 현재 작업 경로에 있는 폴더명과 파일명을 확인합니다.

```
>>> os.listdir()
```

# [참고] parquet 파일의 특징

- Apache Parquet는 Hadoop Distributed File System(HDFS)에서 효율적으로 사용할 수 있도록 설계된 컬럼형 저장 포맷입니다.
    - 열 기반 저장 방식을 사용하므로 데이터를 효율적으로 저장합니다.
    - 스키마 정보를 함께 저장하므로 컬럼명과 데이터 타입을 유지한 상태로 데이터를 교환할 수 있습니다.
    - 기존의 행 기반 포맷(csv, json 등)에 비해 I/O 성능과 압축률이 뛰어납니다.
    - 필요한 컬럼만 선택적으로 읽을 수 있으므로 대용량 데이터 처리 및 분석에 유리합니다.
    - Apache Spark, Hive, Impala, Drill 등 다양한 빅데이터 프레임워크와 호환됩니다.

# [참고] 특정 문자열을 포함하는 파일 삭제

- 현재 작업 경로에서 'test'를 포함하는 파일명 리스트를 생성합니다.

```
>>> filenames = [filename for filename in os.listdir() if 'test' in filename]
```

- 반복문으로 filenames에 포함된 파일을 모두 삭제합니다.

```
>>> for filename in filenames:
        os.remove(path=filename) # [참고] os.remove 함수는 한 번에 하나의 파일을 삭제합니다.
                                 # [주의] 파일명 대신 폴더명을 지정하면 에러를 일으킵니다.
```

- 현재 작업 경로에 있는 폴더명과 파일명을 확인합니다.

```
>>> os.listdir() # [참고] 'test'를 포함하는 파일을 모두 삭제했습니다.
                 # [주의] os.remove 함수는 파일을 휴지통으로 옮기지 않고 바로 삭제합니다.
```

# [참고] xlsx과 csv 파일의 차이

- xlsx과 csv 파일을 입출력할 때의 장단점을 비교한 표입니다.

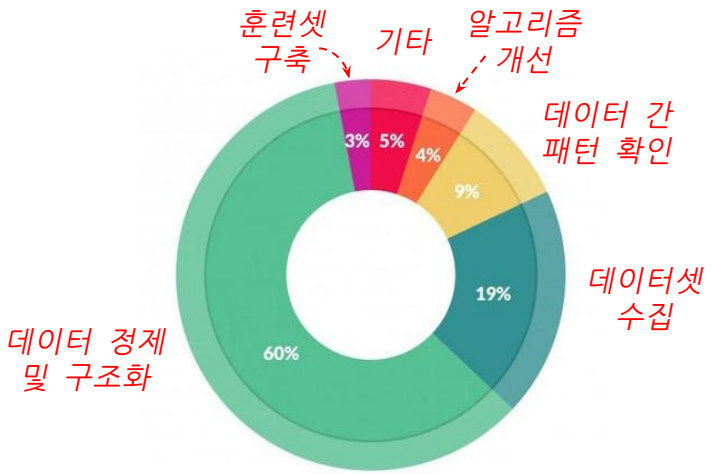
| 구분 | xlsx 파일 | csv 파일 |
|---|---|---|
| **장점** | - 문자 인코딩 방식과 관련 없습니다.<br>- xlsx 파일은 날짜/시간 셀서식을 유지하므로 해당 데이터를 'datetime64[ns]'로 읽습니다. | - 입출력 속도가 매우 빠릅니다. |
| **단점** | - 입출력 속도가 매우 느립니다. | - 문자 인코딩 방식이 인수의 기본값과 다르면 에러가 발생할 수 있습니다.<br>- 날짜시간형 데이터를 object로 생성합니다. |

- csv 파일의 단점을 보완하는 방법을 소개합니다.
  - csv 파일의 문자 인코딩 방식을 확인하고 encoding 매개변수에 지정합니다.
  - parse_dates 매개변수에 날짜시간형으로 읽을 열 이름을 리스트로 지정합니다.

# 데이터 전처리

# 데이터 전처리

- 데이터 품질이 머신러닝 알고리즘 간의 차이보다 분석 결과에 훨씬 큰 영향을 미치기 때문에 데이터 전처리의 중요성을 아무리 강조해도 지나치지 않습니다.
- 데이터 과학자는 데이터를 수집하고 정리하는 업무가 전체 과정의 약 80%를 차지한다고 응답하였습니다.
- 데이터 전처리는 다음과 같은 작업을 수행합니다.
  - 결측값과 이상치를 탐지하고 처리합니다.
  - 올바른 자료형으로 변환합니다.
  - 다양한 파생변수를 생성합니다.




*# 출처: Forbes(2016.03.23)*

# 데이터 전처리 항목

| 구분 | 상세 내용 |
|---|---|
| ① 행과 열 선택 · 삭제 | - 데이터프레임에서 필요한 행과 열을 선택하거나 불필요한 행과 열을 삭제합니다.<br>- 인덱스, 열 이름, 조건식을 기준으로 행과 열을 선택할 수 있습니다. |
| ② 열 이름 변경 | - 데이터프레임의 열 이름을 변경합니다. |
| ③ 파생변수 생성<br>(수식 · 구간화) | - 원본 데이터를 활용하여 문제 해결에 필요한 파생변수를 생성합니다.<br>- 수식 계산, 구간화 또는 인코딩 등을 활용하여 필요한 변수를 생성할 수 있습니다. |
| ④ 자료형 확인 및 변환 | - 열별 자료형을 확인하고, 분석 목적에 맞는 자료형으로 변환합니다.<br>- 특히 날짜시간 정보를 갖는 문자열은 반드시 날짜시간형으로 변환해야 합니다. |
| ⑤ 결측값 확인 및 처리 | - 열별 결측값 개수를 확인하고, 결측값 발생 원인에 맞게 처리합니다.<br>- 원본 데이터 확보, 대표값 · 보간법 등으로 대체, 행 · 열 삭제 등을 활용합니다. |
| ⑥ 데이터프레임 결합 ·<br>병합 · 형태 변환 | - 여러 데이터프레임을 행/열 방향으로 결합하거나, 조인 키를 기준으로 병합합니다.<br>- 데이터프레임을 필요에 따라 Long 또는 Wide 형태로 변환합니다. |

# 데이터 전처리 항목

① 열과 행 선택/삭제

② 열 이름 변경, 행 이름 초기화

⑤ 결측값 처리

| | 거래금액 | 지역코드 | 아파트 | 시도명 | 시군구 | 법정동 | 지번 | 건축년도 | 년 | 월 | 일 | 등기일자 | 매도자 | 전용면적 | 층 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 100500 | 11680 | 삼익대청아파트 | 서울특별시 | 강남구 | 개포동 | 12 | 1992 | 2020 | 1 | 31 | NaT | NaN | 39.53 | 13 |
| 1 | 203000 | 11680 | 개포자이 | 서울특별시 | 강남구 | 개포동 | 12-2 | 2004 | 2020 | 1 | 6 | NaT | NaN | 134.30 | 17 |
| 2 | 180000 | 11680 | 개포주공6단지 | 서울특별시 | 강남구 | 개포동 | 185 | 1983 | 2020 | 1 | 14 | NaT | NaN | 60.13 | 14 |
| 3 | 285000 | 11680 | 디에이치아너힐즈 | 서울특별시 | 강남구 | 개포동 | 1281 | 2019 | 2020 | 1 | 4 | NaT | NaN | 94.29 | 12 |
| 4 | 171000 | 11680 | 개포주공7단지 | 서울특별시 | 강남구 | 개포동 | 185 | 1983 | 2020 | 1 | 12 | NaT | NaN | 60.76 | 2 |

1. 거래금액 / 전용면적 * 3.3
2. 년 - 건축년도
3. 경과년수가 30 이상이면 '충족', 미만이면 '부족'
4. 년, 월, 일을 'yyyy-mm-dd' 형태의 문자열로 결합하고 날짜시간형으로 변환

| | 거래금액 | 단지명 | 시도명 | 시군구 | 법정동 | 지번 | 입주년도 | 전용면적 | 층 | 평당금액 | 경과년수 | 재건축 | 계약일자 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 10.05 | 삼익대청아파트 | 서울특별시 | 강남구 | 개포동 | 12 | 1992 | 39.53 | 13 | 8389.83 | 28 | 부족 | 2020-01-31 |
| 1 | 20.30 | 개포자이 | 서울특별시 | 강남구 | 개포동 | 12-2 | 2004 | 134.30 | 17 | 4988.09 | 16 | 부족 | 2020-01-06 |
| 2 | 18.00 | 개포주공6단지 | 서울특별시 | 강남구 | 개포동 | 185 | 1983 | 60.13 | 14 | 9878.60 | 37 | 충족 | 2020-01-14 |
| 3 | 28.50 | 디에이치아너힐즈 | 서울특별시 | 강남구 | 개포동 | 1281 | 2019 | 94.29 | 12 | 9974.55 | 1 | 부족 | 2020-01-04 |
| 4 | 17.10 | 개포주공7단지 | 서울특별시 | 강남구 | 개포동 | 185 | 1983 | 60.76 | 2 | 9287.36 | 37 | 충족 | 2020-01-12 |

④ 자료형 변환

③ 파생변수 생성

# 필요한 모듈 임포트

- 필요한 모듈을 임포트합니다.

```
>>> import os

>>> import numpy as np

>>> import pandas as pd
```

- pandas에서 실수를 표현하는 소수점 자리수를 설정합니다.

```
>>> pd.set_option('display.float_format', lambda x: f'{x:.2f}')
```

# 작업 경로 확인 및 변경

- 현재 작업 경로를 확인합니다.

  ```
  >>> os.getcwd()
  ```

- data 폴더로 작업 경로를 변경합니다.

  ```
  >>> os.chdir(path='../data')
  ```

- 현재 작업 경로에 있는 폴더명과 파일명을 확인합니다.

  ```
  >>> os.listdir()
  ```

# 실습 데이터셋 준비

- xlsx 파일을 읽고 apt에 할당합니다.

```
>>> %%time  # [참고] %%time은 Cell 전체 코드, %time은 오른쪽 한 줄 코드의 실행 소요 시간을 콘솔에 출력합니다.
           # [주의] %%time은 반드시 Cell의 맨 처음 행에 위치해야 하므로 주석 아래에 있으면 에러를 일으킵니다.

>>> apt = pd.read_excel(io='APT_Price_Seoul_Merged_2021~2025.xlsx')
```

- apt의 처음 5행을 확인합니다.

```
>>> apt.head()
```

- apt의 정보를 확인합니다.

```
>>> apt.info()
```

# 실습 데이터셋 준비(계속)

- csv 파일을 읽고 apt에 할당합니다.

```
>>> %%time

>>> apt = pd.read_csv(filepath_or_buffer='APT_Price_Seoul_Merged_2021~2025.csv',
                      low_memory=False, # [참고] 전체 데이터를 읽고 데이터프레임으로 저장하므로 열별로 자료형이 섞이는 문제가 발생하지 않습니다.
                      parse_dates=['등기일자']) # [참고] 날짜시간형으로 읽을 열 이름을 리스트로 지정합니다. 생략하면 object로 생성합니다.
```

- apt의 정보를 확인합니다.

```
>>> apt.info()
```

# 열 이름으로 열 선택

- 대괄호 안에 열 이름을 지정하면 시리즈로 반환합니다.

```
>>> apt['지역코드']  # [참고] 대괄호 안에 데이터프레임의 원소인 열 이름을 문자열로 지정합니다.
                    # [주의] 행을 선택하려면 반드시 loc 인덱서를 추가해야 합니다.
>>> apt.loc[:, '지역코드']
```

- 대괄호 안에 열 이름 리스트를 지정하면 데이터프레임으로 반환합니다.

```
>>> apt[['지역코드', '거래금액']] # [참고] 리스트 원소의 순서대로 데이터프레임의 열을 정렬합니다.
>>> apt.loc[:, ['지역코드', '거래금액']]
```

- 대괄호 안에 열 이름 슬라이스를 지정하면 데이터프레임으로 반환합니다.

```
>>> apt.loc[:, '거래금액':'아파트']  # [참고] 대괄호 안에 콤마를 추가하여 행과 열을 구분합니다.
                                    # [주의] loc 인덱서를 생략하면 에러를 일으킵니다.
```

# 조건에 맞는 열 선택

- apt에서 정수형인 열을 선택합니다.(불리언 인덱싱)

```
>>> apt.loc[:, apt.dtypes == 'int64'] # [주의] loc 인덱서를 생략하면 에러를 일으킵니다.
```

- apt에서 정수형 또는 실수형인 열을 선택합니다.

```
>>> apt.loc[:, (apt.dtypes == 'int64') | (apt.dtypes == 'float64')]

>>> apt.select_dtypes(include=[int, float]) # [참고] exclude 매개변수에 필요 없는 클래스를 지정하는 방식도 가능합니다.
```

- apt에서 열 이름에 '년', '월', '일'이 포함된 열을 선택합니다.

```
>>> apt.loc[:, apt.columns.str.contains(pat='년|월|일')]

# [참고] 정규 표현식에서 '|'는 or를 의미하는 메타문자입니다.
  위 패턴은 '년', '월' 또는 '일'을 의미하므로, 세 글자 중 하나를 포함하는 열 이름을 모두 선택합니다.
```

# [참고] 문자열 시리즈의 str 접근자 소개

| 구분 | 상세 내용 |
|---|---|
| sr.str[index] | sr의 각 원소(문자열)를 대괄호 안에 지정한 인덱스로 인덱싱합니다.<br>대괄호 안에 슬라이스를 지정하면 sr의 각 원소를 슬라이싱합니다. |
| sr.str.split(pat) | sr의 각 원소를 패턴으로 분리한 리스트를 시리즈로 반환합니다.<br>expand 매개변수에 True를 지정하면 시리즈를 데이터프레임으로 변환합니다. |
| sr.str.contains(pat) | sr의 각 원소에서 패턴의 존재 여부를 불리언 시리즈로 반환합니다. |
| sr.str.replace(pat, repl) | sr의 각 원소에서 패턴에 맞는 문자열을 repl에 지정한 문자열로 변경합니다.<br>정규 표현식일 때 regex 매개변수에 True를 지정합니다.(기본값: False) |
| sr.str.extract(pat) | sr의 각 원소에서 패턴에 맞는 첫 번째 문자열을 데이터프레임으로 반환합니다.<br>캡처링 그룹을 추출하므로 반드시 소괄호를 사용해야 합니다. |
| sr.str.extractall(pat) | sr의 각 원소에서 패턴의 맞는 모든 문자열을 데이터프레임으로 반환합니다.<br>캡처링 그룹을 추출하므로 반드시 소괄호를 사용해야 합니다. |

# [참고] 정규 표현식 기본 문법 1: 한 글자 패턴

| 구분 | 상세 내용 |
|---|---|
| '.' | 줄바꿈을 제외한 모든 문자(공백 포함) |
| '\\w' | 알파벳 대소문자, 숫자, 언더바, 한글 |
| '\\d' | 숫자 |
| '\\s' | 공백, '\r\n'(줄바꿈), '\t'(탭) |
| '\\W' | '\\w'의 반대 |
| '\\D' | '\\d'의 반대 |
| '\\S' | '\\s'의 반대 |
| '\|' | 문자열을 or로 지정 |

| 구분 | 상세 내용 |
|---|---|
| '[a-z]' | 알파벳 소문자(26개) |
| '[A-Z]' | 알파벳 대문자(26개) |
| '[0-9]' | 숫자(10개) |
| '[ㄱ-ㅎ]' | 한글 단자음, 쌍자음, 겹자음(30개) |
| '[ㅏ-ㅣ]' | 한글 단모음, 이중모음(21개) |
| '[가-힣]' | 한글 완성형(11,172개) |
| '[一-龥]' | 한자(20,902개) |
| '[^가-힣]' | 대괄호 안 문자를 제외한 모든 문자 |

*# [참고] 온점(.)과 같이 특정 기능을 갖는 문자를 '메타문자'라고 합니다.*
*기본 한자는 [\u4E00-\u9FFF], CJK 확장 A는 [\u3400-\u4DBF]로 지정할 수 있습니다.*

# [참고] 정규 표현식 기본 문법 2: 수량자 외

| 구분 | 상세 내용 |
|---|---|
| '+' | 앞 패턴이 1~∞ 연속 일치 |
| '*' | 앞 패턴이 0~∞ 연속 일치 |
| '?' | 앞 패턴이 0 또는 1번 일치 |
| '{n}' | 앞 패턴이 정확하게 n번 연속 일치 |
| '{n,m}' | 앞 패턴이 n~m번 연속 일치(n < m) |
| '{n,}' | 앞 패턴이 n~∞ 연속 일치 |
| '\' | 메타문자의 기능 해제(이스케이프) |
| '^' | 문자열의 시작 위치 고정(앵커) |
| '$' | 문자열의 끝 위치 고정(앵커) |

| 구분 | 상세 내용 |
|---|---|
| '\\b' | 단어의 경계 여부 검사(단어 경계) |
| '\\B' | \\b의 반대(비단어 경계) |
| '()' | 일치하는 패턴 저장(캡처링 그룹) |
| '(?:)' | 패턴 저장하지 않음(비캡처링 그룹) |
| '\\n' | n번째 캡처링 그룹 재탐색(역참조) |
| 'A(?=B)' | B 앞에 오는 A 패턴 매칭(긍정 전방탐색) |
| '(?<=A)B' | A 뒤에 오는 B 패턴 매칭(긍정 후방탐색) |
| 'A(?!B)' | ?=의 반대(부정 전방탐색) |
| '(?<!A)B' | ?<=의 반대(부정 후방탐색) |

# [참고] 문자열 시리즈 처리: str 접근자

- 정규 표현식 실습용 시리즈를 생성합니다.

```
>>> nums = pd.Series(data=['M:010-2345-6789', 'T:02-345-9876', '001231-7890123'])
```

- nums의 원소에서 처음 두 글자를 잘라낸 문자열 시리즈를 반환합니다.

```
>>> nums.str[:2] # [참고] nums.str.slice(start=0, stop=2)를 실행한 결과와 같습니다.
```

- nums의 원소를 특정 패턴으로 분리하여 리스트를 원소로 갖는 시리즈를 반환합니다.

```
>>> nums.str.split(pat=':|-') # [참고] pat 매개변수를 생략하면 모든 공백으로 분리합니다.
```

- nums의 원소를 특정 패턴으로 분리한 리스트의 두 번째 원소를 선택합니다.

```
>>> nums.str.split(pat=':|-').str[1]
```

# [참고] 문자열 시리즈 처리: str 접근자(계속)

- nums의 원소에서 특정 패턴 포함 여부를 원소로 갖는 시리즈를 반환합니다.

```
>>> nums.str.contains(pat='010')
```

- nums의 원소에서 특정 패턴을 원하는 문자열로 변경한 시리즈를 반환합니다.

```
>>> nums.str.replace(pat='-', repl='')
```

- nums의 원소에서 시작 위치에 있는 숫자를 추출한 시리즈를 반환합니다.

```
>>> nums.str.extract(pat='(^[0-9]+)') # [주의] str.extract 메서드는 정규 표현식 패턴을 소괄호로 감싸지 않으면 에러를 일으킵니다.
```

- nums의 원소에서 마지막 위치에 있는 숫자를 추출한 시리즈를 반환합니다.

```
>>> nums.str.extract(pat='([0-9]+$)')
```

# 조건에 맞는 행 선택: 연속형 변수

- 거래금액이 200억 이상이면 True, 아니면 False인 cond를 생성합니다.

```
>>> cond = (apt['거래금액'] >= 2_000_000) # [참고] Python에서는 숫자의 가독성을 높이기 위해 밑줄(_) 구분자를 지원합니다.
```

- 거래금액이 200억 이상인 행을 선택합니다.

```
>>> apt.loc[cond, :] # [참고] 데이터프레임에서 행을 불리언 인덱싱할 때 loc 인덱서를 생략할 수 있습니다.
                     # loc 인덱서를 생략하려면 대괄호 안에 cond만 지정합니다.
```

- 거래금액이 200억 미만이고 층이 65 이상이면 True, 아니면 False인 cond를 생성합니다.

```
>>> cond = (apt['거래금액'] < 2_000_000) & (apt['층'] >= 65)
```

- 거래금액이 200억 미만이고 층이 65 이상인 행을 선택합니다.

```
>>> apt.loc[cond, :] # [참고] 시리즈로 논리곱 연산을 실행할 때 비트 연산자를 사용합니다.
                     # [주의] 비트 연산자 앞뒤 코드를 반드시 소괄호로 감싸야 합니다.
```

# [참고] 시리즈의 비교 연산 메서드 소개

| 구분 | 상세 내용 |
|---|---|
| sr.gt(other=value) | sr 원소가 value 초과면 True, 아니면 False인 불리언 시리즈를 반환합니다.<br>gt는 'greater than'이므로 (sr > value)를 실행한 결과와 같습니다. |
| sr.ge(other=value) | sr 원소가 value 이상이면 True, 아니면 False인 불리언 시리즈를 반환합니다.<br>ge는 'greater than or equal to'이므로 (sr >= value)를 실행한 결과와 같습니다. |
| sr.lt(other=value) | sr 원소가 value 미만이면 True, 아니면 False인 불리언 시리즈를 반환합니다.<br>lt는 'less than'이므로 (sr < value)를 실행한 결과와 같습니다. |
| sr.le(other=value) | sr 원소가 value 이하면 True, 아니면 False인 불리언 시리즈를 반환합니다.<br>le는 'less than or equal to'이므로 (sr <= value)를 실행한 결과와 같습니다. |
| sr.eq(other=value) | sr 원소가 value 같으면 True, 아니면 False인 불리언 시리즈를 반환합니다.<br>eq는 'equal to'이므로 (sr == value)를 실행한 결과와 같습니다. |
| sr.ne(other=value) | sr 원소가 value 다르면 True, 아니면 False인 불리언 시리즈를 반환합니다.<br>ne는 'not equal to'이므로 (sr != value)를 실행한 결과와 같습니다. |

# [참고] 시리즈의 비교 연산 메서드 활용법

- 비교 연산 메서드를 실행한 결과가 시리즈이므로 두 조건을 동시에 만족하는 행을 선택할 때 개별 조건을 소괄호로 감싸지 않아도 됩니다.

```
>>> apt['거래금액'].lt(other=2_000_000) & apt['층'].ge(other=65)
```

- 층이 10 이상 20 이하면 True, 아니면 False인 불리언 시리즈를 반환합니다.

```
>>> apt['층'].ge(other=10) & apt['층'].le(other=20)
```

- 위 코드를 between 메서드로 대신할 수 있습니다.

```
>>> apt['층'].between(left=10, right=20, inclusive='both')
# [참고] inclusive 매개변수에 양쪽 경계 포함 여부를 'both'(기본값), 'neither', 'left', 'right' 중에서 지정합니다.
```

# 조건에 맞는 행 선택: 범주형 변수

- 시군구가 '강남구'인 행 개수를 확인합니다.

```
>>> cond = apt['시군구'].eq(other='강남구')

>>> apt.loc[cond, :].shape # [참고] 데이터프레임의 shape 속성은 행 개수와 열 개수를 튜플로 반환합니다.
```

- 시군구가 '강남구', '서초구' 또는 '송파구'인 행 개수를 확인합니다.

```
>>> cond1 = apt['시군구'].eq(other='강남구')

>>> cond2 = apt['시군구'].eq(other='서초구')

>>> cond3 = apt['시군구'].eq(other='송파구')

>>> apt.loc[cond1 | cond2 | cond3, :].shape
```

# 조건에 맞는 행 선택: 범주형 변수(계속)

- isin 메서드는 시리즈 원소가 리스트에 있으면 True, 없으면 False를 반환합니다.

```
>>> cond = apt['시군구'].isin(values=['강남구', '서초구', '송파구'])
>>> apt.loc[cond, :].shape
```

*# [주의] 리스트 원소는 시리즈 원소와 정확하게 일치해야 합니다.*
*'강남', '서초', '송파'처럼 부분 문자열로 지정하면 안됩니다.*

- str.contains 메서드는 시리즈 원소에 패턴 포함 여부를 True 또는 False로 반환합니다.

```
>>> cond = apt['시군구'].str.contains(pat='(강남|서초|송파)구')
>>> apt.loc[cond, :].shape
```

*# [참고] pat 매개변수에 지정한 정규 표현식 패턴이 시리즈 원소의*
*부분 문자열과 일치하면 True를 반환합니다.*

# 열 이름으로 열 삭제

- apt에서 열 이름으로 삭제한 결과를 확인합니다.

```
>>> apt.drop(columns='지역코드') # [주의] columns 매개변수를 생략하면 인덱스에서 찾습니다.
                                 # [참고] 삭제할 열이 두 개 이상이면 리스트로 지정합니다.
```

- apt의 처음 5행을 확인합니다.

```
>>> apt.head() # [참고] apt에는 여전히 지역코드가 포함되어 있습니다.
```

- apt에서 열 이름으로 삭제한 결과를 apt에 재할당합니다.

```
>>> apt = apt.drop(columns=['지역코드', '매도자']) # [참고] 삭제할 열이 두 개 이상이면 반드시
                                                   # 리스트로 지정해야 합니다.
```

- apt의 처음 5행을 확인합니다.

```
>>> apt.head() # [참고] apt에서 지역코드와 매도자가 삭제되었습니다.
```

# 열 이름 변경

- apt의 열 이름을 확인합니다.

```
>>> apt.columns
```

- apt에서 일부 열 이름을 변경합니다.

```
>>> apt = apt.rename(columns={'아파트': '단지명', '건축년도': '입주년도',
                              '년': '계약년도', '월': '계약월', '일': '계약일'})
```

*# [참고] '기존 이름': '새 이름' 형태로 설정합니다.*
*  [주의] columns 매개변수를 생략하면 안됩니다!*

- apt의 처음 5행을 확인합니다.

```
>>> apt.head() # [참고] 열 이름이 변경되었습니다.
```

# 파생변수 생성

- 거래금액을 전용면적으로 나누고 3.3을 곱한 평당금액을 생성합니다.

  ```
  >>> apt['평당금액'] = apt['거래금액'] / apt['전용면적'] * 3.3
  ```

- 거래금액을 10000으로 나누어 거래금액의 단위를 만원에서 억원으로 변경합니다.

  ```
  >>> apt['거래금액'] = apt['거래금액'] / 10_000
  ```

- 계약년도에서 입주년도를 뺀 경과년수를 생성합니다.

  ```
  >>> apt['경과년수'] = apt['계약년도'] - apt['입주년도']
  ```

- apt의 처음 5행을 확인합니다.

  ```
  >>> apt.head() # [참고] 데이터프레임에 없는 열 이름에 시리즈를 할당하면 데이터프레임의 오른쪽에 추가됩니다.
  ```

# 구간화

- 연속형 변수를 특정 기준에 따라 두 개 이상의 구간으로 나누어 범주형 변수로 변환하는 것을 구간화[binning]라고 합니다. *# [참고] 구간화는 비선형, 이상치 및 결측값 문제를 처리할 수 있습니다.*

- 경과년수가 재건축 연한 기준인 30년 이상이면 True, 아니면 False인 cond를 생성합니다.

```
>>> cond = apt['경과년수'].ge(other=30)
```

- 경과년수가 30년 이상이면 '충족', 아니면 '부족'인 재건축을 생성합니다.

```
>>> apt['재건축'] = np.where(cond, '충족', '부족')
```

*# [참고] np.where 함수에 조건, True일 때 반환할 값, False일 때 반환할 값을 인수로 지정합니다.*

*np.where 함수에 첫 번째 인수만 지정하면 조건이 True인 정수 인덱스를 배열로 반환합니다.*

- apt의 처음 5행을 확인합니다.

```
>>> apt.head()
```

# 열별 자료형 변환

- apt의 열별 자료형을 확인합니다.

```
>>> apt.dtypes # [참고] 계약년도, 계약월, 계약일의 자료형은 int64입니다.
```

- 계약년도, 계약월, 계약일을 문자열로 각각 변환합니다.

```
>>> apt['계약년도'] = apt['계약년도'].astype(dtype=str)
>>> apt['계약월'] = apt['계약월'].astype(dtype=str)
>>> apt['계약일'] = apt['계약일'].astype(dtype=str)
```

- apt의 열별 자료형을 확인합니다.

```
>>> apt.dtypes # [참고] 계약년도, 계약월, 계약일의 자료형을 object로 변환했습니다.
```

# 날짜시간형 변수 생성

- 계약년도, 계약월, 계약일의 원소 사이에 하이픈을 추가한 계약일자를 생성합니다.

```
>>> apt['계약일자'] = apt['계약년도'] + '-' + apt['계약월'] + '-' + apt['계약일']
```

```
# [참고] 날짜시간 기본형 문자열('yyyy-mm-dd')로 결합합니다.
```

- apt에서 계약일자를 날짜시간형으로 변환합니다.

```
>>> apt['계약일자'] = pd.to_datetime(arg=apt['계약일자'], errors='coerce')
```

```
# [참고] 날짜시간 기본형이 아닌 문자열을 지정하면 결측값을 반환합니다.
```

- apt의 열별 자료형을 확인합니다.

```
>>> apt.dtypes # [참고] 계약일자를 datetime64[ns]로 변환했습니다.
```

# [참고] 날짜시간형 시리즈의 dt 접근자 소개

| 구분 | 상세 내용 |
|---|---|
| sr.dt.year | sr의 원소에서 년(year)을 정수형 시리즈로 반환합니다. |
| sr.dt.month | sr의 원소에서 월(month)을 정수형 시리즈로 반환합니다. |
| sr.dt.day | sr의 원소에서 일(day)을 정수형 시리즈로 반환합니다. |
| sr.dt.hour | sr의 원소에서 시(hour)을 정수형 시리즈로 반환합니다. |
| sr.dt.minute | sr의 원소에서 분(minute)을 정수형 시리즈로 반환합니다. |
| sr.dt.second | sr의 원소에서 초(second)를 정수형 시리즈로 반환합니다. |
| sr.dt.dayofweek | sr의 원소에서 요일을 정수형 시리즈로 반환합니다.(Monday: 0, Sunday: 6) |
| sr.dt.day_name() | sr의 원소에서 요일을 문자열 시리즈로 반환합니다.(로케일에 따라 다름) |
| sr.dt.strftime() | sr의 원소에서 지정한 날짜시간 포맷의 문자열 시리즈로 반환합니다. |

# [참고] 날짜시간 주요 포맷

| 구분 | 상세 내용 | 예시 |
|---|---|---|
| %Y | 연도(4자리) | '2025' |
| %y | 연도(2자리) | '25' |
| %m | 월(정수, 01~12) | '12' |
| %B | 월(Locale 전체) | 'December' |
| %b | 월(Locale 단축) | 'Dec' |
| %d | 일(정수, 01~31) | '31' |
| %j | 연중 일의 순번(001~366) | '365' |
| %H | 시(24시간, 00~23) | '01' |
| %I | 시(12시간, 01~12) | '01' |

| 구분 | 상세 내용 | 예시 |
|---|---|---|
| %p | AM/PM 표기 | 'AM' |
| %M | 분(정수, 00~59) | '02' |
| %S | 초(정수, 00~59) | '03' |
| %f | 백만 분의 1초 | '456789' |
| %s | 유닉스 시간UNIX time | '1703952123' |
| %W | 연중 주의 순번(00~52) | '52' |
| %w | 주중 요일의 순번(0~6) | '0' |
| %A | 요일(Locale 전체) | 'Sunday' |
| %a | 요일(Locale 단축) | 'Sun' |

# [참고] 날짜시간형 시리즈 처리: dt 접근자

- 계약일자에서 요일을 정수형 시리즈로 반환합니다.

```
>>> apt['계약일자'].dt.dayofweek  # [참고] Monday는 0, Sunday는 6입니다.
```

- 계약일자에서 영문 요일을 문자열 시리즈로 반환합니다.

```
>>> apt['계약일자'].dt.day_name()  # [참고] 요일을 'Monday', …, 'Sunday'로 반환합니다.
```

- 계약일자에서 한글 요일을 문자열 시리즈로 반환합니다.

```
>>> apt['계약일자'].dt.day_name(locale='ko_KR')  # [참고] locale 매개변수에 'ko_KR'을 지정하면
                                                 # 요일을 '월요일', …, '일요일'로 반환합니다.
```

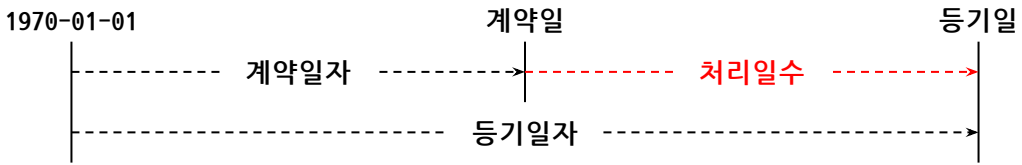
- 계약일자에서 지정한 날짜시간 포맷의 문자열을 시리즈로 반환합니다.

```
>>> apt['계약일자'].dt.strftime(date_format='%A')  # [참고] date_format 매개변수에 날짜시간 포맷을
                                                   # 문자열로 지정합니다.
```

# 날짜시간형의 간격 계산

- 등기일자와 계약일자의 간격을 계산한 처리일수를 생성합니다.

```
>>> apt['처리일수'] = apt['등기일자'] - apt['계약일자']
```




- 처리일수를 확인합니다.

```
>>> apt['처리일수']
```
*# [참고] 처리일수는 Timedelta64[ns]를 원소로 갖는 시리즈입니다.*
*Timedelta64[ns]에는 days와 seconds 속성이 정의되어 있습니다.*

- 처리일수에서 일수만 확인합니다.

```
>>> apt['처리일수'].dt.days
```
*# [참고] 처리일수는 Timedelta64[ns]인 원소를 갖는 시리즈이므로, dt 접근자의*
*days와 seconds 속성에 접근할 수 있습니다.*

# 열별 결측값 개수 확인

- apt의 셀 값별 결측값 여부를 True 또는 False로 반환합니다.

```
>>> apt.isna()  # [참고] 시리즈는 원소별 결측값 여부를 True 또는 False로 반환합니다.
                # isnull 메서드는 isna 메서드와 같은 동작을 수행합니다.
```

- apt의 열별 결측값 개수를 확인합니다.

```
>>> apt.isna().sum()  # [참고] sum 메서드는 데이터프레임의 원소(열)별 합계를 반환합니다.
                      # True는 정수 1, False는 정수 0입니다.
```

- apt의 열별 결측값 비율을 확인합니다.

```
>>> apt.isna().mean()  # [참고] mean 메서드는 데이터프레임의 원소(열)별 평균을 반환합니다.
```

- apt의 열별 결측값 백분율을 확인합니다.

```
>>> apt.isna().mean() * 100
```

# 결측값 처리: 단순대체

- 등기일자의 결측값을 특정 값으로 대체합니다.

```
>>> apt['등기일자'].fillna(value='1900-01-01')
```

*# [참고] 날짜시간형의 결측값을 '모름'과 같은 문자열로 대체할 수 있지만, 그렇게 하면 자료형이 object로 바뀌어서 날짜시간형 연산을 실행할 수 없습니다.*

*등기일자의 원본 데이터를 구할 수 없으면, 계약일자에 처리일수 평균을 더하는 방식으로 결측값을 대체하는 방법이 있습니다.*

- 처리일수 평균을 계산하고 days_avg에 할당합니다.

```
>>> days_avg = apt['처리일수'].dt.days.mean()
```

*# [참고] unit 매개변수에 날짜 간격의 단위를 지정할 수 있습니다.(기본값: 'ns')*

- 등기일자의 결측값을 계약일자에 처리일수 평균을 더한 값으로 대체합니다.

```
>>> apt['등기일자'].fillna(value=apt['계약일자'] + pd.Timedelta(days=days_avg))
```

# [참고] 결측값 삭제 방향

- 결측값이 있는 변수의 중요도에 따라 삭제 방향을 다르게 설정하는 것이 좋습니다.
  - 예제 데이터프레임에서 전용면적이 지번보다 더 의미 있는 변수라 가정하겠습니다.
    결측값이 있는 행을 모두 삭제하면 지번 때문에 결측값이 아닌 전용면적도 삭제됩니다.
  - 지번의 결측값을 적당한 값(예를 들어 '모름')으로 대체하거나, 지번 열을 삭제한 후에
    결측값이 있는 행을 삭제하면 전용면적에서 결측값이 아닌 행을 살릴 수 있습니다.

| | 아파트 | 시 | 구 | 동 | 지번 | 전용면적 | … |
|---|---|---|---|---|---|---|---|
| 0 | 레이크팰리스 | 서울특별시 | 송파구 | 잠실동 | NaN | 135.82 | … |
| 1 | 현대I-PARK | 서울특별시 | 성동구 | 성수동2가 | NaN | 116.22 | … |
| 2 | 개포자이 | 서울특별시 | 강남구 | 개포동 | 12-2 | NaN | … |

# 결측값 처리: 행 또는 열 삭제

- apt에서 결측값이 있는 행을 모두 삭제한 결과를 확인합니다.

  ```
  >>> apt.dropna()
  ```

- apt에서 결측값이 있는 열을 모두 삭제한 결과를 확인합니다.

  ```
  >>> apt.dropna(axis=1)
  ```

- apt에서 결측값이 있는 열을 모두 삭제하고 apt에 재할당합니다.

  ```
  >>> apt = apt.dropna(axis=1)
  ```

- apt의 행 개수와 열 개수를 확인합니다.

  ```
  >>> apt.shape
  ```

# 텍스트 전처리

# 텍스트 전처리

**텍스트 정제**
- 자연어 분석을 위해 수집한 텍스트 데이터셋(말뭉치)에는 오탈자와 불필요한 문자가 많습니다.
- 말뭉치에서 오탈자와 한글 띄어쓰기를 교정하고, 특수문자/기호/이모지/불용어 등을 제거합니다.

⇩

**토큰화**
- 문서를 분석 목적에 따라 토큰(문장, 단어 또는 글자 등)으로 나눕니다.
- 한글 문서는 최소 의미 단위인 형태소(단어)로 나누는 것이 일반적입니다.

⇩

**정규화**
- 단어에서 어간을 남기는 어간 추출Stemming을 수행합니다.(예: 먹었다 → 먹 + 었 + 다)
- 문맥을 고려하여 단어를 원형으로 변환하는 표제어 추출Lemmatization을 수행합니다.(예: 먹었다 → 먹다)

⇩

**품사 부착**
- 단어를 문법적인 기능에 따라 분류하는 품사를 부착합니다.(예: 먹/VV + 었/EP + 다/EF)
- 같은 단어여도 문맥에 따라 품사가 다를 수 있습니다.(예: 오늘/MAG + 은/JX, 오늘/NNG + 날씨/NNG)

⇩

**BoW 생성**
- 각 문서로 형태소 분석을 실행하고 일부 품사만 남긴 리스트를 모은 Bag of Words를 생성합니다.
- Bag of Words는 문서 집합에서 출현한 단어를 한 봉투에 쏟아부어 단어의 순서를 고려하지 않습니다.

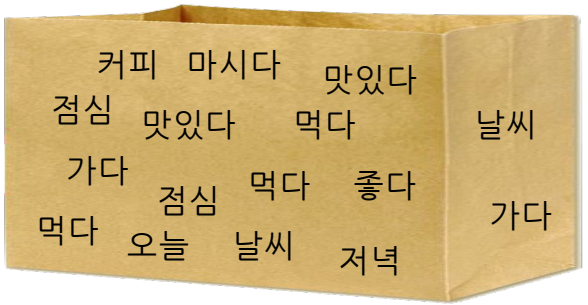
⇩

**DTM 생성**
- Bag of Words를 분석 가능한 형태인 Document-Term Matrix로 변환합니다.
- 문서-단어 행렬은 행과 열은 문서와 단어이고, 각 성분은 해당 문서에서 단어가 출현한 횟수입니다.

# [참고] 텍스트 전처리의 시각적 예시

| | |
|---|---|
| 문서1 | 나는 오늘 맛있는 점심을 먹었다. 저녁에는 뭘 먹을까? |
| 문서2 | 내일은 점심을 먹고 맛있는 커피를 마시러 갈거야. |
| 문서3 | 날씨가 좋으면 어디 가고 싶은데 오늘 날씨 왜 이런가! |

[Tokenization → Normalization → POS Tagging]




[BoW: 특성 집합 생성]

| | 가다 | 날씨 | 마시다 | 맛있다 | 먹다 | 오늘 | 저녁 | 점심 | 좋다 | 커피 |
|---|---|---|---|---|---|---|---|---|---|---|
| 문서1 | 0 | 0 | 0 | 1 | 2 | 0 | 1 | 1 | 0 | 0 |
| 문서2 | 1 | 0 | 1 | 1 | 1 | 0 | 0 | 1 | 0 | 1 |
| 문서3 | 1 | 2 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 |

[DTM: 특성 벡터 → 특성 행렬]

# 필요한 모듈 임포트

- 필요한 모듈을 임포트합니다.

```
>>> import os

>>> import numpy as np

>>> import pandas as pd

>>> import ko_spelling_et5
```

*# [참고] ko_spelling_et5.py 파일은 현재 작업 경로에 포함되어 있어야 임포트할 수 있습니다.*
*위 코드를 처음 실행하면 Hugging Face Hub에서 j5ng/et5-typos-corrector 모델을 Downloads 폴더에 내려받습니다.*

*ko_spelling_et5 모듈을 임포트하기 전에 transformers<4.40, torch, protobuf, sentencepiece 라이브러리를 미리 설치해야 합니다.*

*[주의] 현재 설치된 numpy 버전이 2 이상이면 1 버전으로 컴파일된 모듈은 동작하지 않는다는 안내문을 출력합니다.*
*하지만 이번 실습에 필요한 언어 모델과는 관계 없는 내용이므로 numpy를 2 미만 버전으로 변경하지 않아도 됩니다.*

# 작업 경로 확인 및 변경

- 현재 작업 경로를 확인합니다.

  ```
  >>> os.getcwd()
  ```

- data 폴더로 작업 경로를 변경합니다.

  ```
  >>> os.chdir(path='../data')
  ```

- 현재 작업 경로에 있는 폴더명과 파일명을 확인합니다.

  ```
  >>> os.listdir()
  ```

# 실습 데이터셋 준비

- xlsx 파일을 읽고 데이터프레임을 생성합니다.

```
>>> df = pd.read_excel(io='Galaxy_Book_Review_20250131.xlsx')
```

- df의 처음 5행을 확인합니다.

```
>>> df.head()  # [참고] date는 리뷰 작성일자, star는 리뷰어가 부여한 별점, like는 다른 사용자들의 좋아요 점수,
              # text는 상품 리뷰(텍스트)입니다. text의 원소는 텍스트 전처리에서 문서로 표현할 수 있습니다.
```

- df의 정보를 확인합니다.

```
>>> df.info()  # [참고] df는 13549행, 4열인 데이터프레임입니다.
               # date의 자료형은 datetime64[ns], star와 like는 int64, 그리고 text는 object입니다.
```

- 연도별 리뷰 건수를 확인합니다.

```
>>> df['date'].dt.year.value_counts().sort_index()
```

# 텍스트 정제

- 자연어 연구 및 특성 분석을 위해 수집한 텍스트 데이터셋을 말뭉치corpus라고 합니다.
- 텍스트 정제는 말뭉치의 품질을 향상시켜 언어 모델의 훈련 속도와 성능을 개선하는 과정이며, 주요 작업은 다음과 같습니다.
    - 전화번호, 홈페이지, 이메일, 생년월일, 계좌번호 등의 개인정보를 식별하여 삭제합니다.
    - 한글 자모(초성 · 중성 · 종성이 분리된 형태)와 불필요한 기호 및 이모지를 제거합니다.
    - 알파벳을 소문자 또는 대문자로 통일합니다. *# [참고] 일반명사와 고유명사를 구분합니다. 예를 들어 an apple과 Apple은 서로 다른 단어입니다.*
    - 한글 맞춤법 검사를 수행하여 오탈자 및 띄어쓰기를 교정합니다.

*# [참고] 정규 표현식을 사용하면 특정 패턴에 일치하는 텍스트를 효율적으로 추출, 삭제, 변경할 수 있으므로 텍스트 전처리 작업을 쉽고 빠르게 수행할 수 있습니다.*

# 불필요한 문자/기호 제거 및 알파벳 통일

- 정규 표현식 패턴을 설정합니다.

```
>>> pattern = '[^가-힣A-Za-z0-9]+' # [참고] 한글 완성형(11,172), 알파벳 대문자(26), 알파벳 소문자(26), 숫자(10)를 제외하는 정규 표현식 패턴입니다.
```

- text 열에서 정규 표현식 패턴을 한 칸 공백으로 변경하고 text 열에 재할당합니다.

```
>>> df['text'] = df['text'].str.replace(pat=pattern, repl=' ', regex=True)
```

- text 열에서 모든 알파벳을 대문자로 변경하고 text 열에 재할당합니다.

```
>>> df['text'] = df['text'].str.upper() # [참고] str.lower 메서드는 소문자로 변경합니다.
```

- df의 처음 5행을 확인합니다.

```
>>> df.head()
```

# 한글 맞춤법 검사기 비교

| 구분 | 상세 내용 | 비용 | 속도 | 정확도 | 코딩 복잡도 |
|---|---|---|---|---|---|
| 맞춤법 검사기 | - 네이버에서 제공하는 맞춤법 검사기입니다.<br>- 웹 크롤러를 직접 개발하여 사용합니다. | 무료 | 느림 | 가장 낮음 | 낮음 |
| py-hanspell-aideer | - 네이버 맞춤법 검사기를 파이썬으로 구현한 패키지입니다. | 무료 | 느림 | 가장 낮음 | 가장 낮음 |
| HuggingFace Hub (Text2Text Gen) | - et5-typos-corrector 모델을 사용합니다.<br>(ETRI-et5 기반 한글 구어체 맞춤법 교정기) | 무료 | 느림 | 좋음 | 보통 |
| | - kogrammar-base 모델을 사용합니다.<br>(국립국어원 맞춤법 교정 말뭉치) | 무료 | 매우 느림 | 낮음 | 보통 |
| OpenAI API (gpt-4) | - OpenAI에서 제공하는 gpt-4o, gpt-4o mini 등 다양한 모델을 사용합니다.(API Key 필요) | 유료 | 보통 | 가장 좋음 | 보통 |

*# [참고] https://hds.ai.kr/korean-spelling-et5*

# 한글 맞춤법 검사

- 하나의 텍스트로 한글 맞춤법 검사를 수행하고 corrected에 할당합니다.

```
>>> corrected = ko_spelling_et5.correct(text='복리후생으로워라벨부터챙기자')
```

- corrected(한글 맞춤법 검사 결과)를 확인합니다.

```
>>> corrected  # [참고] 파인 튜닝을 통해 오탈자 교정 품질을 향상시킬 수 있습니다.
               # https://hds.ai.kr/transformer-finetuning
```

- 한글 맞춤법 검사 대상인 여러 텍스트를 리스트로 생성합니다.

```
>>> texts = ['지인짜 감사하빈다', '빠른 배송조아여', '증말 마시따', '고나리자에게 문의']
```

- 여러 텍스트로 한글 맞춤법 검사를 수행한 결과를 확인합니다.

```
>>> ko_spelling_et5.correct(text=texts)  # [참고] text 매개변수에 문자열을 원소로 갖는 리스트, 배열
                                         # 또는 시리즈를 지정하면 한글 맞춤법 검사를 일괄 수행합니다.
```

# 한글 맞춤법 검사(계속)

- 문서 집합의 처음 10개 텍스트로 한글 맞춤법 검사를 수행한 결과를 확인합니다.

```
>>> ko_spelling_et5.correct(text=df['text'].head(n=10)) # [참고] 원본 문서에 포함된 글자수가 많을수록 교정 시간이 오래 걸립니다.
```

- 한글 맞춤법 검사 전 원본 문서와 비교합니다.

```
>>> df['text'].head(n=10) # [참고] 오탈자 및 띄어쓰기 교정 결과에 따라 필요시 수작업으로 변경합니다.
```

- 한글 맞춤법 교정을 완료한 xlsx 파일을 읽고 데이터프레임을 생성합니다.

```
>>> df = pd.read_excel(io='Galaxy_Book_Review_20250131_Corrected.xlsx')
```

- df의 처음 10행을 확인합니다.

```
>>> df.head(n=10)
```

# 형태소 분석

- 형태소morpheme는 더 이상 분리할 수 없는 최소 의미 단위이며, 텍스트를 형태소로 나누는 것을 형태소 분석이라고 합니다. *# [참고] NLP에서 텍스트를 분석할 수 있도록 나눈 최소 단위를 token이라고 하므로 형태소 분석을 영어로 morphological analysis 또는 tokenization이라고 합니다.*
  - 예를 들어 '가방'은 '가'와 '방'으로 나눌 수 없으므로 하나의 형태소로 간주됩니다.
- 형태소 분석기는 내장 형태소 사전과 불용어 사전을 참조하여 형태소 분석을 수행합니다.
  - 내장 형태소 사전에 포함되지 않은 단어는 정확하게 분리되지 않을 가능성이 있습니다.
  - 신조어나 복합명사는 사용자 사전에 추가하거나 수작업으로 정제하는 과정이 필요합니다.
- 형태소 분석을 통해 각 형태소마다 품사를 부착할 수 있으며, 분석 목적에 따라 필요한 품사(예를 들어 명사, 동사, 형용사 등)만 선택하여 활용할 수 있습니다.

*# 출처: https://github.com/bab2min/kiwipiepy*

# [참고] 품사의 종류

| 대분류 | 태그 | 설명 |
|---|---|---|
| 체언(N) | NNG | 일반명사 |
| | NNP | 고유명사 |
| | NNB | 의존명사 |
| | NR | 수사 |
| | NP | 대명사 |
| 용언(V) | VV | 동사 |
| | VA | 형용사 |
| | VX | 보조용언 |
| | VCP | 긍정지시사(이다) |
| | VCN | 부정지시사(아니다) |
| 관형사 | MM | 관형사 |
| 부사(MA) | MAG | 일반부사 |
| | MAJ | 접속부사 |
| 감탄사 | IC | 감탄사 |

| 대분류 | 태그 | 설명 |
|---|---|---|
| 조사(J) | JKS | 주격조사 |
| | JKC | 보격조사 |
| | JKG | 관형격조사 |
| | JKO | 목적격조사 |
| | JKB | 부사격조사 |
| | JKV | 호격조사 |
| | JKQ | 인용격조사 |
| | JX | 보조사 |
| | JC | 접속조사 |
| 어미(E) | EP | 선어말어미 |
| | EF | 종결어미 |
| | EC | 연결어미 |
| | ETN | 명사형 전성어미 |
| | ETM | 관형형 전성어미 |

# [참고] 품사의 종류(계속)

| 대분류 | 태그 | 설명 |
|---|---|---|
| 접두사 | XPN | 체언 접두사 |
| 접미사(XS) | XSN | 명사 파생접미사 |
| | XSV | 동사 파생접미사 |
| | XSA | 형용사 파생접미사 |
| | XSM | 부사 파생접미사 |
| 어근 | XR | 어근 |
| 부호, 외국어, 특수문자(S) | SF | 종결부호(.!?) |
| | SP | 구분부호(,/:;) |
| | SS | 인용부호 및 괄호 |
| | SSO | SS 중 여는 부호 |
| | SSC | SS 중 닫는 부호 |
| | SE | 줄임표(...) |
| | SO | 붙임표(-~) |
| | SW | 기타 특수문자 |

| 대분류 | 태그 | 설명 |
|---|---|---|
| 부호, 외국어, 특수문자(S) | SL | 알파벳(A-Za-z) |
| | SH | 한자 |
| | SN | 숫자(0~9) |
| | SB | 순서 있는 글머리 |
| 분석 불능 | UN | 분석 불능 |
| 웹(W) | W_URL | URL 주소 |
| | W_EMAIL | 이메일 주소 |
| | W_HASHTAG | 해시태그(#abcd) |
| | W_MENTION | 멘션(@abcd) |
| | W_SERIAL | 일련번호(전화번호, 통장번호 등) |
| | W_EMOJI | 이모지 |
| 기타 | Z_CODA | 덧붙은 받침 |
| | Z_SIOT | 사이시옷 |
| | USER0~4 | 사용자 정의 태그 |

# 형태소 분석 실행

- 필요한 모듈을 임포트합니다.

```
>>> from kiwipiepy import Kiwi
```

- 한글 형태소 분석기 객체를 생성합니다.

```
>>> kiwi = Kiwi(model_type='cong'))  # [참고] model_type 매개변수에 'none'(기본값), 'cong', 'cong-global' 중 하나를 지정합니다. 'cong' 모델은 신경망 기반의 언어 모델이며, 일반적으로 빠르고 정확한 결과를 제공합니다.
```

- corrected(한글 맞춤법 검사 결과)로 형태소 분석을 실행한 결과를 확인합니다.

```
>>> kiwi.tokenize(text=corrected, typos='basic')
```

*# [참고] kiwi.tokenize 메서드의 text 매개변수에 리스트를 지정하면 map 객체를 반환하므로 리스트로 변환해야 결과를 확인할 수 있습니다.*

*typos 매개변수에 'basic'(기본 오타), 'continual'(연철 오타), 'lengthening'(장음화 오류) 중 하나를 지정합니다. 연철 오타는 받침을 다음 초성(ㅇ/ㅎ)에 이어적은 경우, 장음화 오류는 한 음절을 여러 음절로 늘려 적은 경우입니다.*

# 불용어 삭제

- 필요한 모듈을 임포트합니다.

```
>>> from kiwipiepy.utils import Stopwords
```

- 내장 불용어 객체를 생성합니다.

```
>>> stopwords = Stopwords()
```

- 내장 불용어 목록을 확인합니다.

```
>>> stopwords.stopwords
```

*# [참고] 내장 불용어 목록은 불용어와 품사를 튜플로 묶은 원소를 갖는 집합입니다.*
*사용자 불용어를 추가할 때 add 메서드, 삭제할 때 remove 메서드를 사용합니다.*

- 형태소 분석을 실행하고 불용어를 삭제한 결과를 확인합니다.

```
>>> kiwi.tokenize(text=corrected, typos='basic', stopwords=stopwords)
```

# 사용자 사전 추가

- 사용자 사전에 새로운 단어를 추가합니다.

```
>>> kiwi.add_user_word(word='복리후생', tag='NNG')
```

- 사용자 사전을 적용하여 형태소 분석을 실행한 결과를 tokens에 할당합니다.

```
>>> tokens = kiwi.tokenize(text=corrected, typos='basic', stopwords=stopwords)
```

- tokens(형태소 분석 결과)를 확인합니다.

```
>>> tokens
```

- 첫 번째 형태소의 단어와 품사를 확인합니다.

```
>>> tokens[0].form, tokens[0].tag # [참고] 두 개 이상의 객체를 콤마로 나열하면 튜플로 묶습니다.
```

# 품사 선택

- 형태소 분석 결과에서 선택할 품사(용언과 체언) 목록을 리스트로 생성합니다.

```
>>> verb_tags = ['VV', 'VV-R', 'VV-I', 'VA', 'VA-R', 'VA-I']

>>> noun_tags = ['NNG', 'NNP']

# [참고] 용언에서 '-R'은 규칙 활용, '-I'는 불규칙 활용을 의미합니다.
  예를 들어 '묻다'는 '물어보다'처럼 어간의 받침 ㄷ이 ㄹ로 바뀌므로 ㄷ 불규칙 활용 동사입니다.
```

- 품사가 용언과 체언인 형태소만 선택하고, 품사가 용언일 때 종결어미 '다'를 결합합니다.

```
>>> [token.form + '다' if token.tag in verb_tags else token.form

     for token in tokens if token.tag in verb_tags + noun_tags]
```

# 형태소 분석 함수 생성

- 문서를 입력받아 형태소 분석을 실행하고 리스트로 반환하는 함수를 생성합니다.

```
>>> def tokenizer(text):
        tokens = kiwi.tokenize(text=text, typos='basic', stopwords=stopwords)
        return [token.form + '다' if token.tag in verb_tags else token.form
                for token in tokens if token.tag in verb_tags + noun_tags]
```

*# [주의] 사용자 정의 함수 내부에 없는 클래스(kiwi)와 객체(verb_tags, noun_tags)는 전역 변수 목록에서 참고합니다.*
*  전역 변수 목록에 해당 변수가 없으면 사용자 정의 함수를 실행했을 때 에러를 일으킵니다.*

- 형태소 분석기를 실행한 결과를 확인합니다.

```
>>> tokenizer(text=corrected)
```

# BoW 생성: 문서 집합으로 형태소 분석 실행

- 필요한 모듈을 임포트합니다.

```
>>> from tqdm.notebook import tqdm
```

- 형태소 분석 결과를 저장할 빈 리스트를 생성합니다.

```
>>> morphs = []  # [주의] morphs는 말뭉치의 각 문서를 형태소 분석하여 하나의 주머니에 담은 특성 집합입니다.
```

- 문서 집합에서 각 문서를 형태소로 나누고 일부 품사를 남긴 morphs를 생성합니다.

```
>>> for document in tqdm(df['corrected']):
        tokens = tokenizer(text=document)
        morphs.append(tokens)
```

# BoW 확인: 사용자 사전에 추가할 단어 탐색

- 한글 맞춤법 검사를 실행한 문서와 형태소 분석 결과를 비교합니다.

```
>>> for i in range(5):
        print(f'{i}: {df["corrected"].iloc[i]}') # 한글 맞춤법 검사를 실행한 문서를 출력합니다.
        print(f'{i}: {morphs[i]}', end='\n\n') # 문서를 형태소로 나눈 결과를 출력합니다.
        # [참고] print 함수의 end 매개변수에 전달되는 인수의 기본값은 '\n'입니다.
```

# 사용자 사전에 추가

- 사용자 사전에 추가할 단어와 품사 목록을 각각 리스트로 생성합니다.

```
>>> words = ['사무용', '저렴', '재구매', '사은 품', '구매 확정']
>>> tags = ['NNG'] * 5
```

- 반복문으로 사용자 사전에 새로운 단어를 추가합니다.

```
>>> for word, tag in zip(words, tags):  # [참고] zip 함수는 반복 가능한 객체에서 같은 인덱스 원소들을 튜플로 묶어서 반환합니다.
        kiwi.add_user_word(word=word, tag=tag)
```

# BoW 생성: 사용자 사전 추가 후 형태소 분석 실행

- 형태소 분석 결과를 저장할 빈 리스트를 생성합니다.

```
>>> morphs = [] # [참고] morphs는 문서 집합에서 출현한 모든 형태소를 한 주머니에 담은 특성 집합이 됩니다.
```

- 사용자 사전을 적용하여 문서 집합으로 형태소 분석을 실행하고 morphs를 생성합니다.

```
>>> for document in tqdm(df['corrected']):
        tokens = tokenizer(text=document)
        morphs.append(tokens)
```

- morphs(형태소로 나눈 문서 집합)의 처음 5개 원소를 확인합니다.

```
>>> [print(morph) for morph in morphs[:5]] # [참고] print 함수는 실행 결과 None을 반환합니다.
```

# [참고] 사용자 사전용 텍스트 파일 생성 및 읽기

- 사용자 사전 파일명을 생성합니다.

```
>>> user_dictionary = 'user_dict.txt'
```

- 사용자 사전을 텍스트 파일로 저장합니다.

```
>>> with open(file=user_dictionary, mode='w') as f:  # [참고] 'w'는 쓰기, 'a'는 추가, 'r'은 읽기 모드로 파일을 엽니다.
        for word, tag in zip(words, tags):
            f.write(f'{word}\t{tag}\n') # [참고] 단어와 품사 사이에 구분자로 탭을 추가하고 마지막에 개행문자를 추가합니다.
```

- 사용자 사전에 등록할 텍스트 파일을 읽습니다.

```
>>> kiwi.load_user_dictionary(dict_path=user_dictionary)
```

# n-gram

- n-gram은 텍스트(BoW 벡터)에서 연속하는 n개의 단어를 하나의 단위로 묶은 것입니다.
  - n의 크기에 따라 monogram(n=1), bigram(n=2), trigram(n=3) 등으로 구분합니다.
- 도입 초기에는 통계적 언어 모델링에서 언어의 패턴과 확률을 추정하기 위해 사용되었습니다.
  - 어떤 단어들이 자주 함께 등장하는지 파악할 수 있습니다.
  - n-1개 단어만 고려하여 n번째 단어의 출현 확률을 추정할 수 있습니다.
- 딥러닝 언어 모델이 등장했지만 n-gram은 여전히 유용하게 사용되고 있습니다.
  - 자연어 처리에서 문서를 벡터화할 때 연속하는 단어의 패턴으로 특징을 추출합니다.
  - 문서 집합에서 오탈자 및 띄어쓰기 오류를 확인할 수 있습니다.(사용자 사전에 추가)

# [참고] bigram의 시각적 예시

'조직문화가 임직원들의 사기에 영향을 끼친다'

tokenizing

'조직' '문화' '임직원' '사기' '영향' '끼치다'

bigram

| | | | | | |
|---|---|---|---|---|---|
| '조직' | '문화' | '임직원' | '사기' | '영향' | '끼치다' |
| '조직' | '문화' | '임직원' | '사기' | '영향' | '끼치다' |
| '조직' | '문화' | '임직원' | '사기' | '영향' | '끼치다' |
| '조직' | '문화' | '임직원' | '사기' | '영향' | '끼치다' |
| '조직' | '문화' | '임직원' | '사기' | '영향' | '끼치다' |

**bigram**

('조직', '문화')

('문화', '임직원')

('임직원', '사기')

('사기', '영향')

('영향', '끼치다')

# 바이그램 생성

- 필요한 모듈을 임포트합니다.

```
>>> from nltk import bigrams
```

- morphs의 첫 번째 원소에서 연속하는 두 단어를 튜플로 묶은 bigram을 생성합니다.

```
>>> bigram = bigrams(sequence=morphs[0])  # [참고] sequence 매개변수에 리스트 대신 문자열을 지정하면 연속하는 두 글자를 튜플로 묶습니다.
```

- bigram을 리스트로 변환한 결과를 확인합니다.

```
>>> list(bigram)  # [참고] bigram은 generator이므로 리스트로 변환해야 원소를 확인할 수 있습니다. morphs의 첫 번째 원소에서 연속하는 두 단어를 튜플로 묶었습니다.
```

- morphs의 첫 번째 원소를 출력하고 위 코드 결과와 비교합니다.

```
>>> print(morphs[0])
```

# 바이그램 생성(계속)

- morphs의 전체 원소에서 생성한 bigram을 저장할 빈 리스트를 생성합니다.

```
>>> bigram_list = []
```

- morphs에서 연속하는 두 단어를 묶은 튜플을 bigram_list에 결합합니다.

```
>>> for morph in morphs:
        bigram = bigrams(sequence=morph)
        bigram_list += list(bigram) # [참고] bigram_list는 bigram(튜플)을 원소로 갖는 리스트입니다.
```

- bigram_list를 시리즈로 변환하여 원소별 도수를 계산하고 처음 20개를 확인합니다.

```
>>> pd.Series(data=bigram_list).value_counts().head(n=20)
```

# 문서-단어 행렬

- 문서-단어 행렬document-term matrix은 행이 문서, 열은 문서에서 출현한 단어인 행렬입니다.
- 이 행렬의 성분은 해당 문서에서 특정 단어 t가 출현한 횟수term frequency, TF를 의미합니다.
  - 단어 t의 도수 합계가 클수록 문서 집합에서 중요한 역할을 할 가능성이 높습니다.
  - 문서-단어 행렬은 대다수 성분이 0인 희소 행렬sparse matrix의 형태인 경우가 많습니다.
- 단어 도수에 역 문서 도수inverse document frequency, IDF를 곱하면 TF-IDF 값을 계산할 수 있습니다.
  - TF-IDF를 활용하면 특정 문서에서 집중적으로 출현한 단어를 강조할 수 있습니다.
  - IDF는 단어 t가 문서 집합에서 얼마나 희귀한지를 반영하는 지표(가중치)입니다.

$$\text{IDF}\,(t) = \log\left(\frac{N}{df_t}\right)$$

$N$ : 전체 문서 개수

$df_t$ : 단어 t를 포함하는 문서 개수

# [참고] 문서-단어 행렬의 시각적 예시

| | 단어1 | 단어2 | 단어3 | 단어4 | 단어5 | 단어6 | … |
|---|---|---|---|---|---|---|---|
| 문서1 | 0 | 0 | 2 | 1 | 0 | 0 | … |
| 문서2 | 0 | 1 | 1 | 0 | 0 | 0 | … |
| 문서3 | 0 | 0 | 1 | 1 | 0 | 0 | … |
| 문서4 | 0 | 0 | 2 | 0 | 3 | 1 | … |
| 문서5 | 1 | 2 | 1 | 0 | 2 | 0 | … |
| ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋱ |

전체 문서에서 거의 출현하지 않은 단어

거의 모든 문서에서 출현하는 단어

일부 문서에서 집중적으로 출현하는 단어

# morphs 원소 전처리

- morphs의 처음 5개 원소를 확인합니다.

```
>>> morphs[:5]
```

*# [참고] BoW로 문서-단어 행렬을 생성하려면 BoW의 원소인 리스트를 문자열로 변경해야 하는데 만약 형태소에 공백이 있으면 문서-단어 행렬을 생성하는 과정에서 두 단어로 분리됩니다. 사용자 사전에 추가한 단어('사은 품', '구매 확정')에 공백이 있으므로 공백을 제거합니다.*

- 형태소(단어)에 있는 공백을 제거하고 morphs에 재할당합니다.

```
>>> morphs = [[token.replace(' ', '') for token in morph] for morph in morphs]
```

- morphs의 원소(리스트)를 하나의 문자열로 결합하고 corpus에 할당합니다.

```
>>> corpus = [' '.join(morph) for morph in morphs]
```

*# [참고] join은 문자열 메서드이며, join 메서드 앞 문자열은 morph(리스트) 원소(문자열) 사이에 추가하는 구분자입니다.*

- corpus의 처음 5개 원소를 확인합니다.

```
>>> corpus[:5]
```

*# [참고] 문서-단어 행렬을 생성할 때 한 칸 공백으로 구분한 문자열을 지정해야 합니다.*

# 문서-단어 행렬 생성

- 필요한 모듈을 임포트합니다.

```
# [참고] CountVectorizer 클래스는 TF를 성분으로 갖는 행렬을 반환합니다.
  TF-IDF를 성분으로 갖는 행렬을 생성하려면 TfidfVectorizer를 임포트합니다.
```

```
>>> from sklearn.feature_extraction.text import CountVectorizer
```

- 문서별 단어 도수를 계산하는 객체를 생성합니다.

```
>>> count_vectorizer = CountVectorizer(min_df=0.01)
```

```
# [참고] min_df 매개변수에 0~1 범위의 실수를 지정하면 문서 집합에서 출현한 비율이 낮은 단어를 제거합니다.
  실수 대신 출현 도수의 최솟값을 정수로 지정할 수 있습니다.(기본값: 1)

  max_features 매개변수에 정수를 지정하면 고빈도 단어만 남깁니다.(기본값: None)
  binary 매개변수에 True를 지정하면 해당 문서에 단어의 출현 여부를 0 또는 1로 표시합니다.(기본값: False)
```

- 단어 도수를 성분으로 갖는 문서-단어 행렬을 생성합니다.

```
>>> dtm = count_vectorizer.fit_transform(raw_documents=corpus).toarray()
```

# 문서-단어 행렬 확인

- dtm을 데이터프레임으로 변환합니다.

```
>>> dtm = pd.DataFrame(data=dtm, columns=count_vectorizer.get_feature_names_out())
```

- dtm의 처음 10행을 확인합니다.

*# [참고] count_vectorizer.get_feature_names_out은 전체 문서 집합에서 출현한 단어 집합을 오름차순 정렬한 결과입니다.*

```
>>> dtm.head(n=10)
```

- dtm의 행 개수와 열 개수를 확인합니다.

```
>>> dtm.shape # [참고] 문서의 글자수가 많을수록 문서-단어 행렬의 열 개수는 기하급수적으로 증가합니다.
```

- dtm의 단어별 도수 합계를 계산하고 상위 20개 단어를 확인합니다.

```
>>> dtm.sum().sort_values(ascending=False).head(n=20) # [참고] 문서 집합에서 가장 많이 출현한 고빈도 단어를 확인할 수 있습니다.
```

# 문서 간 유사도

코사인 거리

# [퀴즈] 다음 중 서로 유사한 고객은 누구일까요?

- 결혼정보업체에서 구축하는 데이터베이스의 예시입니다.

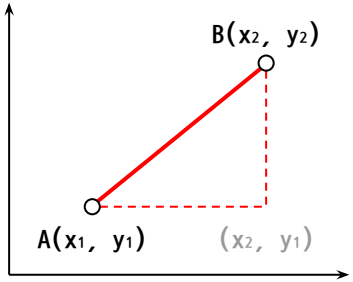
| ID | 성별 | 학력 | 나이 | 키 | 몸무게 | 연봉 | … |
|---|---|---|---|---|---|---|---|
| A | 0 | 1 | 35 | 183 | 78 | 2,900 | … |
| B | 0 | 2 | 39 | 179 | 82 | 8,500 | … |
| C | 1 | 3 | 42 | 165 | 50 | 9,800 | … |

*# [참고] 성별은 0(남자)과 1(여자)로 인코딩하였고, 학력은 1(고졸), 2(대졸), 3(대학원졸)로 인코딩한 것입니다.*

- 세 명의 고객 중 특성이 가장 유사한 2명을 선택한 조합은 어느 것일까요?

① A와 B　　② B와 C　　③ C와 A

# [참고] 유클리드 거리의 시각적 이해

| 1차원 데이터셋 | 2차원 데이터셋 |
| --- | --- |
|  |  |
| d(A, B) $= x_2 - x_1$ | d(A, B) $= \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ |

*# [참고] 유클리드 거리는 두 점 간 직선의 거리이며 피타고라스 정리로 계산합니다.*
*데이터셋의 차원(벡터 A와 B의 성분 개수)이 증가할수록 루트 안에 각 차원의 거리 제곱이 늘어납니다.*

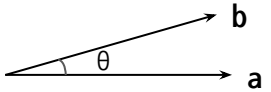
# 문장 벡터 간 유사도 계산

- 문서-단어 행렬은 문서에 출현한 단어의 카운트 벡터를 모은 말뭉치입니다.
  - 문서-단어 행렬은 n개의 문서를 p차원 벡터로 표현한 것을 행렬로 결합한 형태입니다.
- 문서 간 유사도를 측정하면 어떤 문서와 가장 유사한 문서를 말뭉치에서 검색할 수 있습니다. 문서 간 유사도의 척도로 코사인 거리를 가장 많이 사용합니다.
  - 두 문서 간 단어 도수의 크기가 유사도에 반영되는 유클리드 거리는 사용하지 않습니다.
- 코사인 거리는 두 벡터가 이루는 사잇각( θ )에 대한 코사인 값(코사인 유사도)을 이용합니다.
  - 코사인 유사도는 두 문서 간 방향성을 비교한다는 특징이 있습니다.
  - 코사인 거리는 '1 - 코사인 유사도'로 계산합니다.

*# [참고] 코사인 거리는 두 벡터의 유사도가 클수록(사잇각이 작을수록) 값이 작아집니다.*

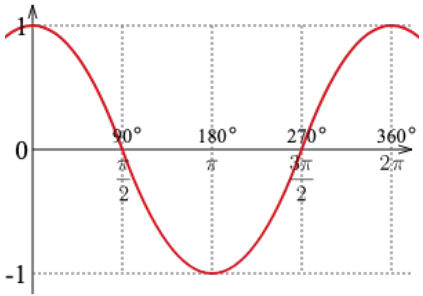
# [참고] 코사인 유사도와 코사인 거리

- 코사인 유사도는 두 벡터의 사잇각( θ )에 대한 코사인 값을 이용하여 유사도를 측정합니다.
  - 두 벡터의 내적을 각 벡터의 크기로 나누면 코사인 유사도를 계산할 수 있습니다.




$$\cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{||\mathbf{a}|| \times ||\mathbf{b}||} \qquad \mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{n} \mathbf{a}_i \times \mathbf{b}_i \qquad ||\mathbf{a}|| = \sqrt{\sum_{i=1}^{n} \mathbf{a}_i^2} \qquad ||\mathbf{b}|| = \sqrt{\sum_{i=1}^{n} \mathbf{b}_i^2}$$

  - 코사인 거리를 '1 - 코사인 유사도'로 정의하면 다음과 같은 관계를 갖습니다.




| 사잇각( θ ) | 코사인 유사도 | 코사인 거리 |
|---|---|---|
| 0 ° | 1 | 0 |
| 90 ° | 0 | 1 |
| 180 ° | -1 | 2 |

*# 출처 : https://www.calculat.org/kr/삼각함수/코사인.html*

# [참고] 문서 간 코사인 거리 행렬의 시각적 표현

| | 문서1 | 문서2 | 문서3 | 문서4 | 문서5 | 문서6 | 문서7 | 문서8 | … |
|---|---|---|---|---|---|---|---|---|---|
| 문서1 | 0.00 | 0.05 | 0.22 | 0.16 | 0.56 | 0.32 | 0.27 | 0.13 | … |
| 문서2 | 0.05 | 0.00 | 0.18 | 0.00 | 0.28 | 0.12 | 0.15 | 0.08 | … |
| 문서3 | 0.22 | 0.18 | 0.00 | 0.13 | 0.16 | 0.09 | 0.02 | 0.17 | … |
| 문서4 | 0.16 | 0.00 | 0.13 | 0.00 | 0.43 | 0.37 | 0.19 | 0.03 | … |
| 문서5 | 0.56 | 0.28 | 0.16 | 0.43 | 0.00 | 0.19 | 0.21 | 0.16 | … |
| 문서6 | 0.32 | 0.12 | 0.09 | 0.37 | 0.19 | 0.00 | 0.45 | 0.38 | … |
| 문서7 | 0.27 | 0.15 | 0.02 | 0.19 | 0.21 | 0.45 | 0.00 | 0.12 | … |
| 문서8 | 0.13 | 0.08 | 0.17 | 0.03 | 0.16 | 0.38 | 0.12 | 0.00 | … |
| ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋱ |

# 문서 벡터 간 코사인 거리 행렬 생성

- 필요한 모듈을 임포트합니다.

```
>>> from sklearn.metrics.pairwise import cosine_distances # [참고] 코사인 유사도를 계산하려면 cosine_similarity를 임포트합니다.
```

- dtm의 행(문서) 벡터 간 코사인 거리를 2차원 배열로 반환합니다.

```
>>> cos_dist = cosine_distances(X=dtm) # [참고] 코사인 거리 행렬의 주대각 성분은 모두 0입니다.
```

- 코사인 거리 행렬의 주대각 성분을 무한대 값으로 변경합니다.

```
>>> np.fill_diagonal(a=cos_dist, val=np.inf) # [참고] np.fill_diagonal 함수는 실행 결과를 배열에 업데이트하므로 원본에 재할당하지 않습니다.
```

- 코사인 거리 행렬을 반올림하면 결과를 확인하기 좋습니다.

```
>>> cos_dist.round(3) # [참고] 코사인 거리를 반올림하여 소수점 셋째 자리까지 표시합니다.
```

# 유사도 높은 문서 확인

- 관심 있는 문서의 정수 인덱스를 설정합니다.

```
>>> idx = 0
```

- 관심 있는 문서의 코사인 거리 벡터에서 최솟값을 확인합니다.

```
>>> np.min(a=cos_dist[idx, :])
```

*# [참고] 배열 원소 중에 결측값이 있으면 최솟값 대신 nan을 반환합니다.*
*2차원 배열에서 행 또는 열을 한 개 선택하면 1차원 배열로 반환합니다.*

- 관심 있는 문서의 코사인 거리 벡터 원소를 오름차순 정렬하고 정수 인덱스를 반환합니다.

```
>>> min_indices = np.argsort(a=cos_dist[idx, :])
```

*# [참고] np.argsort 함수는 원소를 오름차순으로*
*정렬하고 인덱스를 1차원 배열로 반환합니다.*

- 관심 있는 문서와 코사인 거리가 가장 가까운 5개 문서를 확인합니다.

```
>>> df['corrected'].iloc[np.hstack(tup=(idx, min_indices[:5]))]
```

# End of Document